

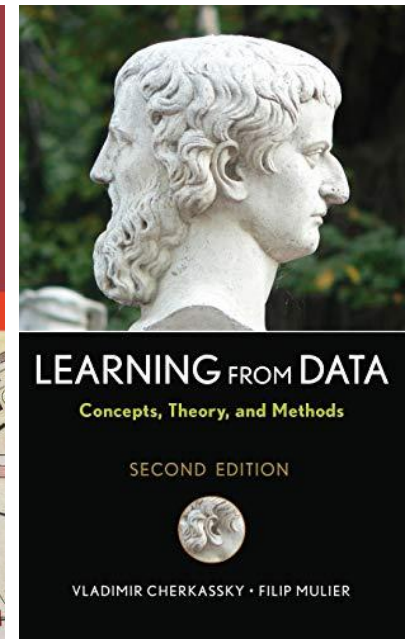
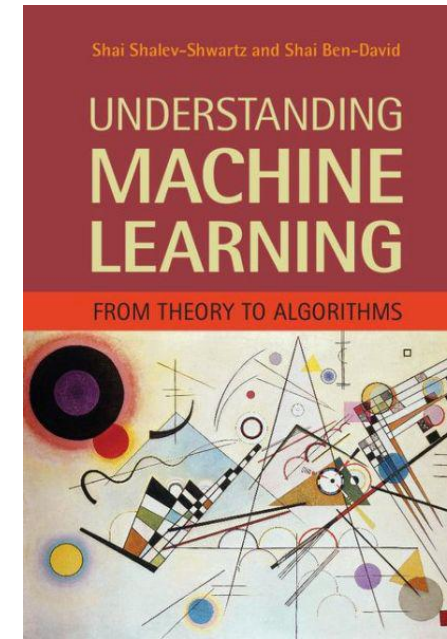
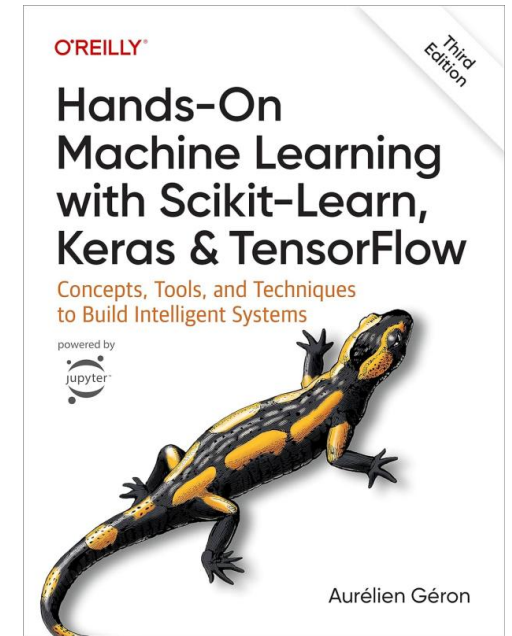
Aprendizaje Automático

Curso 2025-2026

Tema 2. Aprendizaje Supervisado



Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - **Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación**
 - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
 - Tema 4: Aprendizaje no Supervisado
 - Tema 5: Aprendizaje Profundo. Fundamentos
 - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Máquinas de Vector Soporte

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Máquinas de Vector Soporte

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición

Clasificación SVM lineal

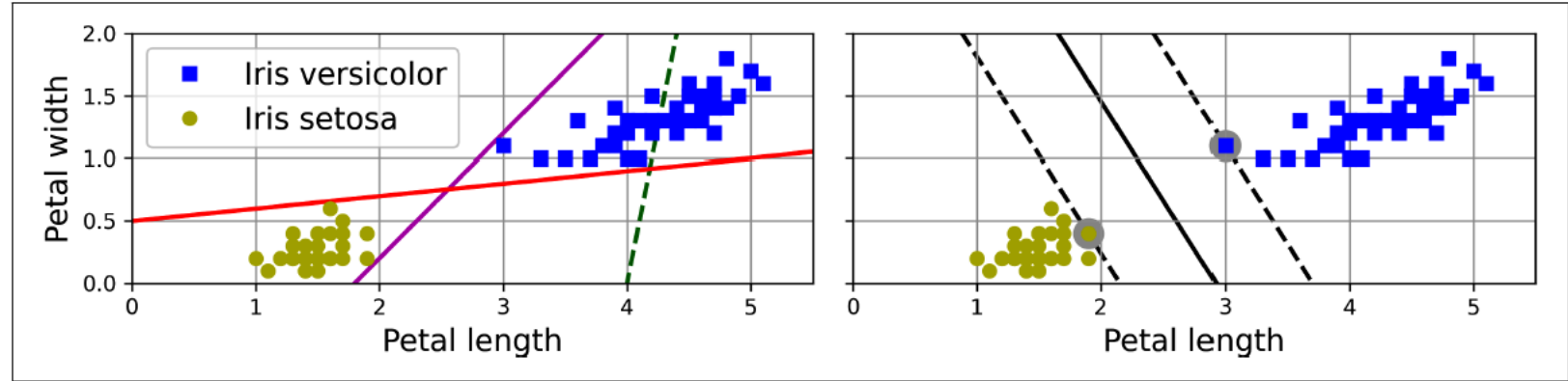


Figure 5-1. Large margin classification

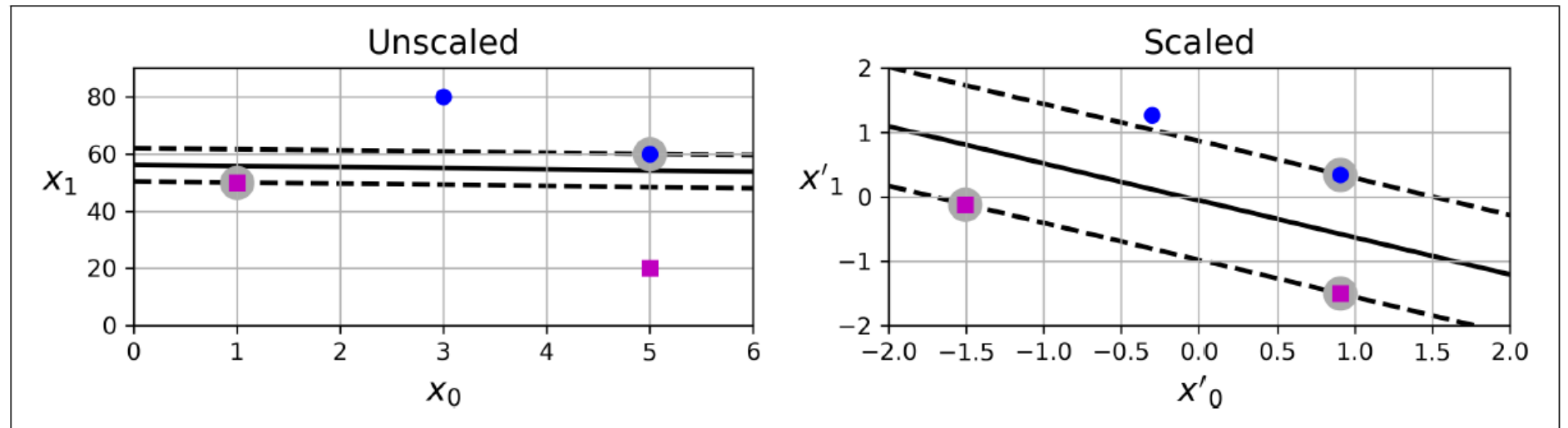
- **Hipótesis: Las dos clases son linealmente separables.**
- El gráfico de la izquierda muestra los límites de decisión de tres posibles clasificadores lineales. El modelo cuyo límite de decisión está representado por la línea discontinua es tan malo que ni siquiera separa correctamente las clases. Los otros dos modelos funcionan perfectamente con este conjunto de entrenamiento, pero sus límites de decisión se acercan tanto a las instancias que probablemente no funcionen tan bien con instancias nuevas.
- Por el contrario, la línea continua del gráfico de la derecha representa el límite de decisión de un clasificador SVM; esta línea no sólo separa las dos clases, sino que también se mantiene lo más alejada posible de las instancias de entrenamiento más cercanas. Puede pensar que un clasificador SVM se ajusta a la calle más ancha posible (representada por las líneas discontinuas paralelas) entre las clases.
- Esto se denomina *clasificación de gran margen o margen máximo*.

Funcionamiento de los clasificadores SVM lineales

- Un clasificador SVM lineal predice la clase de una nueva instancia x calculando primero la función de decisión $\theta^T x = \theta_0 x_0 + \dots + \theta_n x_n$, donde x_0 es la característica de sesgo (siempre igual a 1). Si el resultado es positivo, la clase predicha $\hat{y}$ es la clase positiva (1); en caso contrario, es la clase negativa (0). Es exactamente igual que *la regresión logística*.
- Hasta ahora, hemos utilizado la convención de poner todos los parámetros del modelo en un vector θ , incluyendo el término de sesgo θ_0 y los pesos de las características de entrada θ_1 a θ_n . Esto requería añadir una entrada de sesgo $x_0 = 1$ a todas las instancias.
- Otra convención muy común es separar el término de sesgo b (igual a θ_0) y el vector de pesos de características w (que contiene θ_1 a θ_n). En este caso, no es necesario añadir ninguna característica de sesgo a los vectores de características de entrada, y la función de decisión de la SVM lineal es igual a $w^T x + b = w_1 x_1 + \dots + w_n x_n + b$.

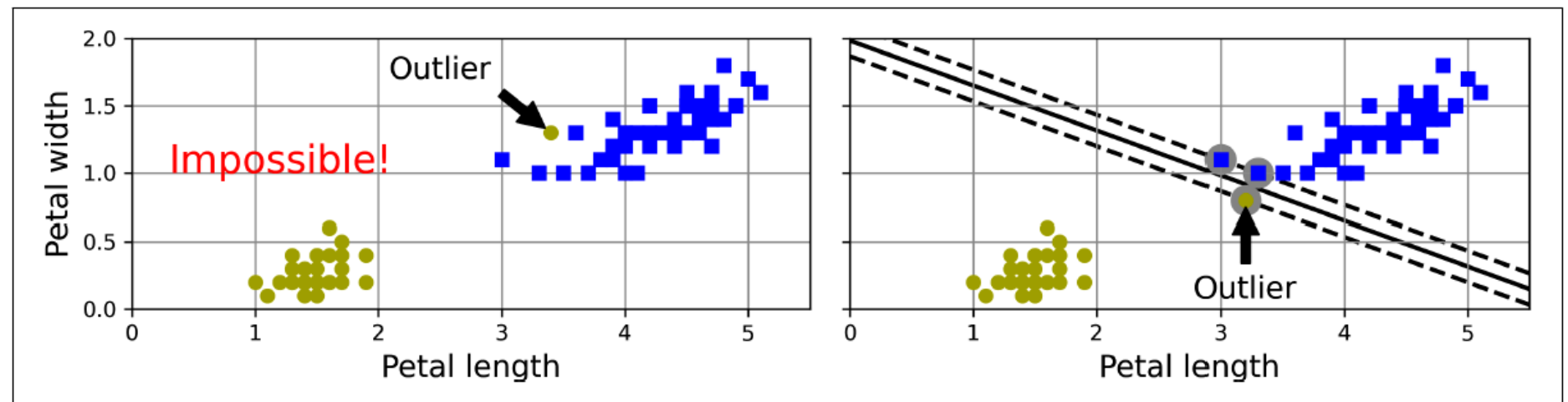
Clasificación SVM lineal

- Añadir más instancias de entrenamiento "fuera de la calle" no afectará en absoluto al límite de decisión: está totalmente determinado (o "apoyado") por las instancias situadas en el borde de la calle.
- Estas instancias de borde se denominan *vectores de soporte*.
- Las SVM son sensibles a las escalas de las características, como puede verse en esta figura.



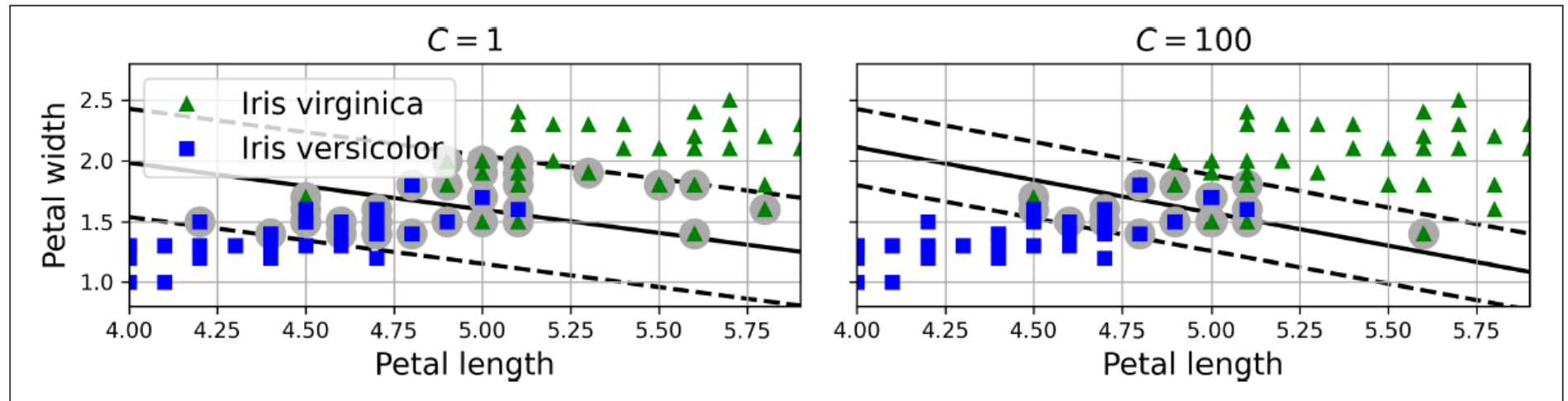
Clasificación del margen blando

- Si imponemos estrictamente que todas las instancias deben estar fuera de la calle y en el lado correcto, esto se denomina *clasificación de margen duro*.
- Dos problemas con la clasificación de márgenes duros.
 - Sólo funciona si los datos son linealmente separables.
 - Es sensible a los valores atípicos.



Clasificación del margen blando

- ▶ Para evitar estos problemas, tenemos que utilizar un modelo más flexible. El objetivo es encontrar un buen equilibrio entre mantener la calle lo más grande posible y limitar las violaciones del margen. Esto se denomina *clasificación de margen suave*.
- ▶ Al crear un modelo SVM con Scikit-Learn, puede especificar varios hiperparámetros, incluido el hiperparámetro de regularización C .



Clasificación SVM no lineal

Una **SVM (support vector machine)** es un modelo de aprendizaje que se fundamenta en la *Teoría de Aprendizaje Estadístico*.

La idea básica es encontrar un **hiperplano canónico** que **maximice el margen** del conjunto de datos de entrenamiento, esto nos garantiza una **buena capacidad de generalización**."

Representación dual de un problema

+

Funciones Kernel

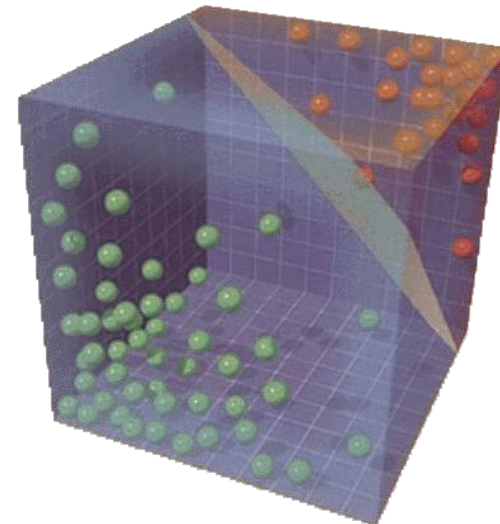
+

Teoría de Aprendizaje Estadística

+

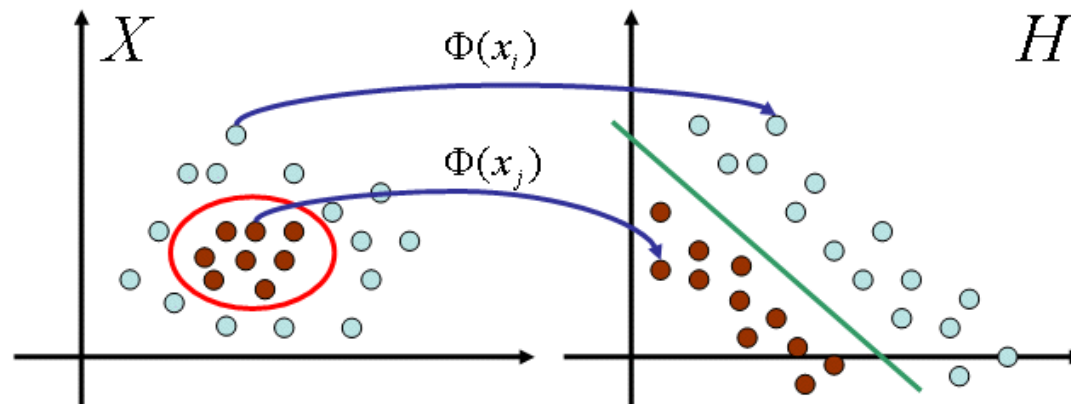
Teoría Optimización Lagrange

=



Clasificación SVM no lineal

- ▶ Una **SVM** es un **máquina de aprendizaje lineal** (requiere que los datos sean linealmente separables).
- ▶ Estos métodos explotan la información que proporciona el **producto interno (escalar)** entre los datos disponibles.
- ▶ La idea básica es:



Clasificación SVM no lineal

► Problemas de esta aproximación:

- ¿Cómo encontrar la función ϕ ?
- El espacio de características inducido H es de **alta dimensión**.
 - Problemas de cómputo y memoria.

Clasificación SVM no lineal

► Solución:

- Uso de funciones kernel.
- **Función kernel** = producto interno de dos elementos en algún espacio de características inducido (**potencialmente de gran dimensionalidad**).
- Si usamos una función kernel no hay necesidad de especificar la función ϕ .

Clasificación SVM no lineal

- Polinomial: $K(x, y) = \langle x, y \rangle^d$
- Gausiano: $K(x, y) = e^{-\|x-y\|^2 / 2\sigma}$
- Sigmoido: $K(x, y) = \tanh(\alpha \langle x, y \rangle + \beta)$

Clasificación SVM no lineal

- ▶ Para resolver el problema de optimización planteado se usa la **teoría de Lagrange**.

Minimizar

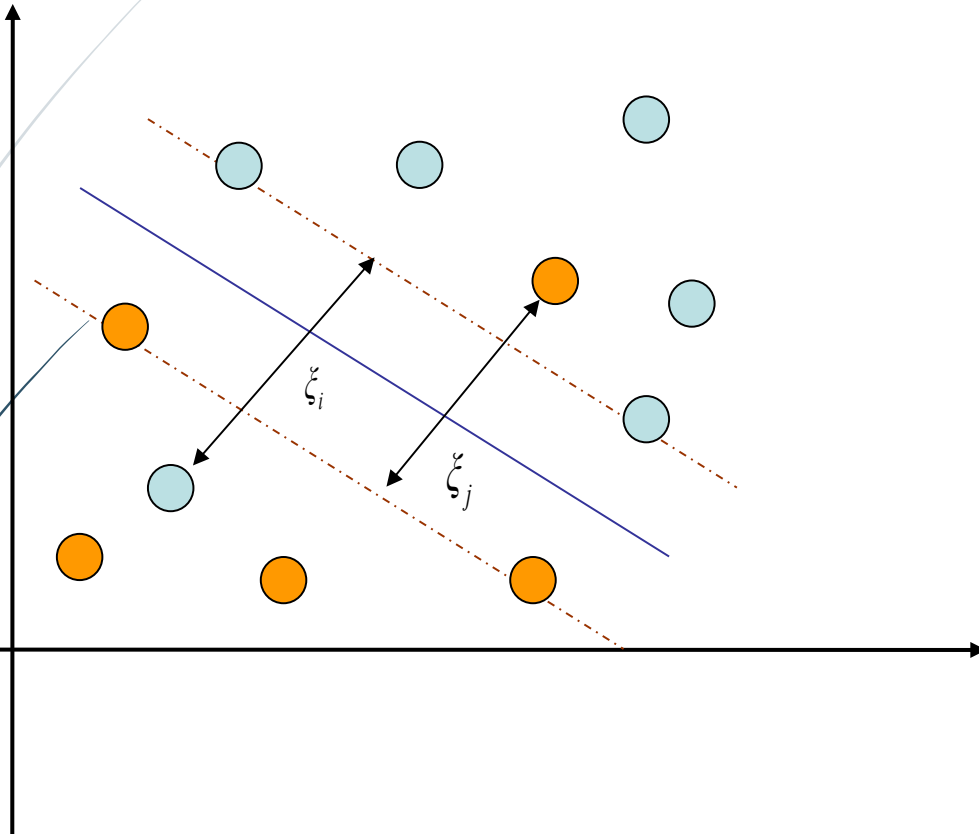
$$L(w, b) = \frac{1}{2} \langle w, w \rangle - \sum_{i=1}^l \alpha_i [y_i (\langle w, x_i \rangle + b) - 1]$$

Condicionado a

$$\alpha_i \geq 0$$

- ▶ Cada una de las variables α_i es un multiplicador de Lagrange y existe una variable por cada uno de los datos de entrada.

Clasificación SVM no lineal



$$y_i [\langle w, x_i \rangle + b] \geq 1 - \xi_i$$

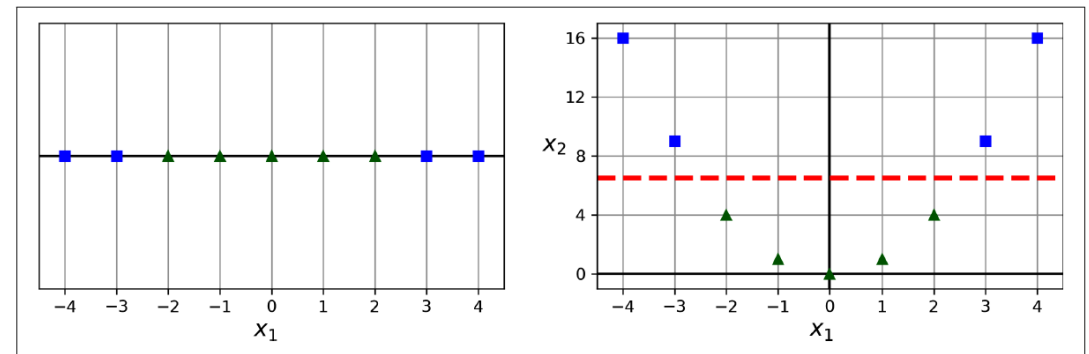


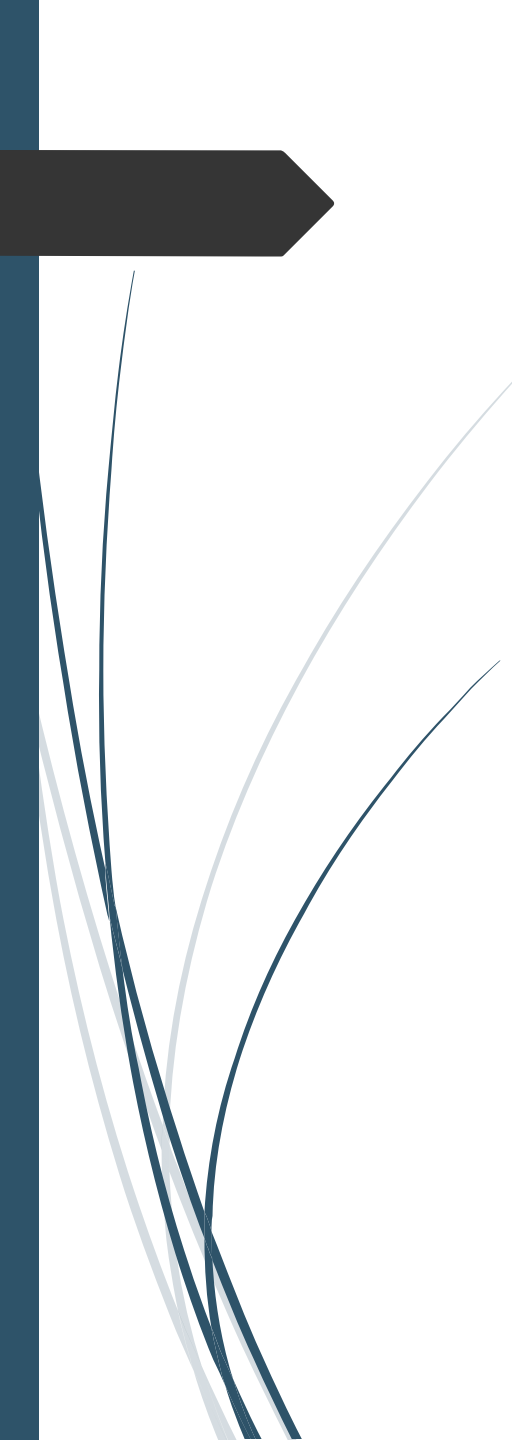
Clasificación SVM no lineal

- ▶ Originariamente el **modelo de aprendizaje basado en SVMs** fue diseñado para **problemas de clasificación binaria**.
- ▶ La extensión a multi-clases se realiza mediante combinación de clasificadores binarios (se estudia en la siguiente sección, modelos One vs One).

Clasificación SVM no lineal

- Aunque los clasificadores SVM lineales son eficientes y a menudo funcionan sorprendentemente bien, muchos conjuntos de datos ni siquiera están cerca de ser linealmente separables.
- Un método para tratar los conjuntos de datos no lineales consiste en añadir más características, como las polinómicas (como hicimos con la regresión polinómica); en algunos casos, esto puede dar lugar a un conjunto de datos linealmente separable.
- Consideremos el gráfico de la izquierda de la siguiente figura: representa un conjunto de datos simple con una sola característica, x_1 . Este conjunto de datos no es linealmente separable. Pero si se añade una segunda característica $x_2 = (x_1)^2$, el conjunto de datos 2D resultante es perfectamente separable linealmente.



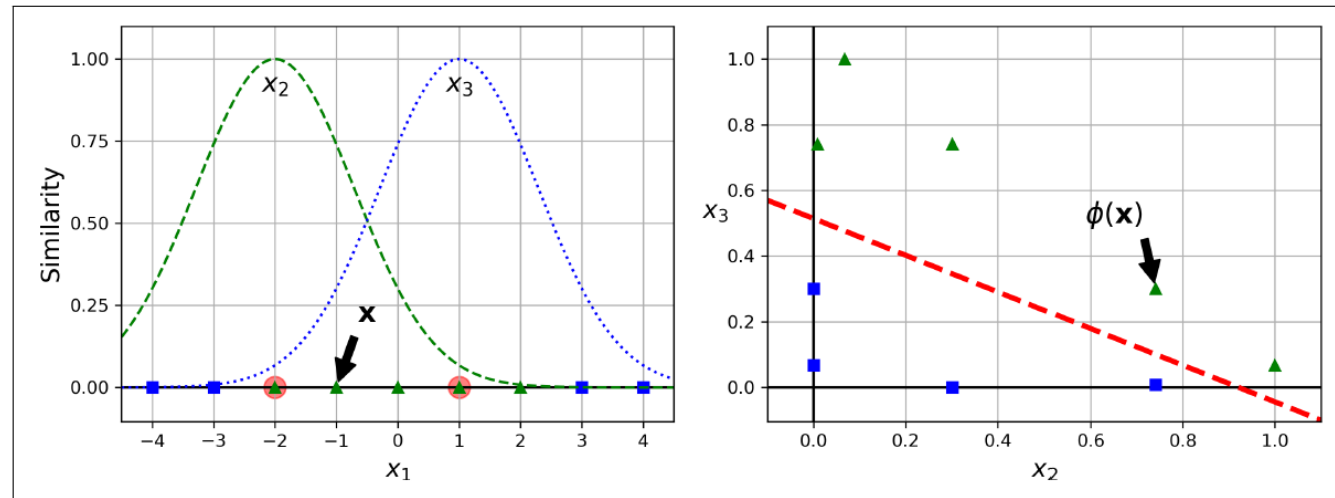


Kernel polinómico

- Añadir características polinómicas es sencillo de implementar y puede funcionar muy bien con todo tipo de algoritmos de aprendizaje automático.
- Con un grado polinómico bajo, este método no puede tratar conjuntos de datos muy complejos, y con un grado polinómico alto crea un número enorme de características, lo que hace que el modelo sea demasiado lento.
- Afortunadamente, cuando se utiliza SVM se puede aplicar una técnica matemática llamada el *truco del kernel*.
- El truco del kernel permite obtener el mismo resultado que si hubiera añadido muchas características polinómicas, incluso con un grado muy alto, sin tener que añadirlas realmente.
- Esto significa que no hay explosión combinatoria del número de características.

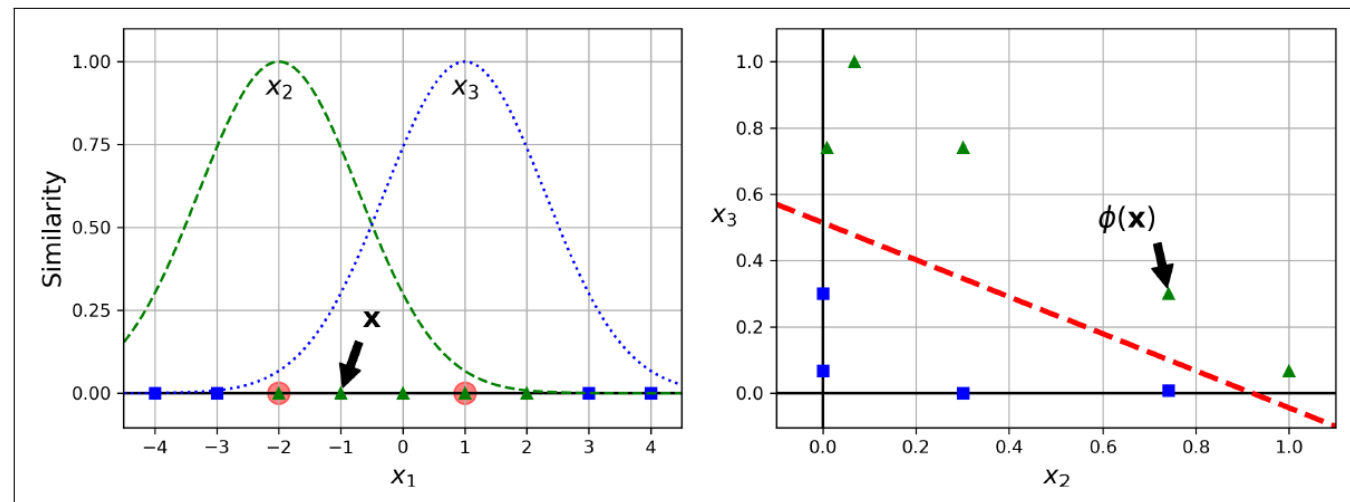
Características de similitud

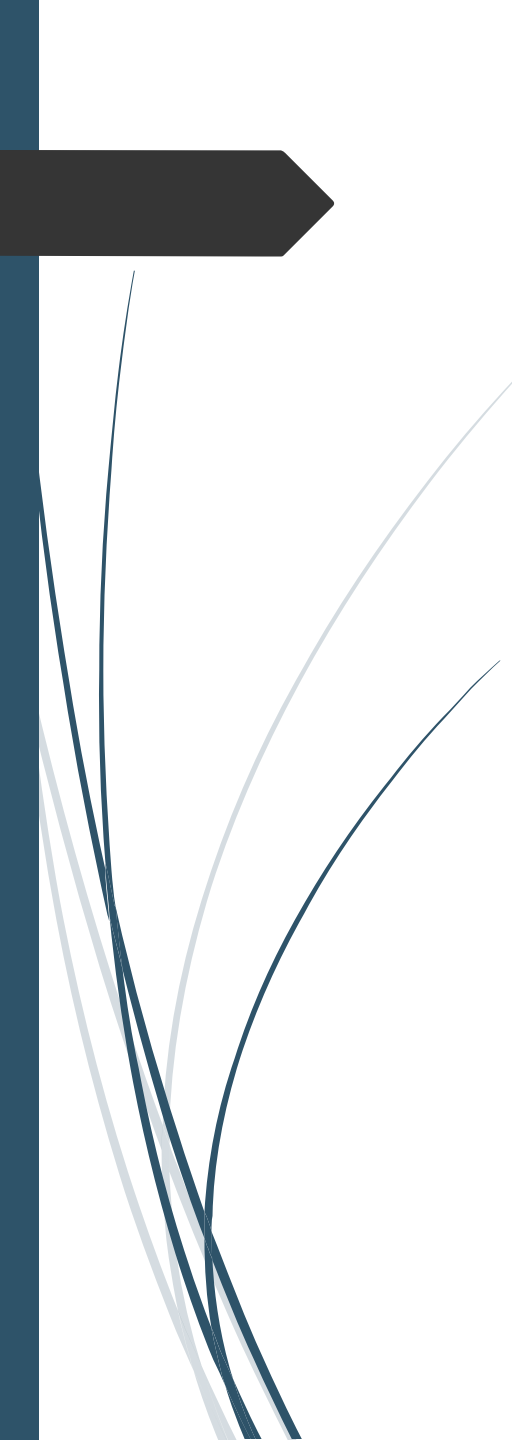
- Otra técnica para abordar problemas no lineales consiste en añadir características calculadas mediante una función de similitud, que mide cuánto se parece cada instancia a un punto de referencia concreto.
- Por ejemplo, tomemos el conjunto de datos 1D de antes y añadámosle dos puntos de referencia en $x_1 = -2$ y $x_1 = 1$ (véase el gráfico de la izquierda de la siguiente figura).
- A continuación, definiremos la función de similitud como la RBF gaussiana con $\gamma = 0,3$. Se trata de una función en forma de campana que varía de 0 (muy lejos del punto de referencia) a 1 (en el punto de referencia).



Características de similitud

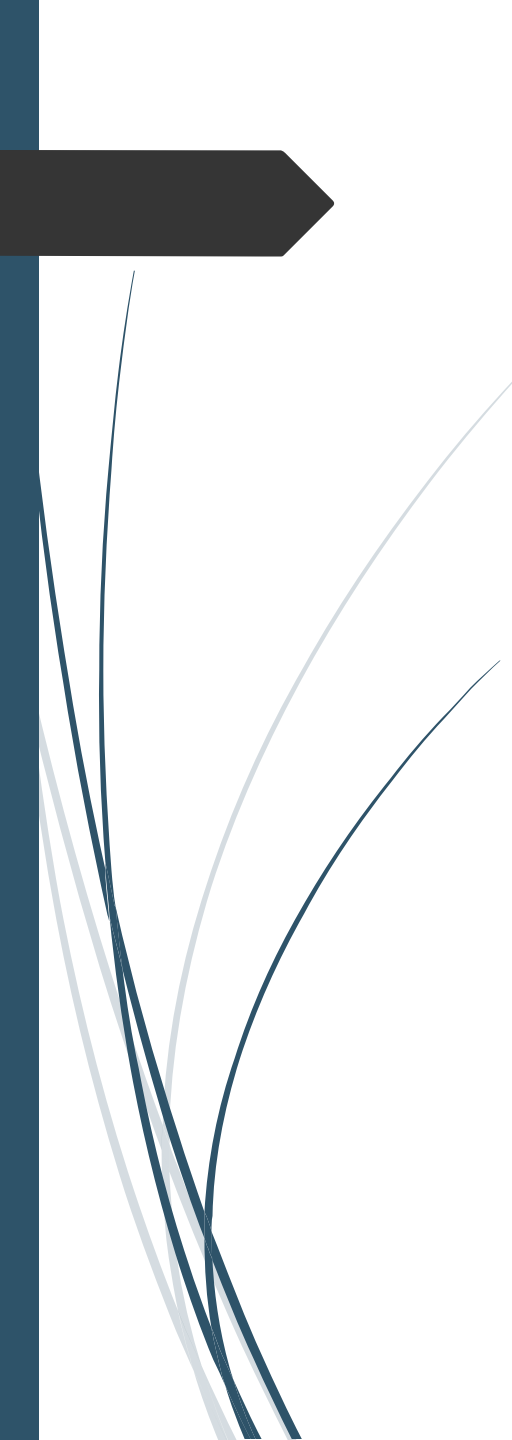
- Ahora estamos listos para calcular las nuevas características.
- Por ejemplo, veamos la instancia $x_1 = -1$: se encuentra a una distancia de 1 del primer punto de referencia y de 2 del segundo. Por tanto, sus nuevas características son $x_2 = \exp(-0,3 \times 1^2) \approx 0,74$ y $x_3 = \exp(-0,3 \times 2^2) \approx 0,30$. El gráfico de la derecha muestra el conjunto de datos transformado (sin las características originales).
- Como puede ver, ahora es linealmente separable.





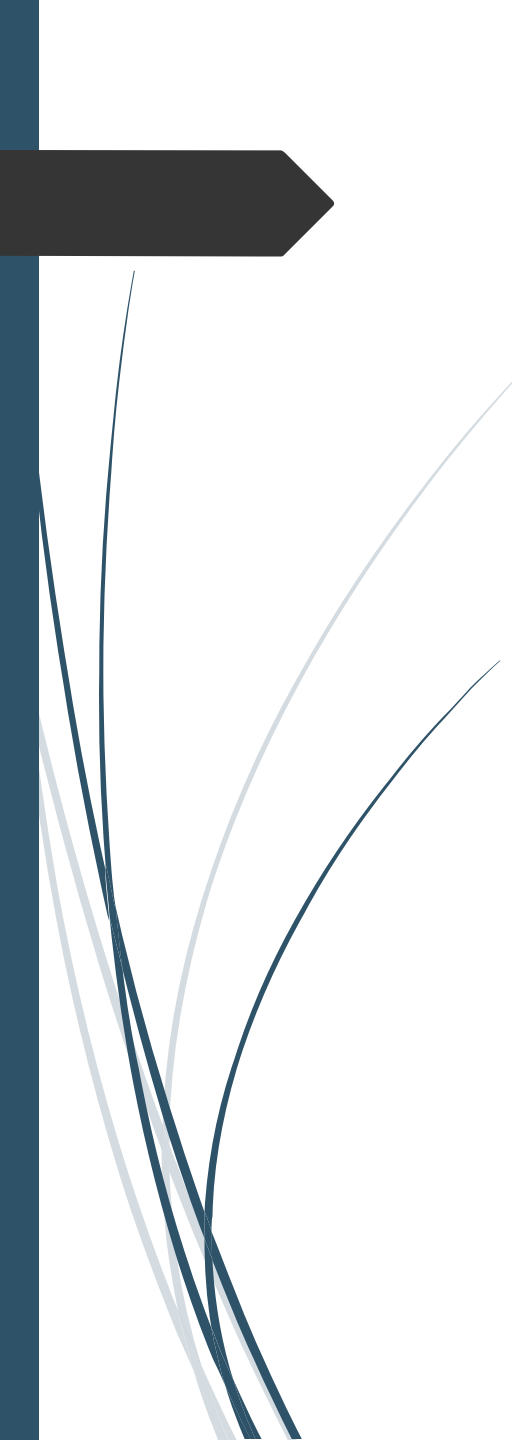
Kernel RBF gaussiano

- ¿Cómo seleccionar los puntos de referencia?
- El enfoque más sencillo consiste en crear un punto de referencia en la ubicación de todos y cada uno de los casos del conjunto de datos. Esto crea muchas dimensiones y aumenta las posibilidades de que el conjunto de entrenamiento transformado sea linealmente separable.
- El método de las características de similitud puede ser útil con cualquier algoritmo de aprendizaje automático, pero calcular todas las características adicionales puede resultar costoso desde el punto de vista informático.
- Una vez más el truco del kernel hace su magia SVM, haciendo posible obtener un resultado similar al que se obtendría si se hubieran añadido muchas características de similitud, pero sin hacerlo realmente.



Kernels para SVM

- Con tantos kernels para elegir, ¿cómo decidir cuál usar?
- Como regla general, siempre debe probar primero el kernel lineal. La clase `LinearSVC` es mucho más rápida que `SVC(kernel="linear")`, especialmente si el conjunto de entrenamiento es muy grande.
- Si no es demasiado grande, también debería probar las SVM kernelizadas, empezando por el kernel RBF gaussiano; suele funcionar realmente bien.
- Si tiene tiempo libre y potencia de cálculo, puede experimentar con otros núcleos utilizando la búsqueda de hiperparámetros. Si hay núcleos especializados para la estructura de datos de su conjunto de entrenamiento, asegúrese de probarlos también.



Clases de SVM y complejidad computacional

- La clase *LinearSVC* se basa en la biblioteca *liblinear*, que implementa un algoritmo optimizado para SVM lineales.
- No admite el truco del kernel, pero se escala casi linealmente con el número de instancias de entrenamiento y el número de características.
- Su complejidad en tiempo de entrenamiento es aproximadamente $O(m \times n)$.
- El algoritmo tarda más si se requiere una precisión muy alta.
- Esto se controla mediante el hiperparámetro de tolerancia ϵ (llamado *tol* en Scikit-Learn).
- En la mayoría de las tareas de clasificación, la tolerancia por defecto está bien.

Chih-Jen Lin et al., "A Dual Coordinate Descent Method for Large-Scale Linear SVM", Proceedings of the 25th International Conference on Machine Learning (2008): 408-415.

Clases de SVM y complejidad computacional

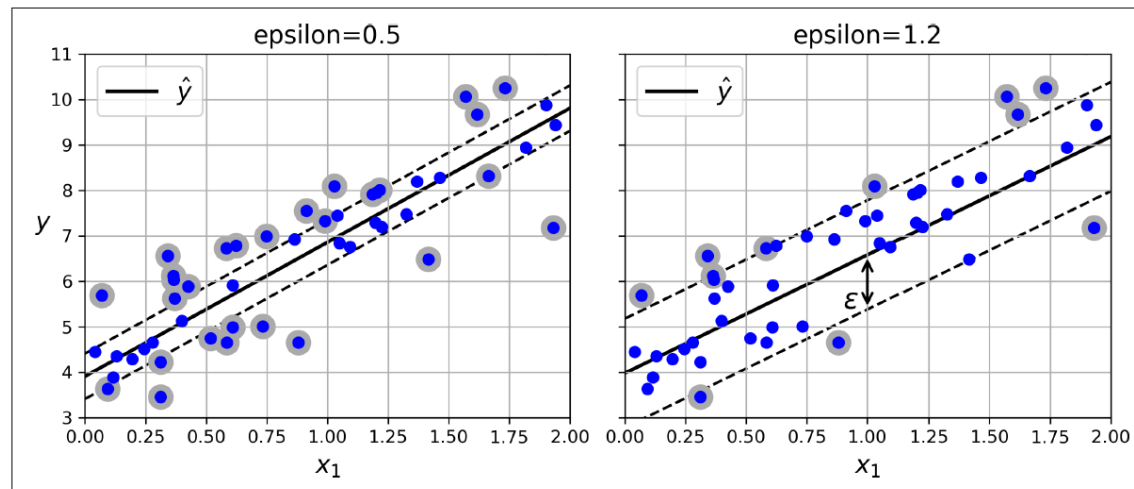
- La clase SVC se basa en la biblioteca *libsvm*, que implementa un algoritmo compatible con el truco del kernel.
- La complejidad del tiempo de entrenamiento suele estar entre $O(m^2 \times n)$ y $O(m^3 \times n)$.
- Por desgracia, esto significa que se vuelve terriblemente lento cuando el número de instancias de entrenamiento es grande (por ejemplo, cientos de miles de instancias), por lo que este algoritmo es mejor para conjuntos de entrenamiento no lineales de tamaño pequeño o mediano.
- Se adapta bien al número de características, sobre todo si son dispersas.
- En este caso, el algoritmo se escala aproximadamente con el número medio de características distintas de cero por instancia.

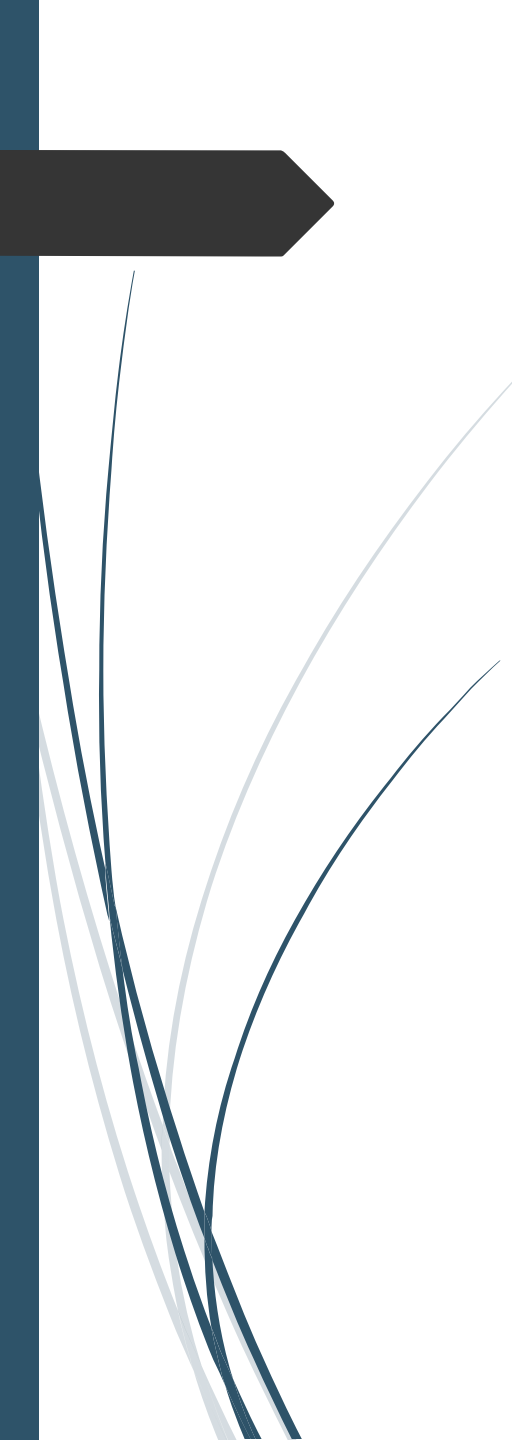
Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes
SGDClassifier	$O(m \times n)$	Yes	Yes	No

John Platt, " Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines "

Regresión SVM

- Para utilizar las SVM para la regresión en lugar de la clasificación, el truco consiste en modificar el objetivo: en lugar de intentar ajustar la mayor calle posible entre dos clases limitando al mismo tiempo las violaciones del margen, la regresión SVM intenta ajustar el mayor número posible de instancias en la calle limitando al mismo tiempo las violaciones del margen (es decir, instancias fuera de la calle).
- La anchura de la calle se controla mediante un hiperparámetro, ϵ .
- Esta figura muestra dos modelos de regresión lineal SVM entrenados sobre unos datos lineales, uno con un margen pequeño ($\epsilon = 0.5$) y otro con un margen mayor ($\epsilon = 1.2$).



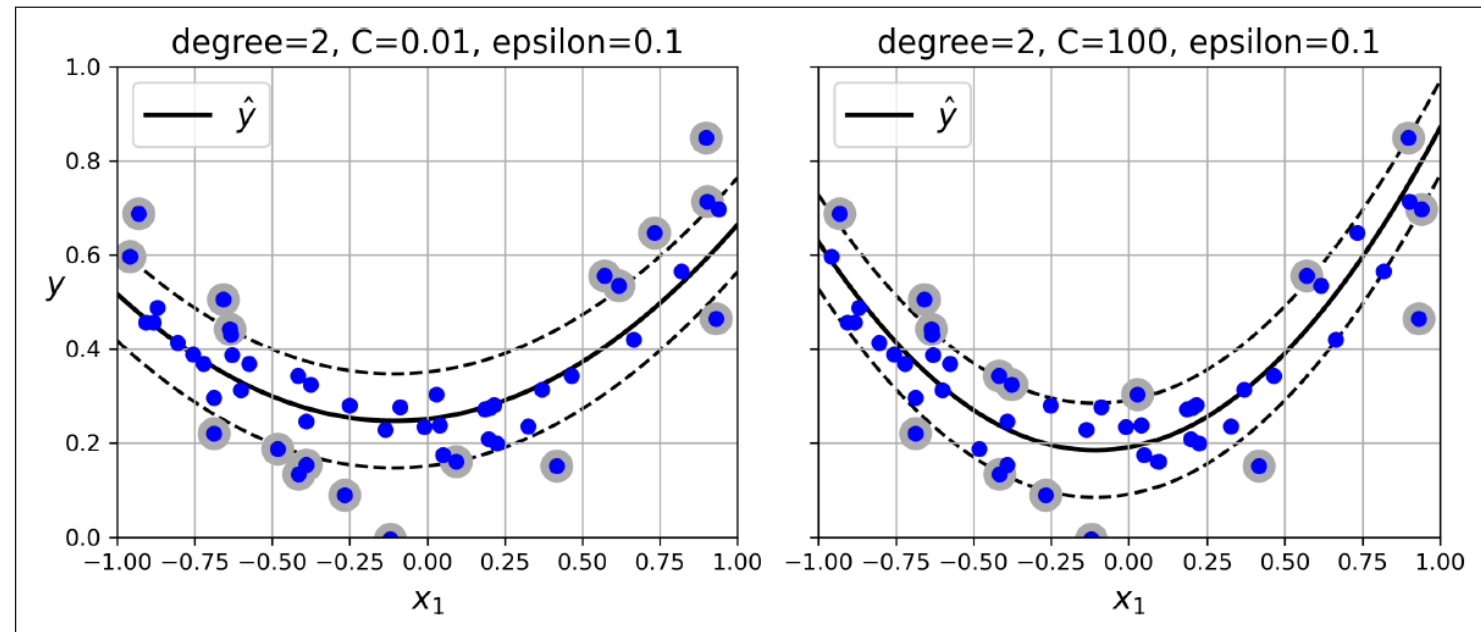


Regresión SVM

- Reducir ϵ aumenta el número de vectores de soporte, lo que regulariza el modelo.
- Además, si añades más instancias de entrenamiento dentro del margen, no afectará a las predicciones del modelo; por tanto, se dice que el modelo es ϵ -insensible.
- Para abordar tareas de regresión no lineal, puede utilizar un modelo SVM kernelizado.

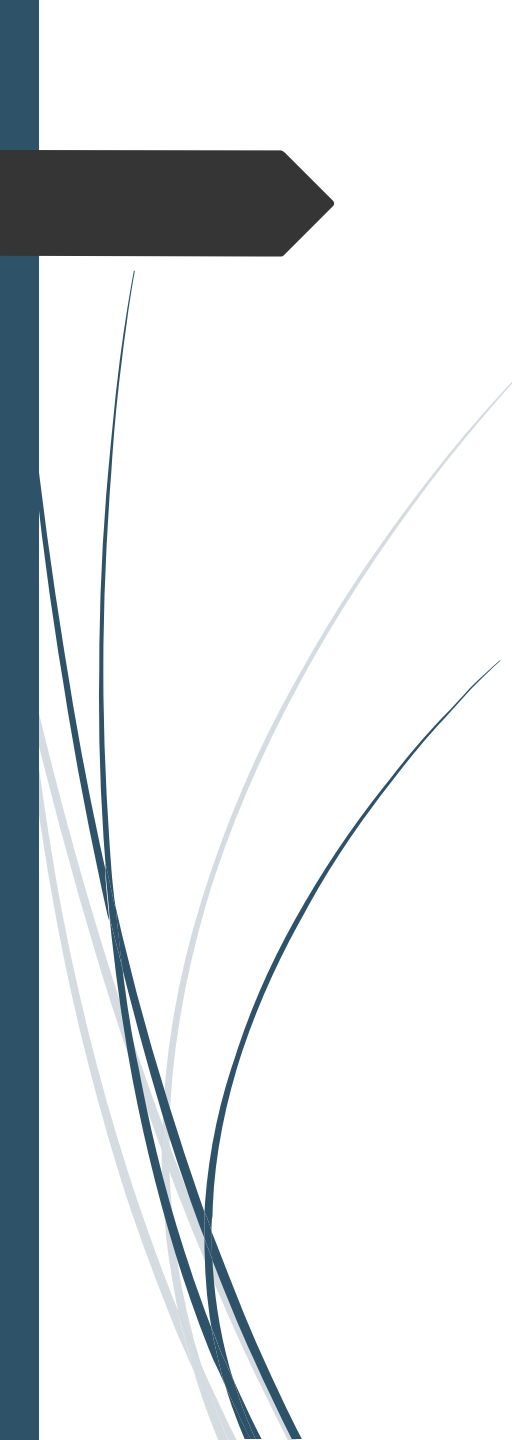
Regresión SVM

- La siguiente figura muestra la regresión SVM en un conjunto de entrenamiento cuadrático aleatorio, utilizando un kernel polinómico de segundo grado. Hay algo de regularización en el gráfico de la izquierda (es decir, un valor de C pequeño), y mucho menos en el gráfico de la derecha (es decir, un valor de C grande).



Funcionamiento de los clasificadores SVM lineales

- Un clasificador SVM lineal predice la clase de una nueva instancia x calculando primero la función de decisión $\theta^T x = \theta_0 x_0 + \dots + \theta_n x_n$, donde x_0 es la característica de sesgo (siempre igual a 1). Si el resultado es positivo, la clase predicha $\hat{y}$ es la clase positiva (1); en caso contrario, es la clase negativa (0). Es exactamente igual que *la regresión logística*.
- Hasta ahora, hemos utilizado la convención de poner todos los parámetros del modelo en un vector θ , incluyendo el término de sesgo θ_0 y los pesos de las características de entrada θ_1 a θ_n . Esto requería añadir una entrada de sesgo $x_0 = 1$ a todas las instancias.
- Otra convención muy común es separar el término de sesgo b (igual a θ_0) y el vector de pesos de características w (que contiene θ_1 a θ_n). En este caso, no es necesario añadir ninguna característica de sesgo a los vectores de características de entrada, y la función de decisión de la SVM lineal es igual a $w^T x + b = w_1 x_1 + \dots + w_n x_n + b$.

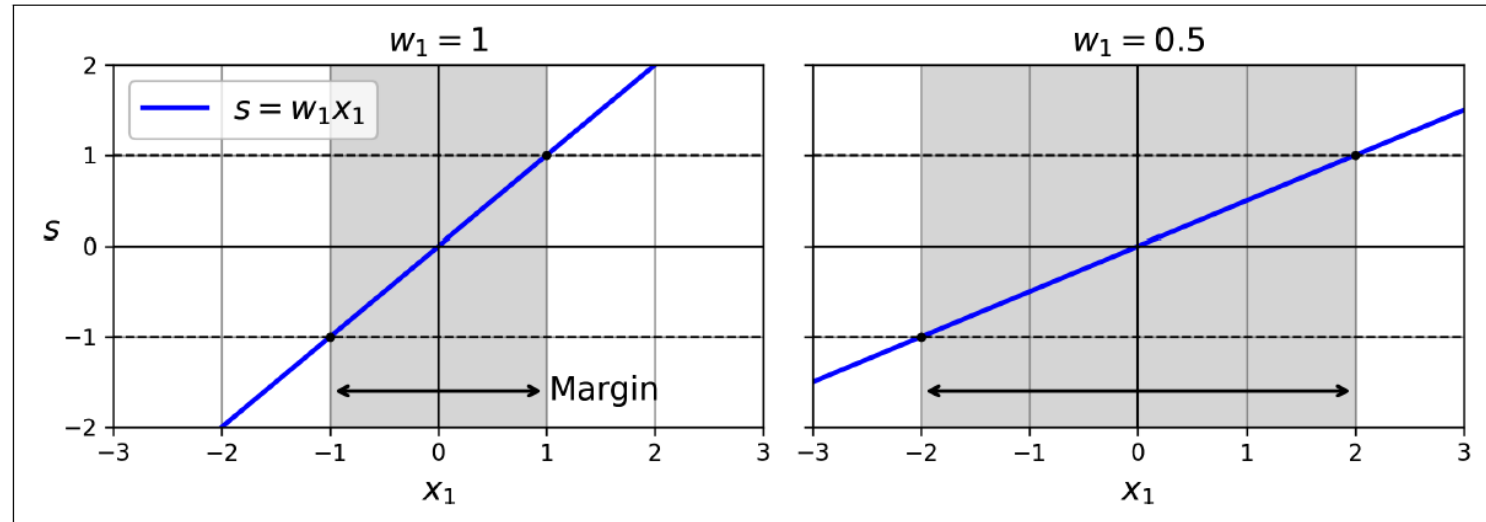


Funcionamiento de los clasificadores SVM lineales

- Hacer predicciones con un clasificador SVM lineal es bastante sencillo. ¿Y el entrenamiento?
- Para ello es necesario encontrar el vector de ponderaciones $\mathbf{w}$ y el término de sesgo b que hagan que la calle, o margen, sea lo más amplia posible limitando al mismo tiempo el número de violaciones del margen.
- Empecemos por la anchura de la calle: para hacerla más grande, tenemos que hacer $\mathbf{w}$ más pequeña.
- Esto puede ser más fácil de visualizar en 2D, como se muestra a continuación.

Funcionamiento de los clasificadores SVM lineales

- Definamos los márgenes de la calle como los puntos en los que la función de decisión es igual a -1 o $+1$. En el gráfico de la izquierda, el peso w_1 es 1 , por lo que los puntos en los que $w_1 x_1 = -1$ o $+1$ son $x_1 = -1$ y $+1$: por tanto, el tamaño del margen es 2 . En el gráfico de la derecha, el peso es $0,5$, por lo que los puntos en los que $w_1 x_1 = -1$ o $+1$ son $x_1 = -2$ y $+2$: el tamaño del margen es 4 . Por tanto, debemos mantener $\mathbf{w}$ lo más pequeño posible. Tenga en cuenta que el término de sesgo b no influye en el tamaño del margen: modificarlo sólo desplaza el margen, sin afectar a su tamaño.



Funcionamiento de los clasificadores SVM lineales

- También queremos evitar violaciones del margen, por lo que necesitamos que la función de decisión sea mayor que 1 para todas las instancias de entrenamiento positivas y menor que -1 para las instancias de entrenamiento negativas.
- Si definimos $t^{(i)} = -1$ para instancias negativas (cuando $y^{(i)} = 0$) y $t^{(i)} = 1$ para instancias positivas (cuando $y^{(i)} = 1$), entonces podemos escribir esta restricción como $t^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1$ para todas las instancias.
- Por lo tanto, podemos expresar el objetivo del clasificador lineal SVM de margen duro como el problema de optimización restringido en la siguiente ecuación:

$$\begin{aligned} & \underset{\mathbf{w}, b}{\text{minimize}} && \frac{1}{2} \mathbf{w}^\top \mathbf{w} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$

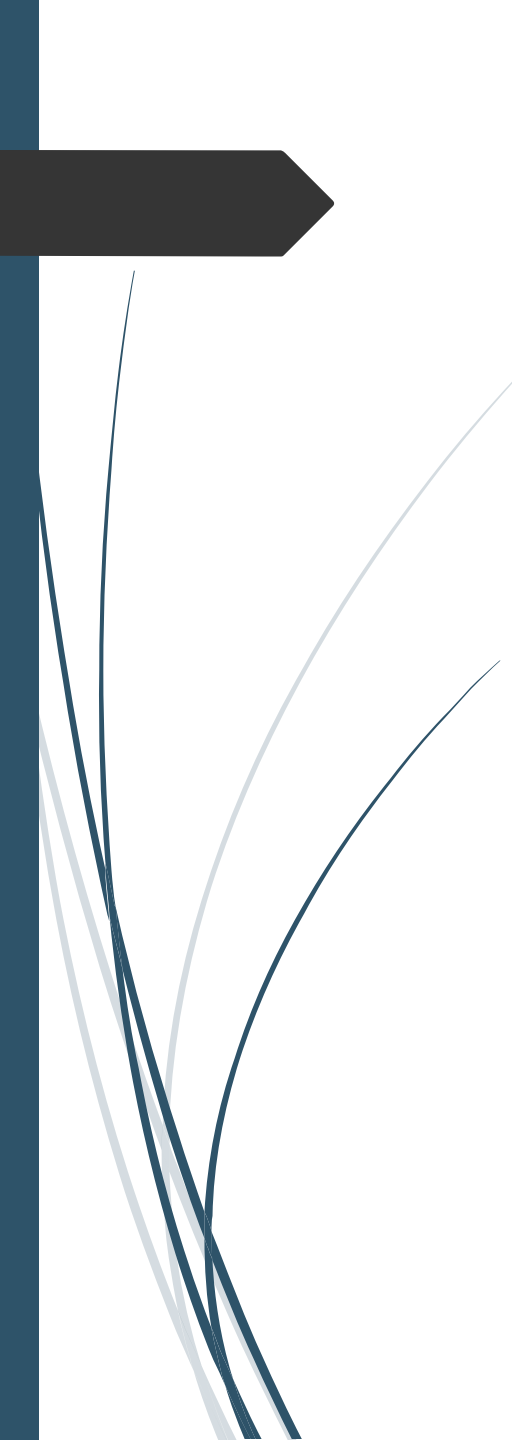
Estamos minimizando $\frac{1}{2} \mathbf{w}^\top \mathbf{w}$, que es igual a $\frac{1}{2} \|\mathbf{w}\|^2$, en lugar de minimizar $\|\mathbf{w}\|$ (la norma de $\mathbf{w}$). De hecho, $\frac{1}{2} \|\mathbf{w}\|^2$ tiene una derivada bonita y sencilla (es sólo $\mathbf{w}$), mientras que $\|\mathbf{w}\|$ no es diferenciable en $\mathbf{w} = 0$. Los algoritmos de optimización suelen funcionar mucho mejor con funciones diferenciables.

Funcionamiento de los clasificadores SVM lineales

- Para obtener el objetivo de margen suave, necesitamos introducir una variable de holgura $\zeta^{(i)} \geq 0$ para cada instancia: $\zeta^{(i)}$ mide cuánto se permite que la instancia i^{th} viole el margen.
- Ahora tenemos dos objetivos en conflicto: hacer que las variables de holgura sean lo más pequeñas posible para reducir las violaciones del margen, y hacer que $\frac{1}{2} \mathbf{w}^T \mathbf{w}$ sea lo más pequeño posible para aumentar el margen. Aquí es donde entra en juego el hiperparámetro C : nos permite definir el equilibrio entre estos dos objetivos. Esto nos da el problema de optimización restringida de la siguiente ecuación:

$$\underset{\mathbf{w}, b, \zeta}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^m \zeta^{(i)}$$

$$\text{subject to} \quad t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \dots, m$$

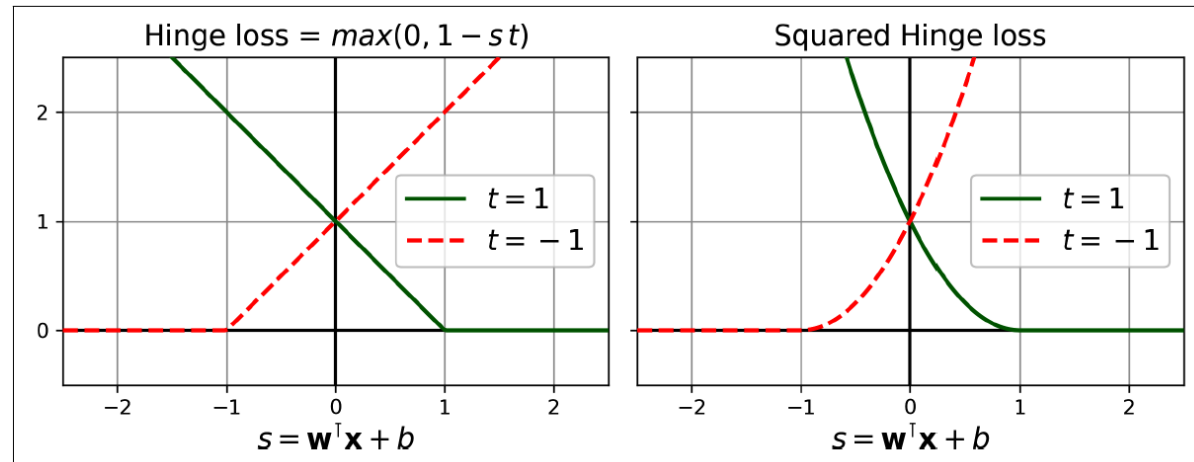


Funcionamiento de los clasificadores SVM lineales

- Tanto el problema del margen duro como el del margen blando son problemas de optimización cuadrática convexa con restricciones lineales.
- Estos problemas se conocen como problemas de *programación cuadrática* (QP).
- Hay muchos solucionadores disponibles para resolver problemas QP utilizando diversas técnicas (no dedicamos tiempo a esto en esta asignatura).
- Utilizar un solucionador QP es una forma de entrenar una SVM. Otra es utilizar el descenso de gradiente para minimizar el *hinge loss* o el *squared hinge loss*, en la siguiente diapositiva.

Funcionamiento de los clasificadores SVM lineales

- Dada una instancia $\mathbf{x}$ de la clase positiva (es decir, con $t = 1$), la pérdida es 0 si la salida s de la función de decisión ($s = \mathbf{w}^T \mathbf{x} + b$) es mayor o igual que 1.
- Dada una instancia de la clase negativa (es decir, con $t = -1$), la pérdida es 0 si $s \leq -1$.
- Cuanto más alejada esté una instancia del lado correcto del margen, mayor será la pérdida: crece linealmente para la hinge loss y cuadráticamente para la squared hinge loss. Esto hace que la hinge loss al cuadrado sea más sensible a los valores atípicos. Sin embargo, si el conjunto de datos está limpio, tiende a converger más rápidamente.



El doble problema

- Dado un problema de optimización con restricciones, conocido como *problema primario*, es posible expresar un problema diferente pero estrechamente relacionado, llamado su *problema dual*.
- La solución del problema dual suele dar una cota inferior a la solución del problema primario, pero en algunas condiciones puede tener la misma solución que el problema primario.
- Afortunadamente, el problema SVM pasa a cumplir estas condiciones (la función objetivo es convexa, y las restricciones de desigualdad son continuamente diferenciables y optimización convexa), por lo que puede optar por resolver el problema primal o el problema dual; ambos tendrán la misma solución.
- Forma dual del objetivo SVM lineal:

$$\begin{aligned} & \underset{\alpha}{\text{minimize}} \quad \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} \mathbf{x}^{(i)\top} \mathbf{x}^{(j)} - \sum_{i=1}^m \alpha^{(i)} \\ & \text{subject to } \alpha^{(i)} \geq 0 \text{ for all } i = 1, 2, \dots, m \text{ and } \sum_{i=1}^m \alpha^{(i)} t^{(i)} = 0 \end{aligned}$$

El doble problema

- Una vez que encuentre el vector $\hat{\alpha}$ que minimiza esta ecuación (utilizando un solucionador QP), utilice la siguiente ecuación para calcular el $\hat{\mathbf{w}}$ y $\hat{b}$ que minimizan el problema primal. En esta ecuación, n_s representa el número de vectores de soporte.

$$\hat{\mathbf{w}} = \sum_{i=1}^m \hat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \hat{\mathbf{w}}^T \mathbf{x}^{(i)} \right)$$

- El problema dual es más rápido de resolver que el primal cuando el número de instancias de entrenamiento es menor que el número de características. Y lo que es más importante, el problema dual hace posible el truco del kernel, mientras que el problema primario no.

SVM kernelizadas

- Supongamos que desea aplicar una transformación polinómica de segundo grado a un conjunto de entrenamiento bidimensional y, a continuación, entrenar un clasificador SVM lineal en el conjunto de entrenamiento transformado.
- Esta ecuación muestra la función cartográfica polinómica de segundo grado ϕ que se desea aplicar.

$$\varphi(\mathbf{x}) = \varphi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{pmatrix}$$

- Observe que el vector transformado es 3D en lugar de 2D.

SVM kernelizadas

- Veamos ahora qué ocurre con un par de vectores 2D, $\mathbf{a}$ y $\mathbf{b}$, si aplicamos este polinomio de segundo grado y calculamos el producto escalar de los vectores transformados.

$$\begin{aligned}\varphi(\mathbf{a})^\top \varphi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\top \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^\top \mathbf{b})^2\end{aligned}$$

- El producto punto de los vectores transformados es igual al cuadrado del producto punto de los vectores originales: $\phi(\mathbf{a})^\top \phi(\mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$.
- Aquí está la clave: si aplica la transformación ϕ a todas las instancias de entrenamiento, entonces el problema dual contendrá el producto punto $\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)})$. Pero si ϕ es la transformación polinómica de segundo grado, entonces se puede sustituir este producto punto de vectores transformados simplemente por $(\mathbf{x}^{(i)\top} \mathbf{x}^{(j)})^2$.

SVM kernelizadas

- No es necesario transformar las instancias de entrenamiento; basta con sustituir el producto punto por su cuadrado. El resultado será estrictamente el mismo que si hubiera pasado por la molestia de transformar el conjunto de entrenamiento y luego ajustar un algoritmo SVM lineal, pero este truco hace que todo el proceso sea mucho más eficiente computacionalmente.
- La función $K(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$ es un núcleo polinómico de segundo grado. En aprendizaje automático, un *núcleo* es una función capaz de calcular el producto punto $\phi(\mathbf{a})^\top \phi(\mathbf{b})$, basándose únicamente en los vectores originales $\mathbf{a}$ y $\mathbf{b}$, sin tener que calcular (o incluso conocer) la transformación ϕ .
- Kernels más utilizados.

Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^\top \mathbf{b} + r)^d$

Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$

Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^\top \mathbf{b} + r)$

SVM kernelizadas

- Mostramos cómo pasar de la solución dual a la solución primaria en el caso de un clasificador SVM lineal. Pero si aplicamos el truco del kernel, se termina con ecuaciones que incluyen $\phi(\mathbf{x}^{(i)})$. De hecho, $\hat{\mathbf{w}}$ debe tener el mismo número de dimensiones que $\phi(\mathbf{x}^{(i)})$, que puede ser enorme o incluso infinito, por lo que no se puede calcular.
- Pero, ¿cómo se pueden hacer predicciones sin conocer $\hat{\mathbf{w}}$? Bueno, la buena noticia es que puedes introducir la fórmula para $\hat{\mathbf{w}}$ de la ecuación anterior en la función de decisión para una nueva instancia $\mathbf{x}^{(n)}$, y obtendrás una ecuación con sólo productos escalares entre vectores de entrada. Esto permite utilizar el truco del kernel.

$$\begin{aligned}h_{\hat{\mathbf{w}}, \hat{b}}(\varphi(\mathbf{x}^{(n)})) &= \hat{\mathbf{w}}^\top \varphi(\mathbf{x}^{(n)}) + \hat{b} = \left(\sum_{i=1}^m \hat{\alpha}^{(i)} t^{(i)} \varphi(\mathbf{x}^{(i)}) \right)^\top \varphi(\mathbf{x}^{(n)}) + \hat{b} \\&= \sum_{i=1}^m \hat{\alpha}^{(i)} t^{(i)} (\varphi(\mathbf{x}^{(i)})^\top \varphi(\mathbf{x}^{(n)})) + \hat{b} \\&= \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \hat{\alpha}^{(i)} t^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(n)}) + \hat{b}\end{aligned}$$

SVM kernelizadas

- Tenga en cuenta que como $\alpha^{(i)} \neq 0$ sólo para los vectores de soporte, hacer predicciones implica calcular el producto punto del nuevo vector de entrada $\mathbf{x}^{(n)}$ sólo con los vectores de soporte, no con todas las instancias de entrenamiento. Por supuesto, es necesario utilizar el mismo truco para calcular el término de sesgo b :

$$\begin{aligned}\hat{b} &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \hat{\mathbf{w}}^\top \varphi(\mathbf{x}^{(i)}) \right) = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \left(\sum_{j=1}^m \hat{\alpha}^{(j)} t^{(j)} \varphi(\mathbf{x}^{(j)}) \right)^\top \varphi(\mathbf{x}^{(i)}) \right) \\ &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \sum_{\substack{j=1 \\ \hat{\alpha}^{(j)} > 0}}^m \hat{\alpha}^{(j)} t^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \right)\end{aligned}$$

Teorema de Mercer

- Según el *teorema de Mercer*, si una función $K(\mathbf{a}, \mathbf{b})$ respeta unas condiciones matemáticas llamadas *condiciones de Mercer* (por ejemplo, K debe ser continua y simétrica en sus argumentos de modo que $K(\mathbf{a}, \mathbf{b}) = K(\mathbf{b}, \mathbf{a})$, etc.), entonces existe una función ϕ que mapea $\mathbf{a}$ y $\mathbf{b}$ en otro espacio (posiblemente con dimensiones mucho mayores) tal que $K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\top \phi(\mathbf{b})$.
- Puedes usar K como kernel porque sabes que ϕ existe, aunque no sepas qué es ϕ . En el caso del núcleo RBF gaussiano, se puede demostrar que ϕ asigna cada instancia de entrenamiento a un espacio de dimensión infinita, por lo que es bueno no tener que realizar la asignación.
- Obsérvese que algunos kernels utilizados con frecuencia (como el kernel sigmoide) no respetan todas las condiciones de Mercer, aunque suelen funcionar bien en la práctica.



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Máquinas de Vector Soporte

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición



Árboles de decisión

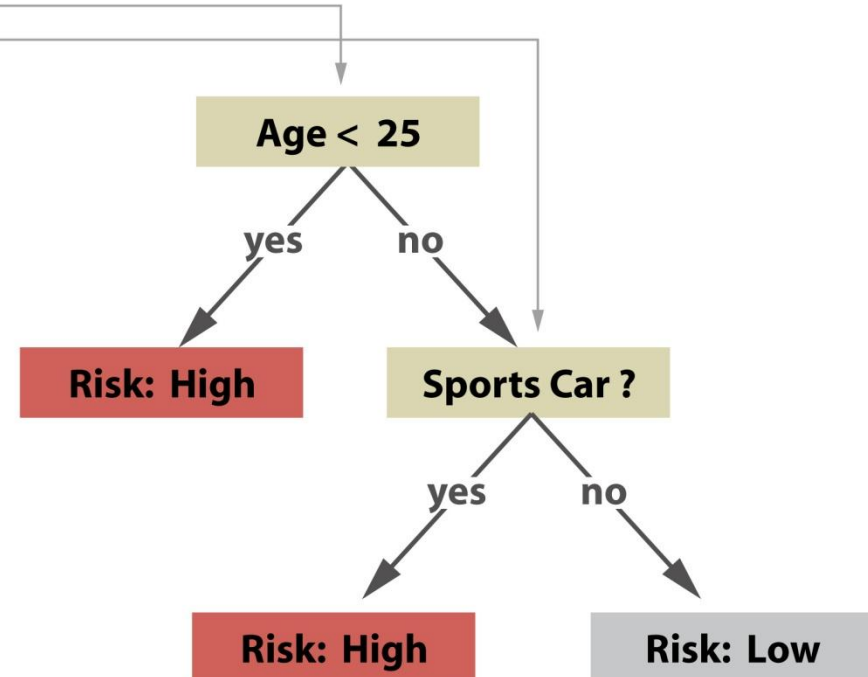
- Los árboles de decisión son algoritmos versátiles de aprendizaje automático que pueden realizar tareas tanto de clasificación como de regresión.
- Son algoritmos potentes, capaces de ajustar conjuntos de datos complejos.
- Los árboles de decisión son también los componentes fundamentales de los *random forests*, que se encuentran entre los algoritmos de aprendizaje automático más potentes de la actualidad.
- Aquí, empezaremos discutiendo cómo entrenar, visualizar y hacer predicciones con árboles de decisión. A continuación, repasaremos el algoritmo de entrenamiento CART utilizado por Scikit-Learn, y exploraremos cómo regularizar árboles y utilizarlos para tareas de regresión. Por último, discutiremos algunas de las limitaciones de los árboles de decisión.

Árboles de decisión



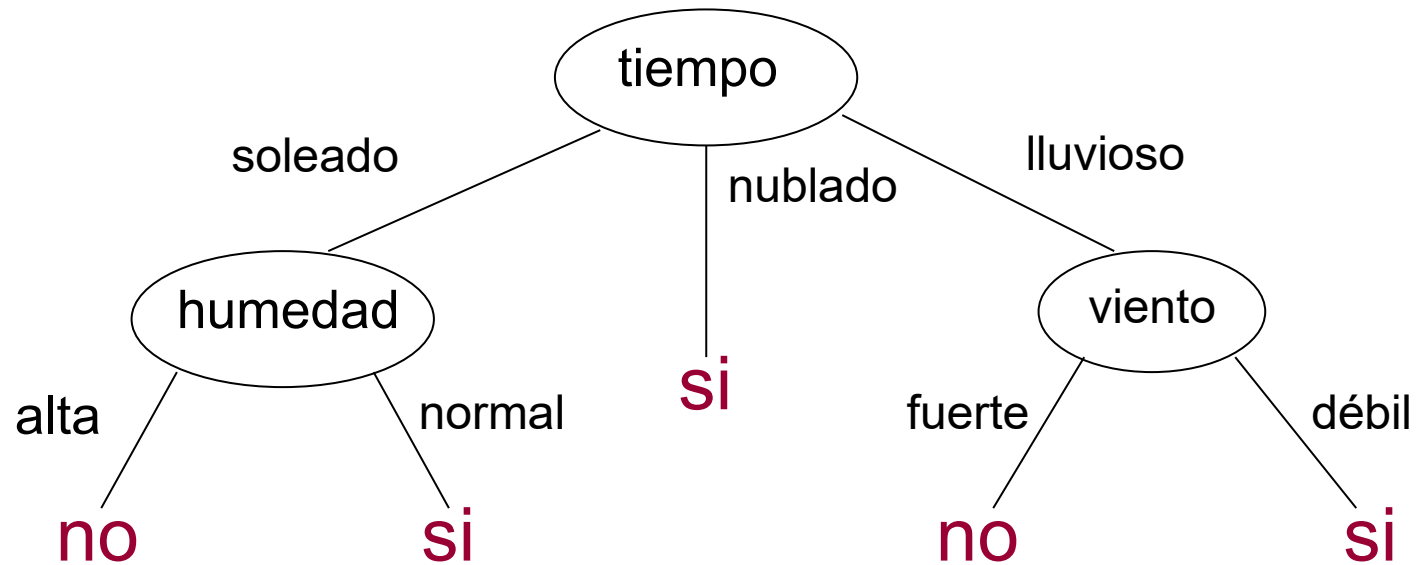
Insurance Risk Assessment

Age	Car Type	Risk
23	family	High
17	sports	High
43	sports	High
68	family	Low
32	truck	Low
20	family	High



Árboles de decisión

- **Ejemplo:** Decidir si se puede jugar al tenis dependiendo del tiempo que hace, de la humedad y del viento. Un posible árbol de decisión es



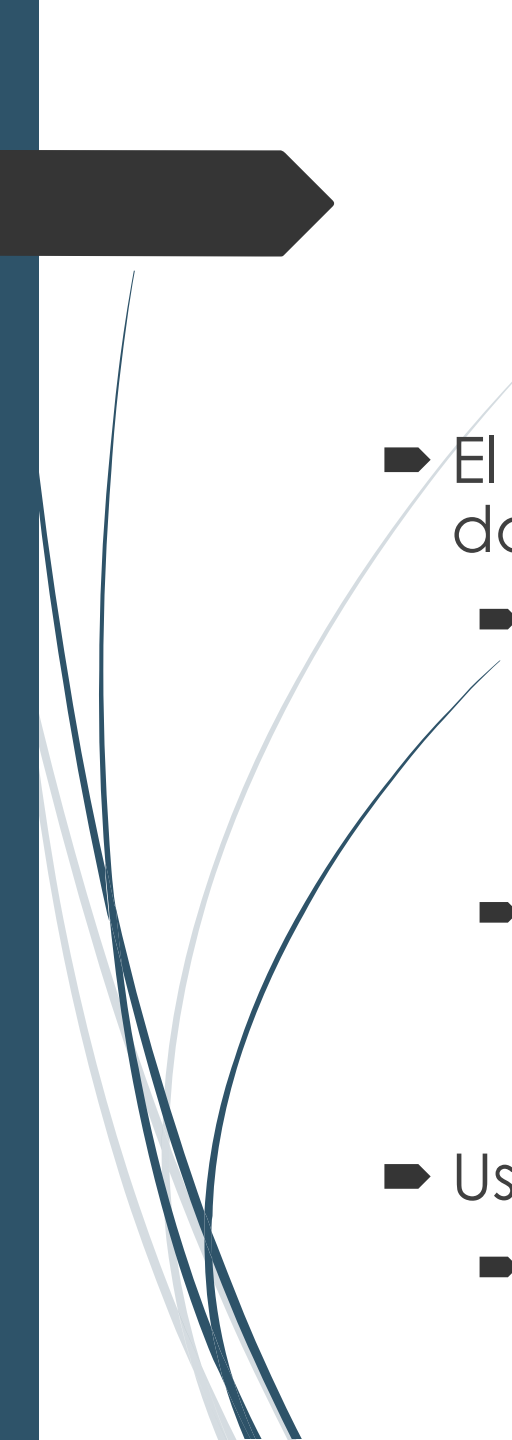
Para clasificar se comienza por la raíz del árbol y se van haciendo las pruebas necesarias hasta llegar a una hoja (clase). P.e.

(tiempo=s, viento=d, temperatura=72, humedad=n) → si

Árboles de decisión

- Otra opción es generar el conjunto de reglas asociado al árbol y utilizar un sistema basado en reglas (SBR). Del árbol anterior:

- 1) Si (tiempo soleado) y (humedad alta) entonces (jugar no)
- 2) Si (tiempo soleado) y (humedad normal) entonces (jugar si)
- 3) Si (tiempo nublado) entonces (jugar si)
- 4) Si (tiempo lluvioso) y (viento fuerte) entonces (jugar no)
- 5) Si (tiempo lluvioso) y (viento débil) entonces (jugar si)



Árboles de decisión

Construcción de árboles de decisión

- El proceso de generación de un árbol de decisión consiste en dos fases
 - Construcción del árbol
 - Al principio todos los ejemplos de entrenamiento están en el nodo raíz
 - Se van dividiendo recursivamente los ejemplos en base a los atributos seleccionados
 - Poda del árbol
 - Identificar y quitar ramas que describen ruido o datos anómalos
- Uso del árbol de decisión: Clasificar un ejemplo desconocido
 - Comprobar los valores de los atributos del ejemplo contra el árbol de decisión

Árboles de decisión

Construcción de árboles de decisión

- **Algoritmo básico** (algoritmo voraz)
 - Se construye el árbol mediante la técnica divide y vencerás aplicada de forma recursiva
 - Al principio todos los ejemplos de entrenamiento están en el nodo raíz
 - Los atributos son categóricos (si son continuos, se discretizan previamente)
 - Los ejemplos se dividen recursivamente basándose en atributos seleccionados
 - Los atributos de test se seleccionan en base a una medida heurística o estadística (por ejemplo, la ganancia de información)
- **Condiciones para terminar** el particionamiento
 - Todos los ejemplos para un nodo dado pertenecen a la misma clase
 - No quedan más atributos para seguir particionando. En este caso se utiliza el voto de la mayoría para clasificar en el nodo hoja
 - NO quedan ejemplos

Árboles de decisión

Construcción de árboles de decisión

- Entrada: Sea T el conjunto de ejemplos, $A=\{A_1, \dots, A_n\}$ el conjunto de atributos y $C=\{C_1, \dots, C_k\}$ el conjunto de valores que puede tomar la clase

ConstruirArbol (T, C, A)

1. Crear un nodo RAIZ para el árbol
2. Si todos los ejemplos en T pertenecen a la misma clase C_i , devolver el nodo RAIZ con etiqueta C_i
3. Si $A=\emptyset$ devolver el nodo RAIZ con etiqueta C_i donde C_i es la clase mayoritaria en T
4. $a \leftarrow$ el atributo de A que mejor clasifica T
5. Etiquetar RAIZ con a
6. Para cada valor v_i de a hacer
 1. Añadir una nueva rama debajo de RAIZ con el test $a=v_i$
 2. Sea T_i el subconjunto de T en el que $a = v_i$
 3. ConstruirArbol ($T_i, C, A-a$)
7. Devolver RAIZ

Árboles de decisión

Construcción de árboles de decisión

Problema de asignación de crédito

crédito	ingresos	propietario	Gastos-mensuales
N	Bajos	N	Altos
N	Bajos	S	Altos
N	Medios	S	Altos
N	Medios	N	Altos
N	Altos	N	Altos
S	Altos	S	Altos
N	Bajos	N	Bajos
N	Medios	N	Bajos
N	Altos	N	Bajos
s	Medios	S	Bajos

Árboles de decisión

Construcción de árboles de decisión

1. Se llama al algoritmo sobre el nodo raíz

crédito	ingresos	propietario	Gastos-mensuales
N	Bajos	N	Altos
N	Bajos	S	Altos
N	Medios	S	Altos
N	Medios	N	Altos
N	Altos	N	Altos
S	Altos	S	Altos
N	Bajos	N	Bajos
N	Medios	N	Bajos
N	Altos	N	Bajos
S	Medios	S	Bajos

Árboles de decisión

Construcción de árboles de decisión

2. Seleccionamos **gastos** como atributo que mejor clasifica

¿gastos?

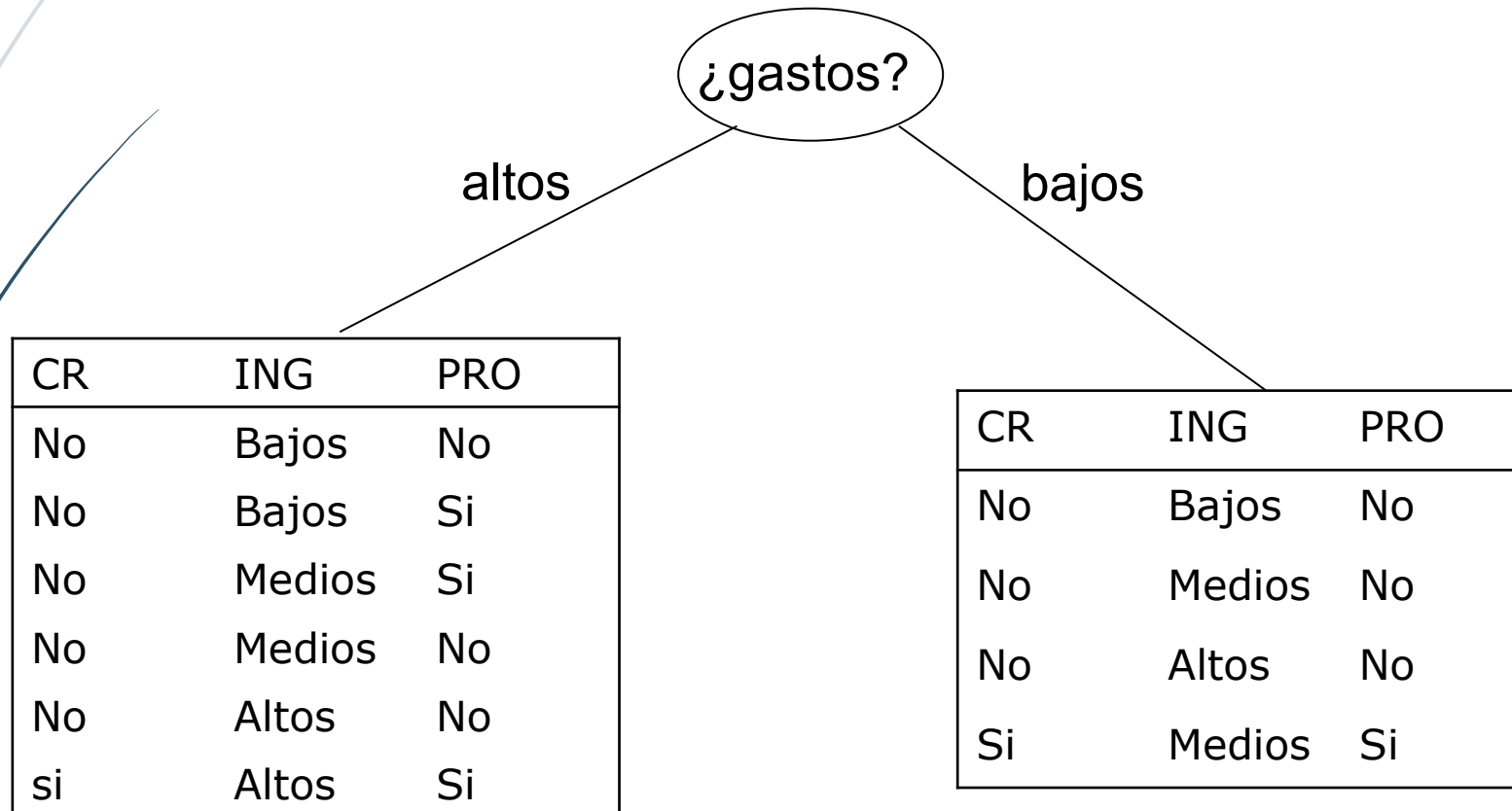
CR	ING	PRO	gastos
No	Bajos	No	Altos
No	Bajos	Si	Altos
No	Medios	Si	Altos
No	Medios	No	Altos
No	Altos	No	Altos
si	Altos	Si	Altos

CR	ING	PRO	gastos
No	Bajos	No	Bajos
No	Medios	No	Bajos
No	Altos	No	Bajos
Si	Medios	Si	Bajos

Árboles de decisión

Construcción de árboles de decisión

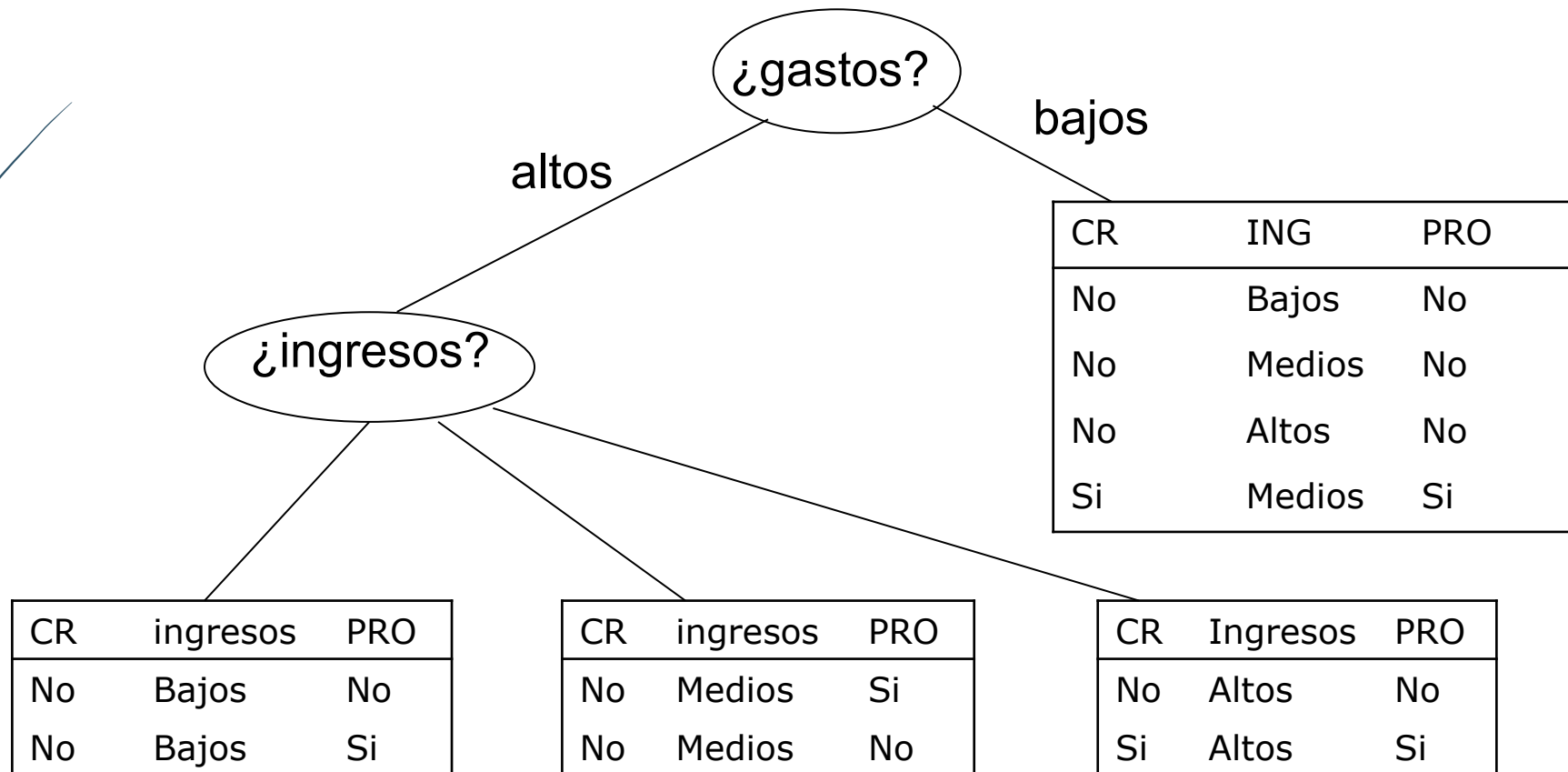
3. Preparamos los nodos para las llamadas recursivas



Árboles de decisión

Construcción de árboles de decisión

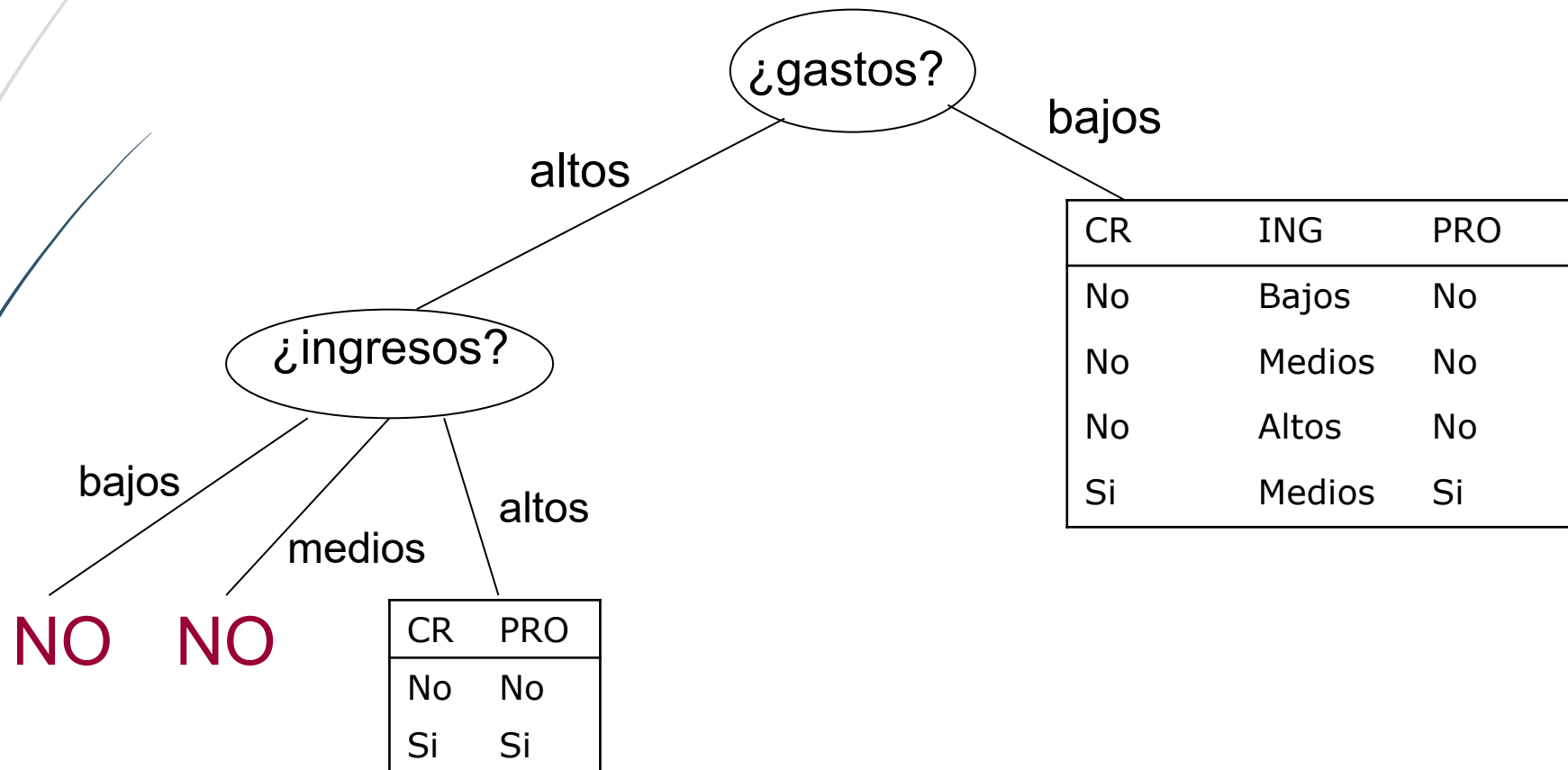
4. Seleccionamos **ingresos** como atributo en gastos = altos



Árboles de decisión

Construcción de árboles de decisión

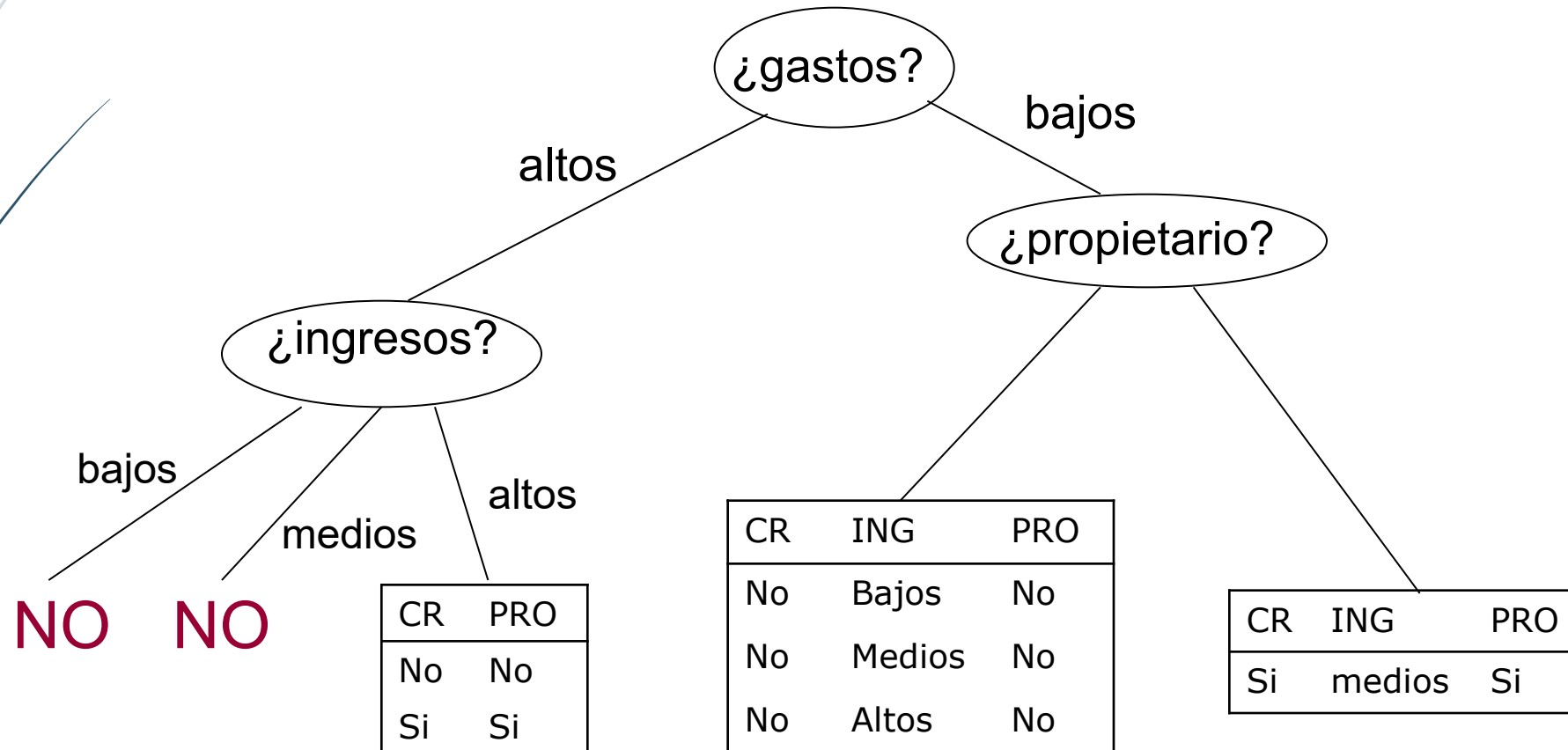
5. Creamos nodos hoja



Árboles de decisión

Construcción de árboles de decisión

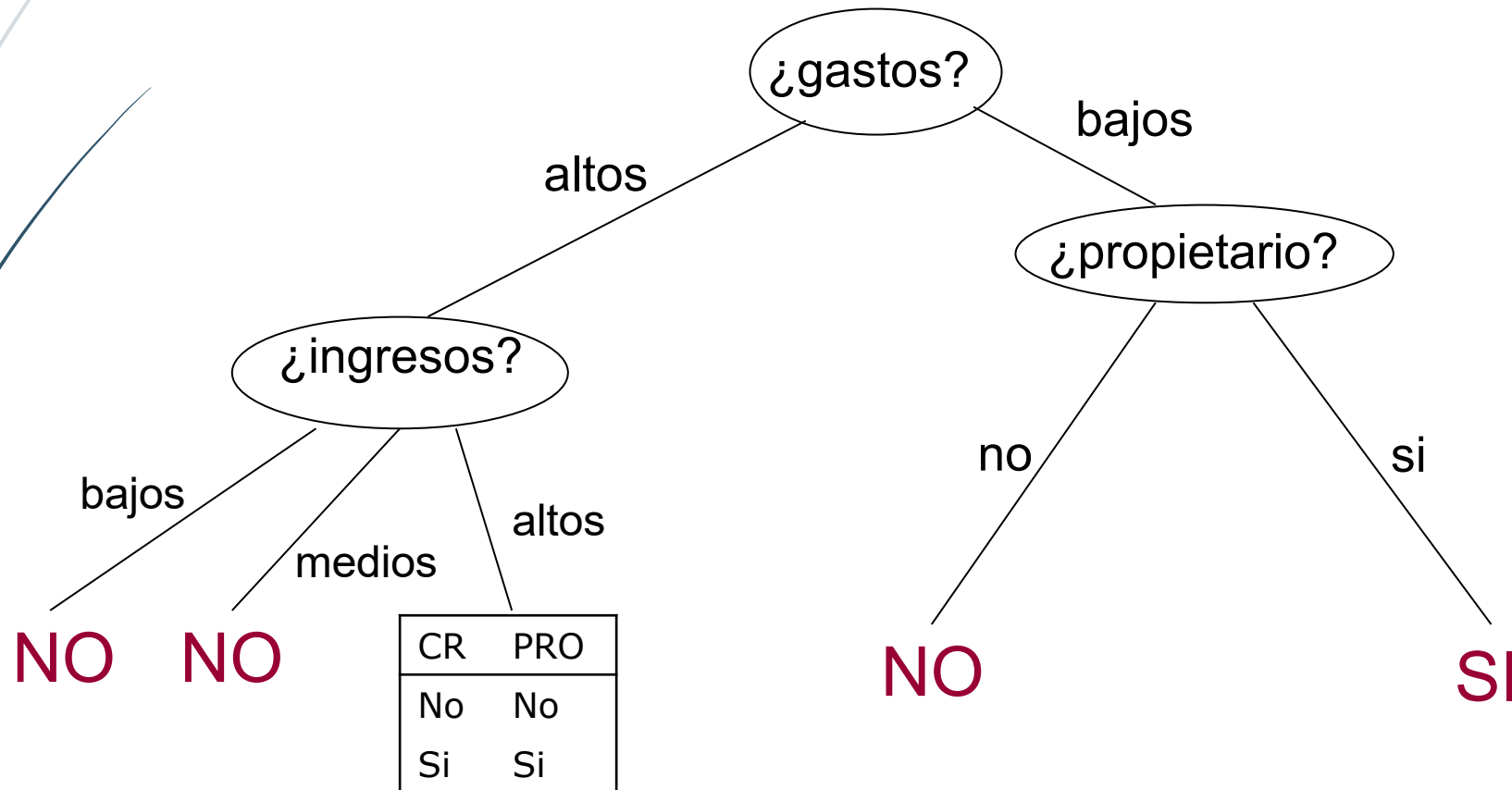
6. Seleccionamos **propietario** como atributo en gastos = bajos



Árboles de decisión

Construcción de árboles de decisión

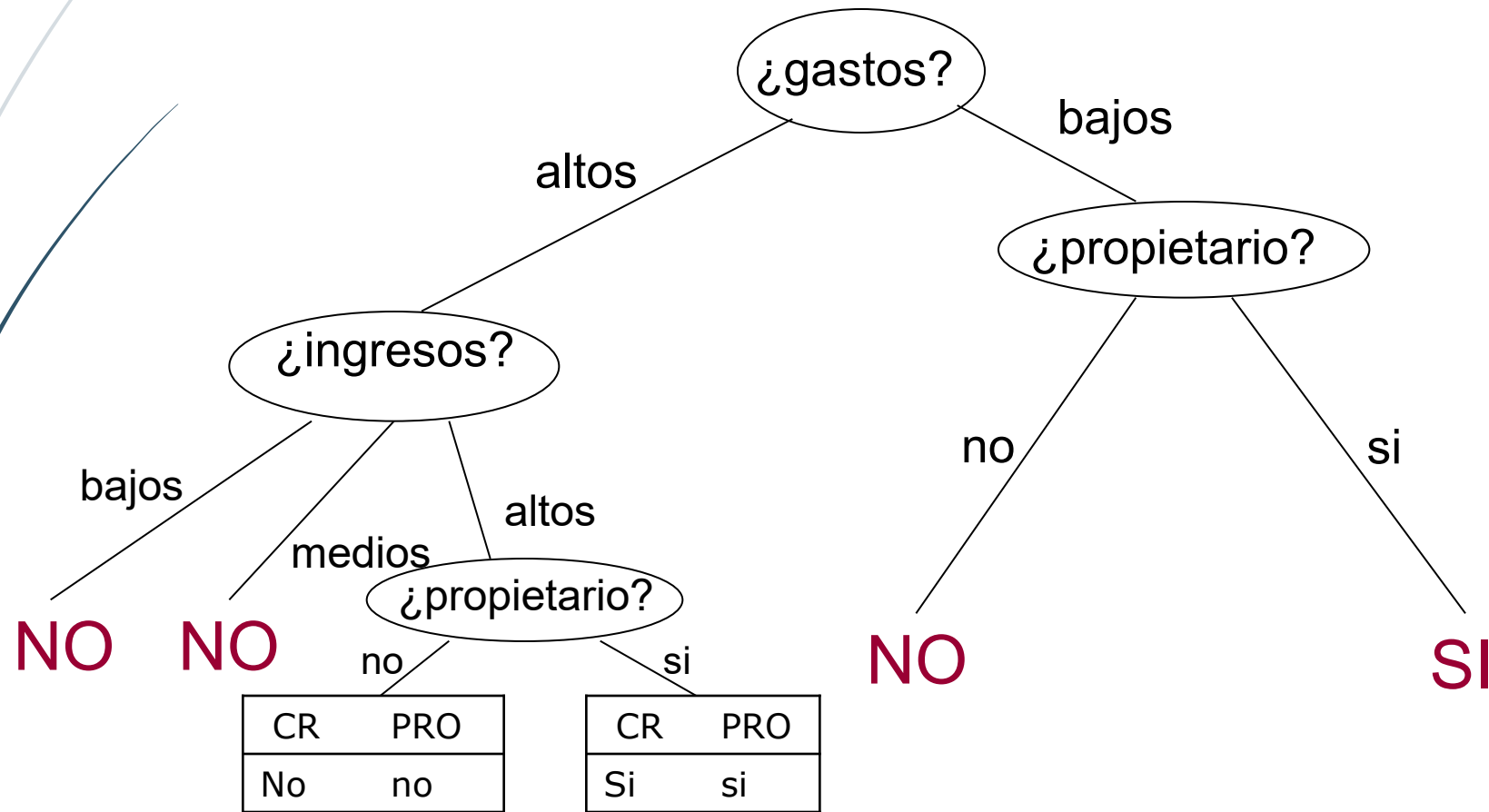
7. Creamos nodos hoja



Árboles de decisión

Construcción de árboles de decisión

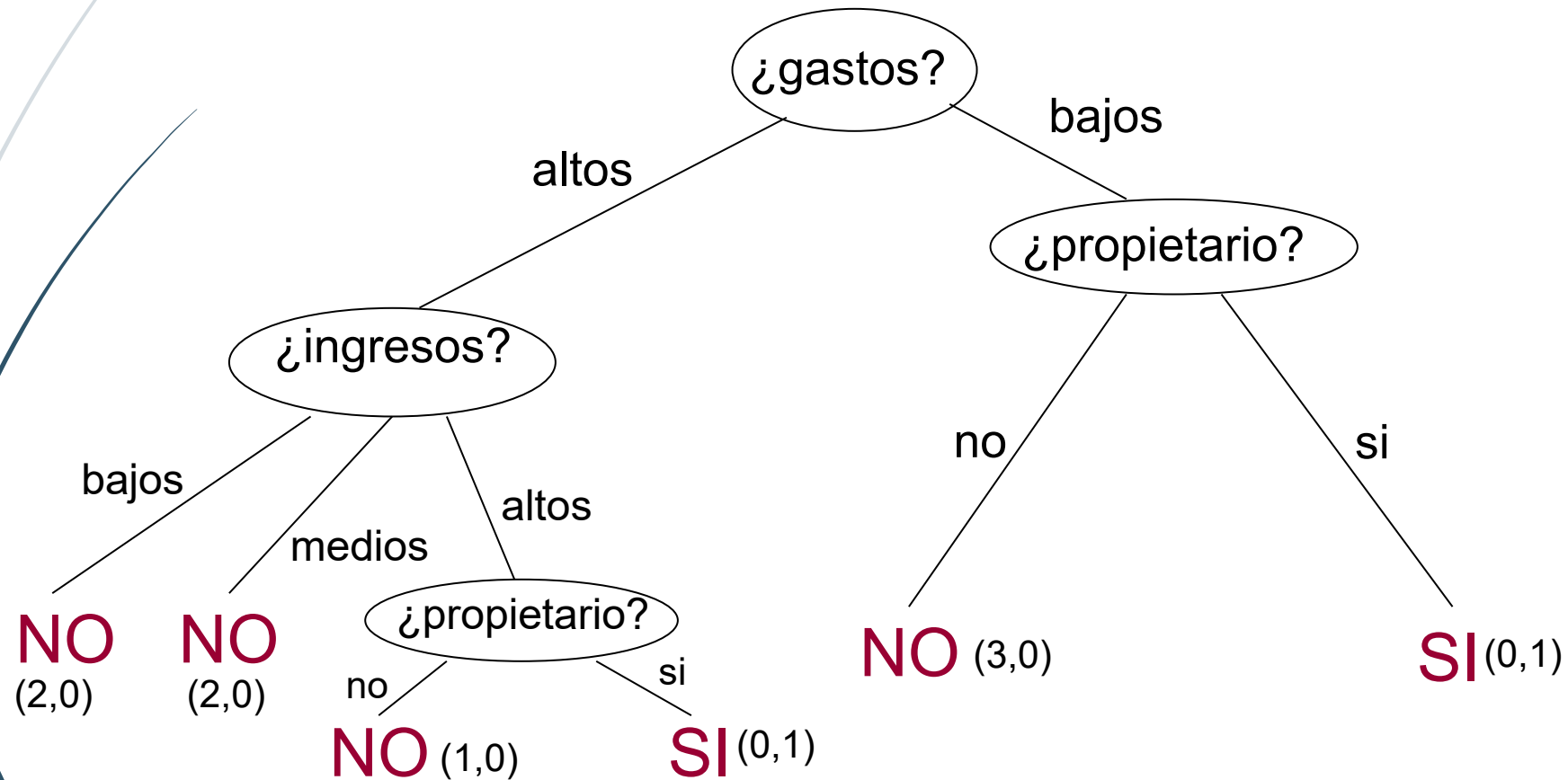
8. Seleccionamos **propietario** como atributo en gastos=altos, ingresos=altos



Árboles de decisión

Construcción de árboles de decisión

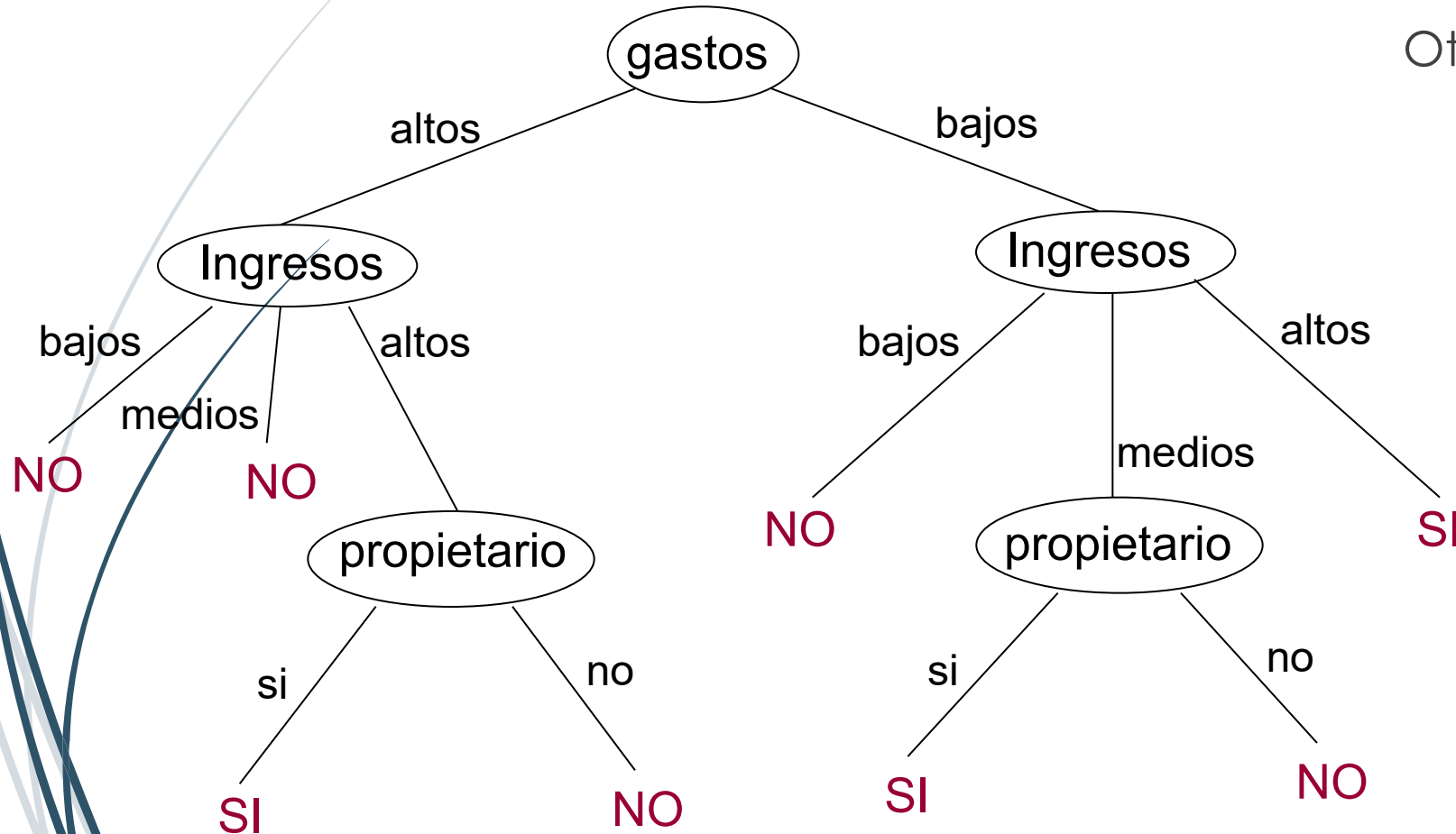
9. Árbol de decisión (#no, #si)



Árboles de decisión

Construcción de árboles de decisión

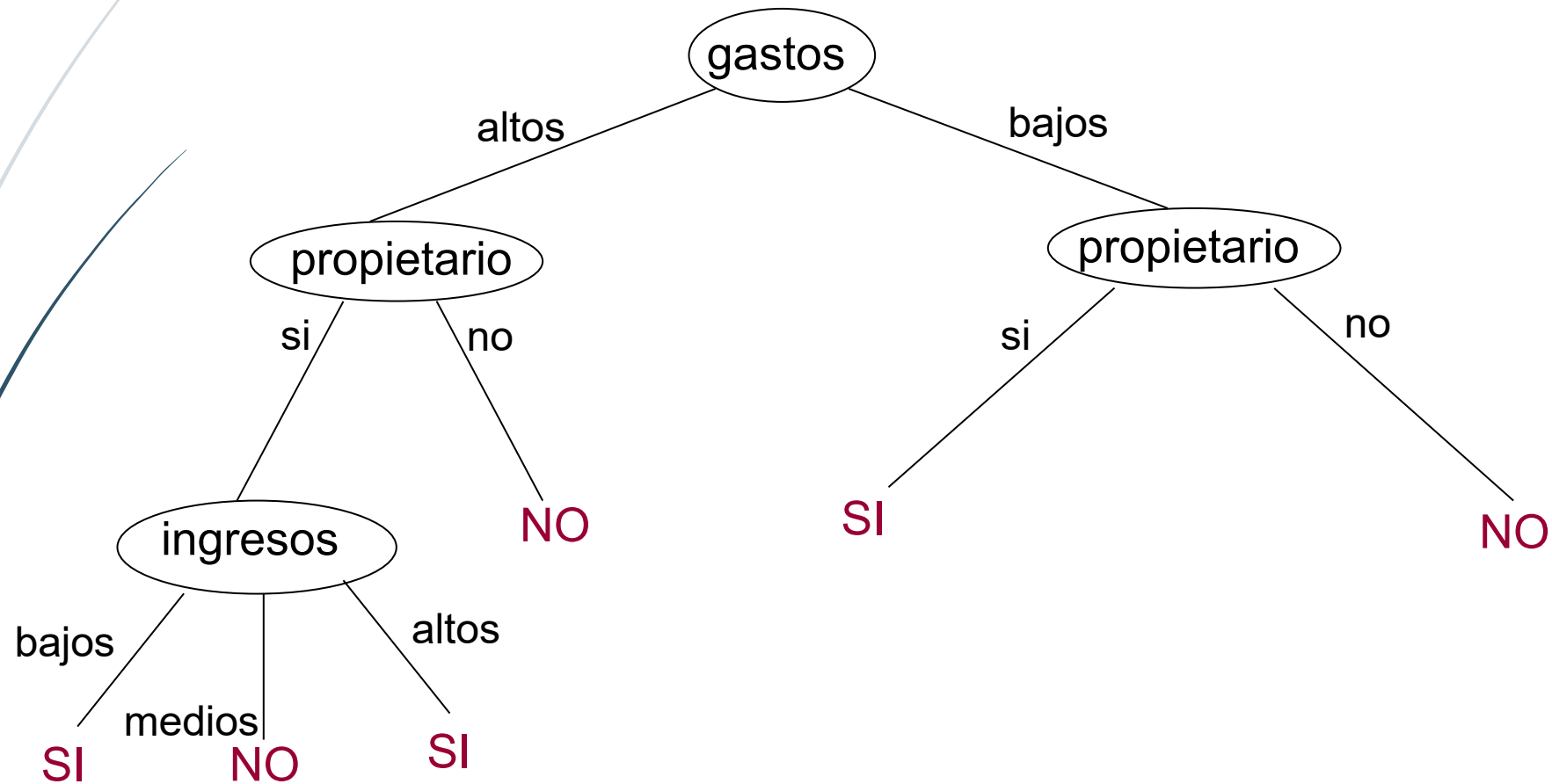
Otro árbol de decisión para crédito (árbol 2)



Árboles de decisión

Construcción de árboles de decisión

Otro árbol de decisión para crédito (árbol 3)





Árboles de decisión

Construcción de árboles de decisión

- 1er. árbol: 6 reglas (2.33 premisas por regla)
- 2do. árbol: 8 reglas (2.5 premisas por regla)
- 3er. árbol: 6 reglas (2.5 premisas por regla)

- Dependiendo del orden en el que se van tomando los atributos obtenemos clasificadores de distinta complejidad

- Lo ideal sería tomar en todo momento el atributo que mejor clasifica

¿Cómo decidir qué atributo es el mejor?

Árboles de decisión

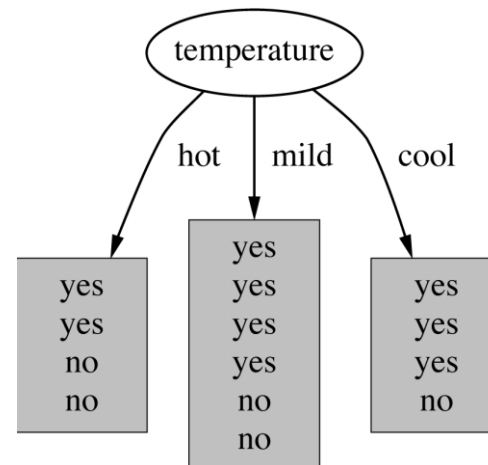
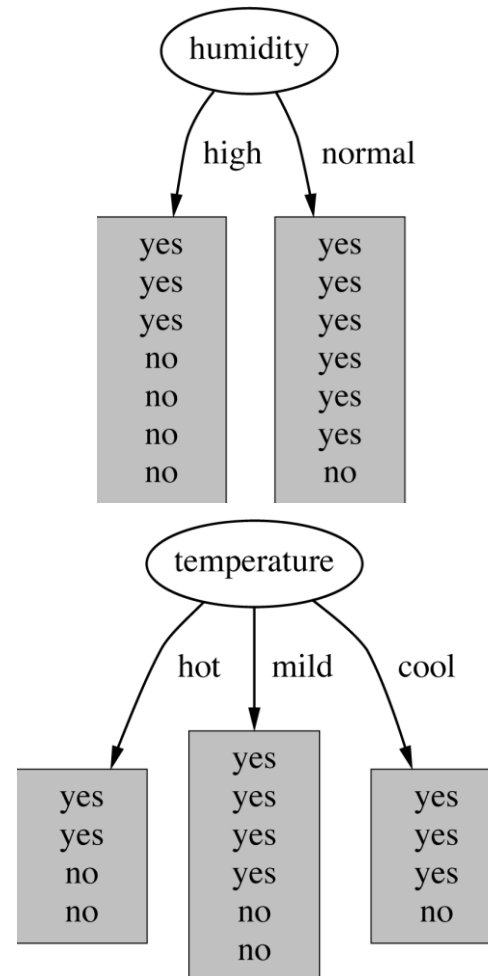
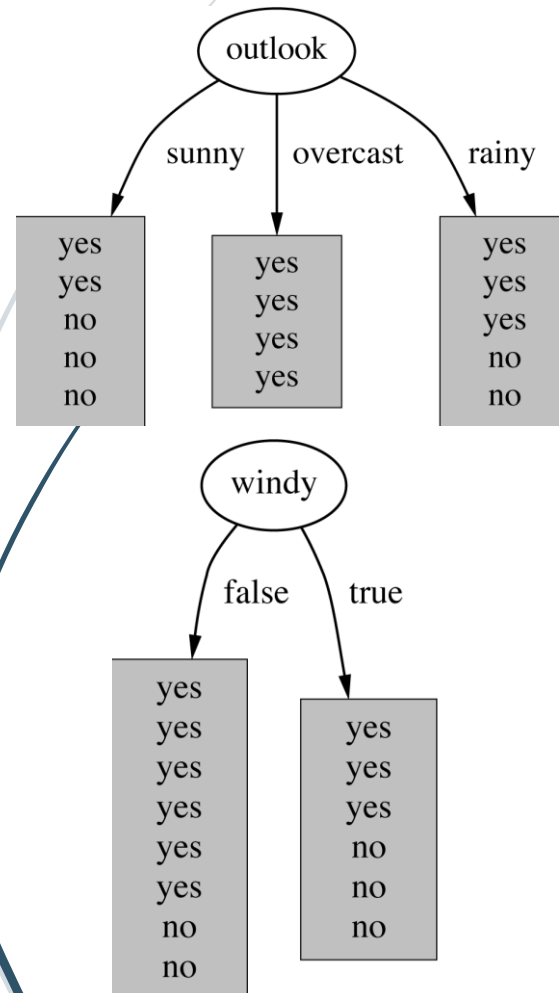
Construcción de árboles de decisión

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

Árboles de decisión

Construcción de árboles de decisión

¿qué atributo seleccionamos?



Árboles de decisión

Criterios de selección de variables

Existen distintos criterios para seleccionar el atributo X^* de test en cada momento. Algunos de los más conocidos son:

- *InfoGain*: Ganancia de información (ID3) (H es la entropía de la variable)

$$X^* = \max_X (H(C) - H(C|X))$$

$$H(X) = - \sum_i p(x_i) \log_2 p(x_i)$$

- *GainRatio*: Ganancia de información modificada (C4.5)

$$X^* = \max_X \frac{H(C) - H(C|X)}{H(X)}$$

- GINI(CART) $X^* = \max_X (G(C) - G(C|X))$ con $G = 1 - \sum_{i=1}^n p_i^2$

P.e.: $G(\text{gastos}) \approx 0.008$; $G(\text{crédito}) = 0.32$; $G(\text{credito} | \text{gastos}) \approx 0.312$

Árboles de decisión

Criterios de selección de variables

Ganancia de información

- Objetivo: Seleccionar el atributo con la mayor ganancia de información
- Consideremos un problema con dos clases: P y N
 - Sea S el conjunto de ejemplos que contiene p elementos de la clase P y n elementos de la clase N
 - La cantidad de información necesaria para decidir si un ejemplo arbitrario dentro de S pertenece a P o a N viene definido por

$$H(C)=H(p,n)=-\frac{p}{p+n}\log_2\frac{p}{p+n}-\frac{n}{p+n}\log_2\frac{n}{p+n}$$

La entropía mide la aleatoriedad o sorpresa o incertidumbre al predecir una clase

Árboles de decisión

Criterios de selección de variables

- Consideremos que utilizando un atributo A , el conjunto S se puede dividir en v conjuntos $\{S_1, S_2, \dots, S_v\}$
 - Si S_i contiene p_i ejemplos de P y n_i ejemplos de N , la entropía o la información que se espera sea necesaria para clasificar los objetos en todos los subárboles S_i es

$$H(C / A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} H(p_i, n_i)$$

- La información que se podría ganar con una rama que considere A es

$$Gain(A) = H(p, n) - H(C / A)$$

Es la reducción en la entropía al considerar el atributo A

Árboles de decisión

Criterios de selección de variables

GINI

- Si un conjunto de datos T tiene ejemplos pertenecientes a n clases, el índice gini, se define como

$$gini(T) = 1 - \sum_{j=1}^n p_j^2$$

donde p_j es la frecuencia relativa de la clase j in T

- Si se divide un conjunto de datos T en dos subconjuntos T_1 y T_2 de tamaños N_1 y N_2 respectivamente, el índice *gini* de los datos separados conteniendo ejemplos de las n clases se define como

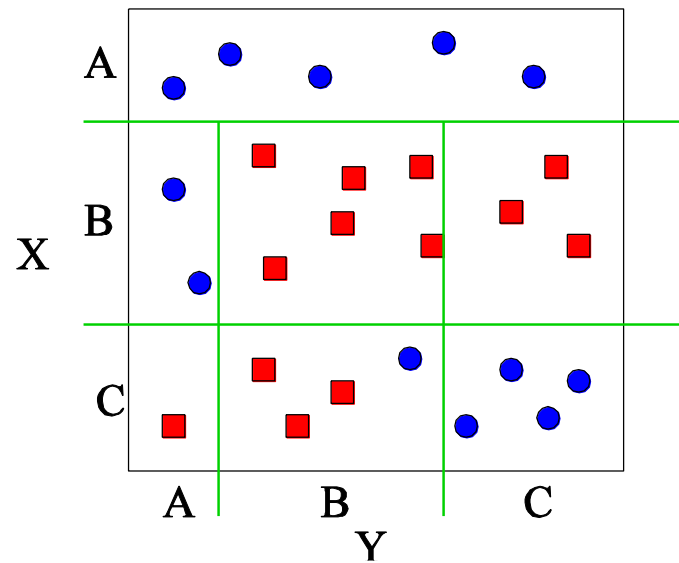
$$gini_{split}(T) = \frac{N_1}{N} gini(T_1) + \frac{N_2}{N} gini(T_2)$$

- Se elige para dividir el nodo, el atributo que proporciona el índice $gini_{split}(T)$ más pequeño (es necesario enumerar todos los posibles puntos de división para cada atributo)

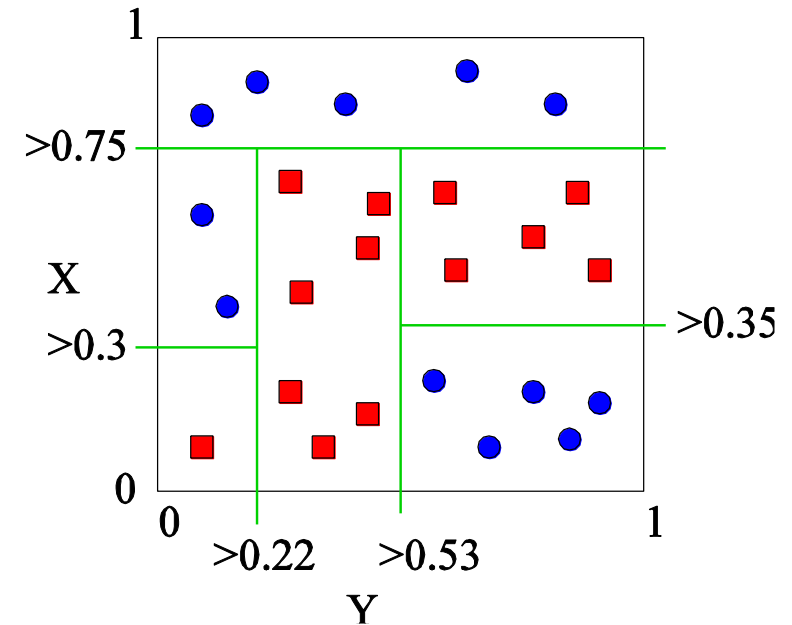
Árboles de decisión

Particionamiento del espacio

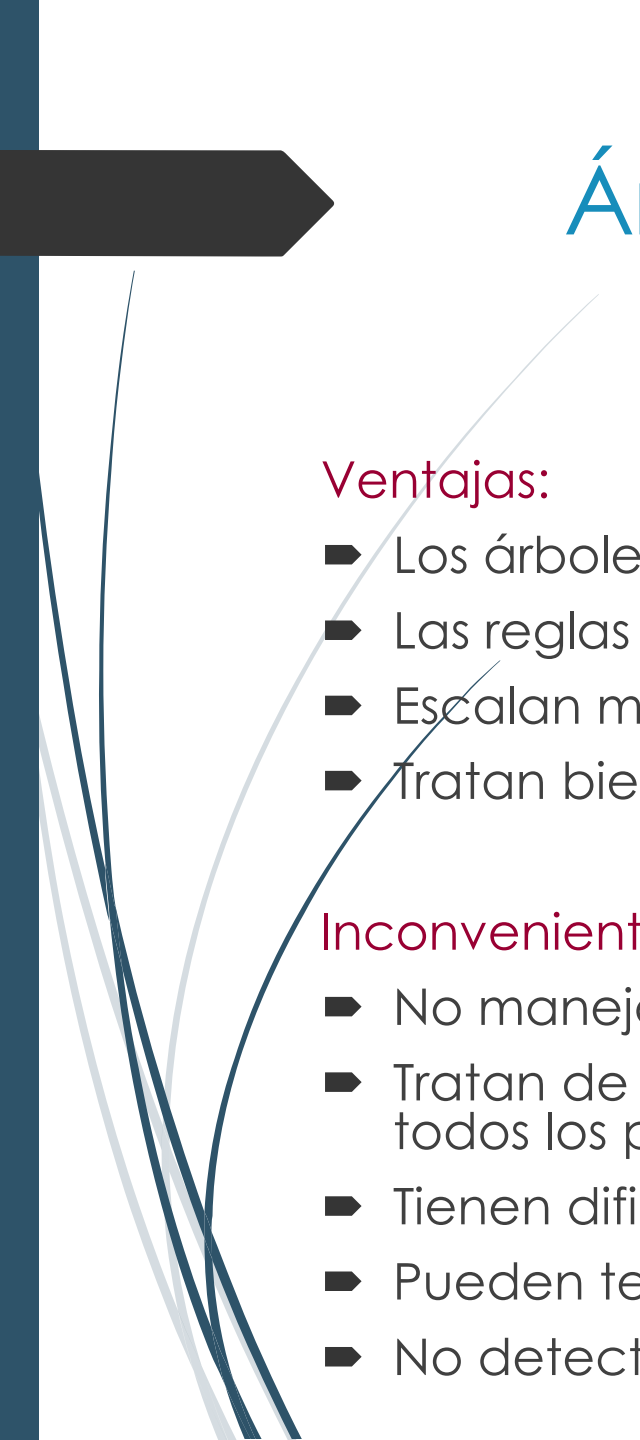
Los árboles particionan el espacio de soluciones de forma exhaustiva



Variables X e Y discretas



Variables X e Y continuas



Árboles de decisión

Ventajas e inconvenientes

Ventajas:

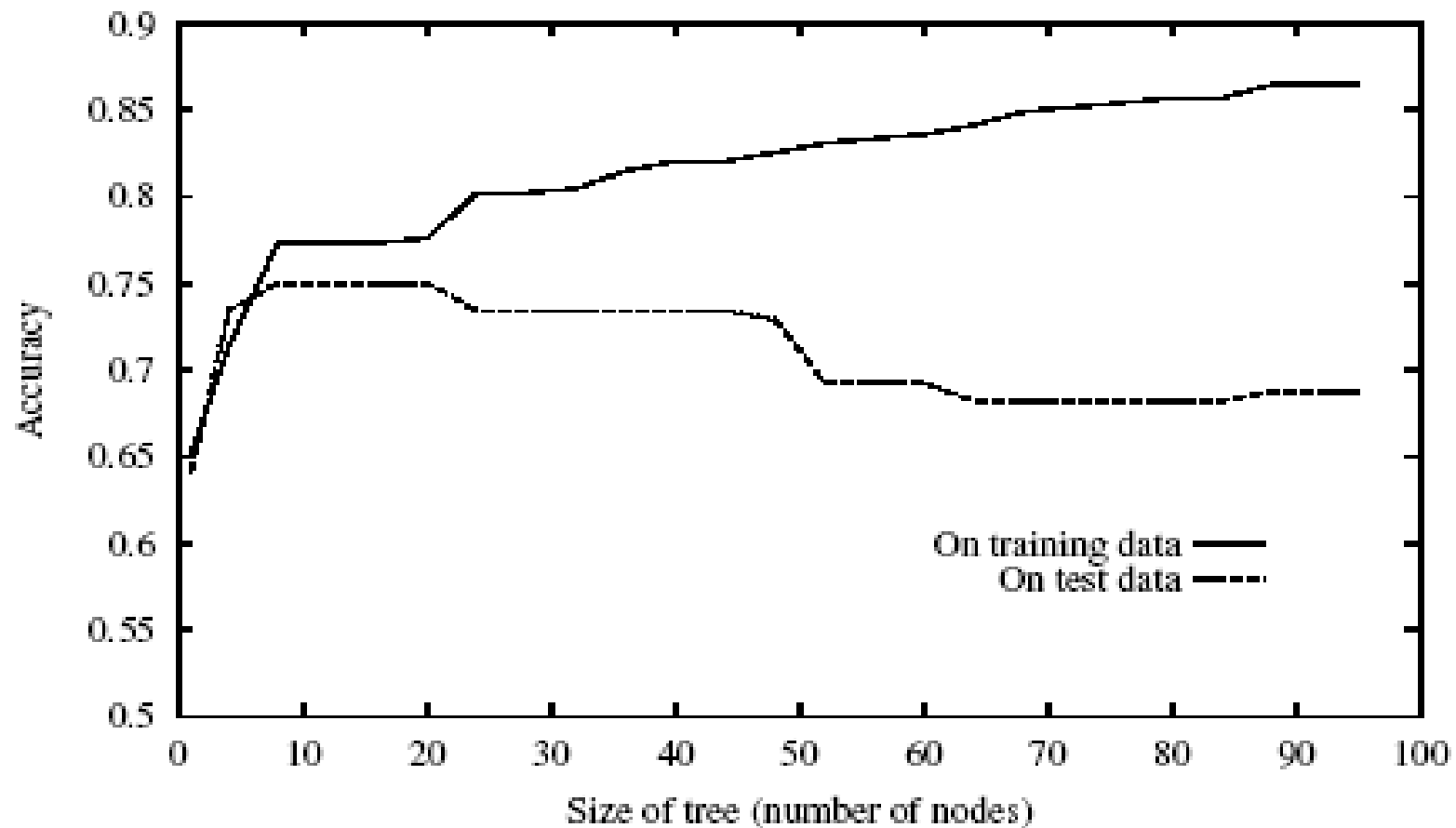
- Los árboles de decisión son fáciles de utilizar y eficientes
- Las reglas que generan son fáciles de interpretar
- Escalan mejor que otros tipos de técnicas
- Tratan bien los datos con ruido

Inconvenientes:

- No manejan de forma fácil los atributos continuos
- Tratan de dividir el dominio de los atributos en regiones rectangulares y no todos los problemas son de ese tipo
- Tienen dificultad para trabajar con valores perdidos
- Pueden tener problemas de sobreaprendizaje
- No detectan correlaciones entre atributos

Árboles de decisión

Ventajas e inconvenientes



Árboles de decisión

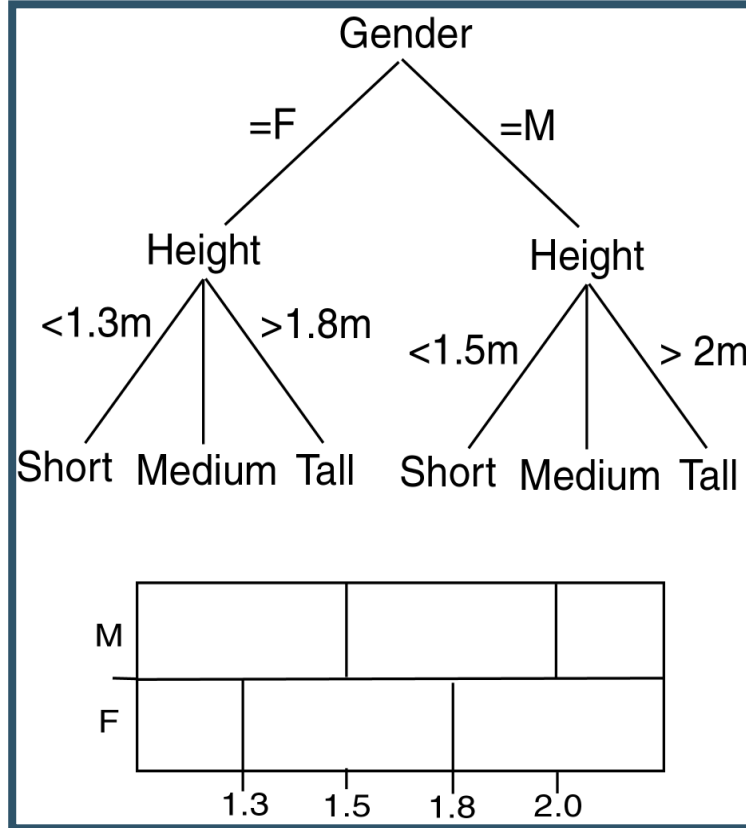
Algunos algoritmos

Aspectos importantes en cualquier algoritmo de MD basado en árboles de decisión:

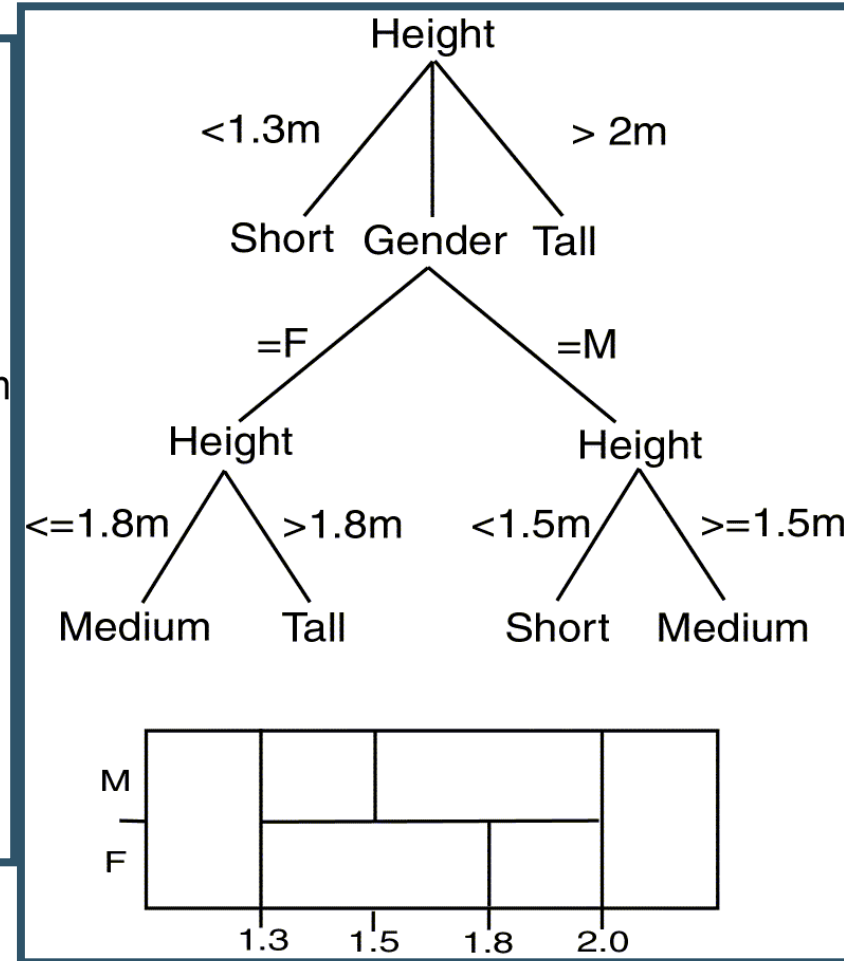
- **Elección de los atributos test:** La forma en que se eligen las variables test tiene gran influencia en el rendimiento del árbol de decisión
 - En ocasiones la elección no sólo implica la revisión de los datos de entrenamiento sino también el uso de la información de expertos
- **Ordenación de los atributos test.** El orden en que se eligen los atributos test es importante para evitar comparaciones innecesarias
- **Divisiones:** Para algunos atributos y dominios las divisiones son obvias mientras que para otros es difícil de determinar
- **Estructura del árbol:** Para mejorar el rendimiento al aplicar el árbol de decisión es preferible un árbol balanceado
- **Criterio de parada.** En muchos problemas puede ser necesario parar antes para prevenir árboles muy grandes. Es un equilibrio entre precisión y rendimiento
- **Ejemplos de entrenamiento.** La estructura del árbol de decisión depende de los ejemplos de entrenamiento
- **Poda:** En ocasiones es necesario realizar un proceso de poda para mejorar el rendimiento del árbol durante la clasificación

Árboles de decisión

Algunos algoritmos



Balanceado



Profundidad

Árboles de decisión

Algunos algoritmos ID3

ID3

J.R. Quinlan. Induction of Decision Trees. Machine Learning, vol. 1, pp 81-106, 1986

- Crea el árbol utilizando conceptos de teoría de información
- Intenta reducir el número de comparaciones
- ID3 elige el atributo test con máxima ganancia de información
 - Basada en la entropía que se utiliza como una medida de la cantidad de incertidumbre o sorpresa en un conjunto de datos

Árboles de decisión

Algunos algoritmos ID3

Definición de **entropía**. Dados:

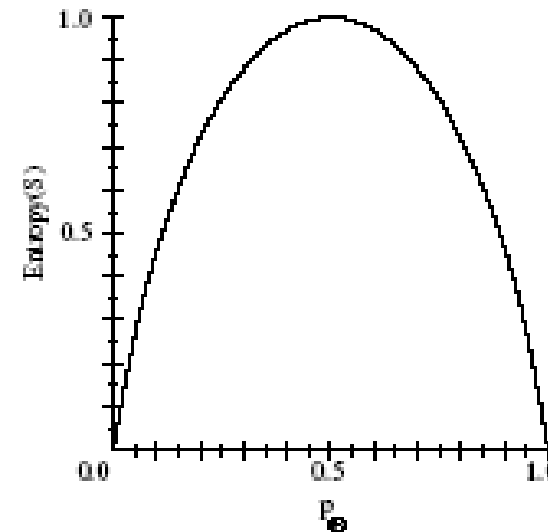
- Un problema con dos clases (positiva y negativa)
- S , un conjunto de ejemplos

La entropía mide la incertidumbre en S

$$\text{Entropia}(S) = H(S) = -p_+ \cdot \log_2 p_+ - p_- \cdot \log_2 p_- = p_+ \cdot \log_2(1/p_+) + p_- \cdot \log_2(1/p_-)$$

- Con k clases:

$$\text{Entropia}(S) = \sum_{i=1}^k p_i \cdot \log_2(1/p_i)$$



Árboles de decisión

Algunos algoritmos ID3

- ID3 elige el atributo con mayor ganancia de información
- Ganancia de información: diferencia entre la cantidad de información que se necesita para hacer una clasificación antes de hacer la división y después
- Se calcula determinando las diferencias entre la entropía del conjunto de datos de partida y la suma ponderada de las entropías una vez dividido el conjunto de ejemplos

$$Gain(S, A) = Entropia(S) - \sum_{v \in valores(A)} \frac{|S_v|}{|S|} Entropia(S_v)$$

Árboles de decisión

Algunos algoritmos ID3

Esquema algoritmo ID3

1. Seleccionar el atributo A que maximice la ganancia $G(S,A)$
 2. Crear un nodo para ese atributo con tantos sucesores como valores tenga
 3. Introducir los ejemplos en los sucesores según el valor que tenga el atributo A
 4. Por cada sucesor:
 - Si sólo hay ejemplos de una clase, C_k
Entonces etiquetarlo con C_k
 - Si no, llamar a ID3 con un conjunto de ejemplos formado por los ejemplos de ese nodo, eliminando la columna del atributo A
- Termina cuando todos los datos del nodo son de la misma clase y la entropía es cero

Árboles de decisión

Algunos algoritmos ID3

Ejemplo:

Name	Gender	Height	Output1
Kristina	F	1.6m	Short
Jim	M	2m	Tall
Maggie	F	1.9m	Medium
Martha	F	1.88m	Medium
Stephanie	F	1.7m	Short
Bob	M	1.85m	Medium
Kathy	F	1.6m	Short
Dave	M	1.7m	Short
Worth	M	2.2m	Tall
Steven	M	2.1m	Tall
Debbie	F	1.8m	Medium
Todd	M	1.95m	Medium
Kim	F	1.9m	Medium
Amy	F	1.8m	Medium
Wynette	F	1.75m	Medium

Árboles de decisión

Algunos algoritmos ID3

1. Inicialmente, la entropía del conjunto es

$$H(S) = \text{Entropia}(S) = 4/15 \log(15/4) + 8/15 \log(15/8) + 3/15 \log(15/3) = 0.4384$$

2. Elegir el atributo test con máxima ganancia de información

Para genero,

$$H(S, \text{gender}=F) = 3/9 \log(9/3) + 6/9 \log(9/6) = 0.2764$$

$$H(S, \text{gender}=M) = 1/6 \log(6/1) + 2/6 \log(6/2) + 3/6 \log(6/3) = 0.4392$$

$$\text{Gain}(S, \text{gender}) = 0.4384 - 9/15 \cdot 0.2764 - 6/15 \cdot 0.4392 = 0.09688$$

Para altura, es necesario dividir en rangos: 1:[0,1.6], 2:(1.6,1.7], 3:[1.7,1.8], 4:(1.8,1.9], 5:(1.9,2.0], 6:(2.0,∞)

Name	Gender	Height	Output1
Kristina	F	1.6m	Short
Maggie	F	1.9m	Medium
Martha	F	1.88m	Medium
Stephanie	F	1.7m	Short
Kathy	F	1.6m	Short
Debbie	F	1.8m	Medium
Kim	F	1.9m	Medium
Amy	F	1.8m	Medium
Wynette	F	1.75m	Medium

Name	Gender	Height	Output1
Jim	M	2m	Tall
Bob	M	1.85m	Medium
Dave	M	1.7m	Short
Worth	M	2.2m	Tall
Steven	M	2.1m	Tall
Todd	M	1.95m	Medium

Árboles de decisión

Algunos algoritmos ID3

Name	Gender	Height	Output1
Kristina	F	1	Short
Jim	M	5	Tall
Maggie	F	4	Medium
Martha	F	4	Medium
Stephanie	F	2	Short
Bob	M	4	Medium
Kathy	F	1	Short
Dave	M	2	Short
Worth	M	6	Tall
Steven	M	6	Tall
Debbie	F	3	Medium
Todd	M	5	Medium
Kim	F	4	Medium
Amy	F	3	Medium
Wynette	F	3	Medium

Árboles de decisión

Algunos algoritmos ID3

$$H(S, \text{height}=1) = 2/2 \log(2/2) = 0$$

$$H(S, \text{height}=2) = 2/2 \log(2/2) = 0$$

$$H(S, \text{height}=3) = 3/3 \log(3/3) = 0$$

$$H(S, \text{height}=4) = 4/4 \log(4/4) = 0$$

$$H(S, \text{height}=5) = 1/2 \log 2 + 1/2 \log 2 = 0.301$$

$$H(S, \text{height}=6) = 2/2 \log(2/2) = 0$$

$$\text{Gain}(S, \text{height}) = 0.4384 - 2/15 \cdot 0.301 = 0.3983$$

Es mayor, por lo que se elige el atributo height

Name	Gender	Height	Output1
Kristina	F	1	Short
Kathy	F	1	Short

Name	Gender	Height	Output1
Stephanie	F	2	Short
Dave	M	2	Short

Name	Gender	Height	Output1
Debbie	F	3	Medium
Amy	F	3	Medium
Wynette	F	3	Medium

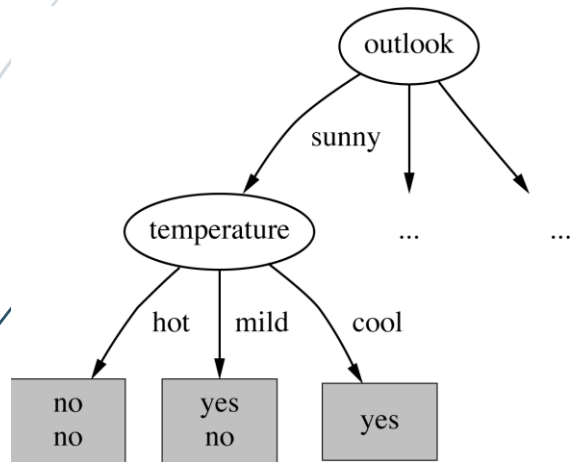
Name	Gender	Height	Output1
Maggie	F	4	Medium
Martha	F	4	Medium
Bob	M	4	Medium
Kim	F	4	Medium

Name	Gender	Height	Output1
Jim	M	5	Tall
Todd	M	5	Medium

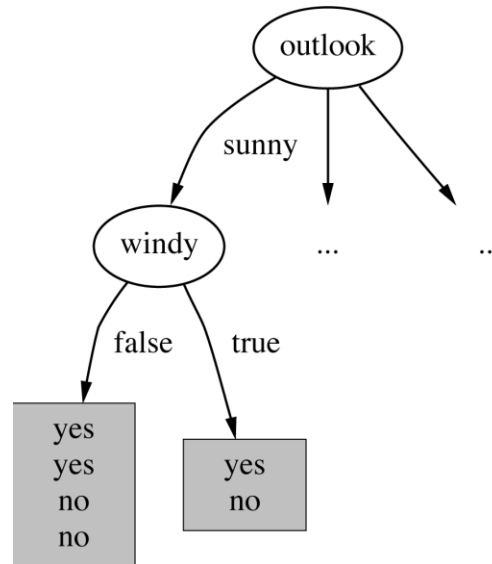
Árboles de decisión

Algunos algoritmos ID3

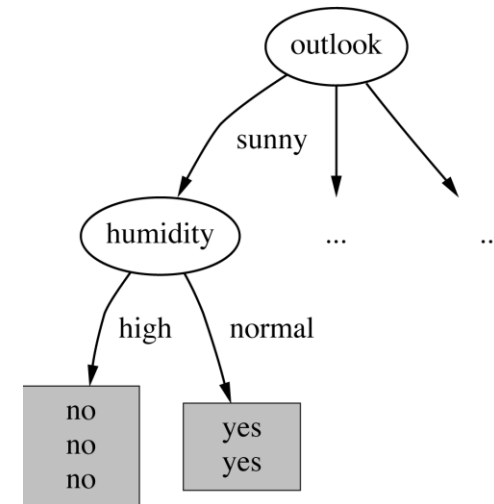
Ejemplo 2: Jugar al tenis



$\text{gain}(\text{" Temperature"}) = 0.571$



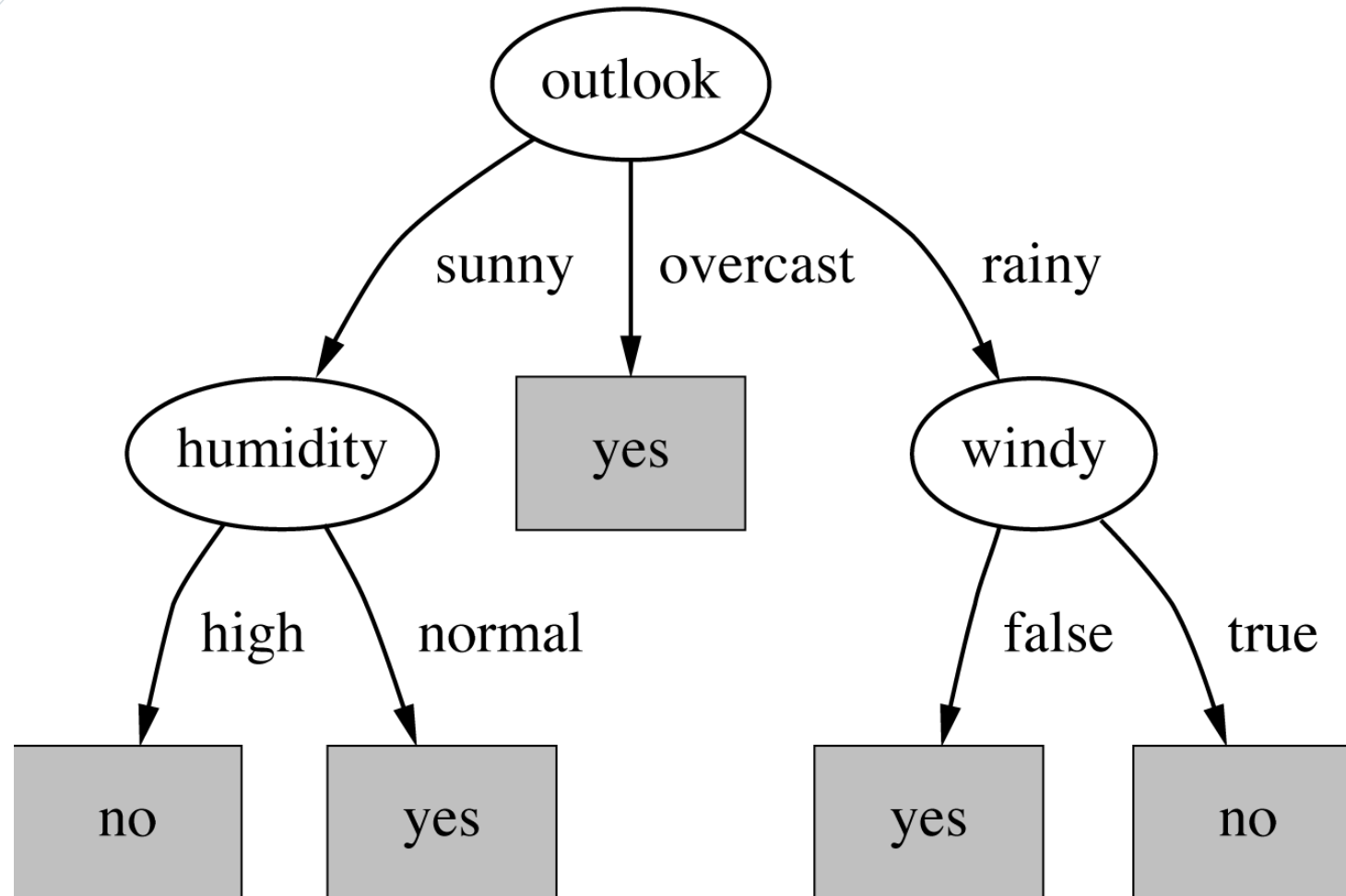
$\text{gain}(\text{" Windy"}) = 0.020$



$\text{gain}(\text{" Humidity"}) = 0.971$

Árboles de decisión

Algunos algoritmos ID3



Árboles de decisión

Algunos algoritmos ID3

- El espacio de hipótesis es completo, la función objetivo está incluida en él
- Selecciona una única hipótesis
 - No es capaz de determinar todos los árboles compatibles con los ejemplos de entrenamiento
 - No puede proponer ejemplos que reduzcan el espacio de búsqueda
- No hay vueltas atrás
 - Encuentra un óptimo local que puede no ser el óptimo global (hill-climbing)
- En cada paso utiliza información estadística de todos los ejemplos en lugar de considerar los ejemplos uno a uno
 - Permite ruido en los datos de entrenamiento
- Por la ganancia de información, tiene tendencia a elegir atributos con muchos valores

Árboles de decisión

Algunos algoritmos ID3

De todos los árboles compatibles con los ejemplos de entrenamiento ¿Cuál elige ID3 ?

- Se prefieren árboles cortos frente a largos, con los atributos que producen una mayor ganancia de información cerca de la raíz

Refinamiento de ID3

- Cuándo parar en la construcción del árbol
- Atributos continuos
- Otras medidas de selección de atributos
- Atributos con coste diferente
- Ejemplos incompletos

Árboles de decisión

Algunos algoritmos C4.5

C4.5

J.R. Quinlan. C4.5: Programs for Machine Learning. San Francisco: Morgan Kaufmann, 1993

Mejora a ID3 en los siguientes aspectos:

- **Datos perdidos:**
 - Cuando se construye el árbol, los datos perdidos se ignoran (el árbol de decisión se construye mirando sólo los registros que tienen valor para ese atributo)
 - Para clasificar un ejemplo con valor perdido, éste se predice en base a lo que se sabe sobre los valores del atributo para otros registros
- **Datos continuos:** Se divide en rangos en base a los valores encontrados en el conjunto de entrenamiento
- Propone soluciones para el sobreaprendizaje. Posibilidades
 - pre-poda: se decide cuándo dejar de subdividir el árbol
 - post-poda: se construye el árbol y después se poda

Árboles de decisión

Algunos algoritmos C4.5

- Reglas: C4.5 permite clasificación mediante el árbol o a través de las reglas que se generan a partir de él
 - Además, incluye técnicas para simplificar las reglas
- Selección de atributos:
 - ID3 favorece atributos con muchas divisiones
 - En C4.5 se utiliza como criterio de selección el ratio de ganancia de información que tiene en cuenta la cardinalidad de cada división

$$GainRatio(S, A) = \frac{Gain(S, A)}{H\left(\frac{|S_1|}{|S|}, \dots, \frac{|S_v|}{|S|}\right)}$$

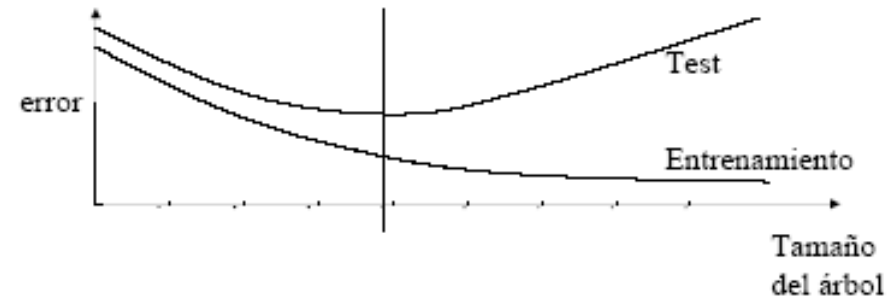
$v \in \text{Valores de } A$

Árboles de decisión

Algunos algoritmos C4.5

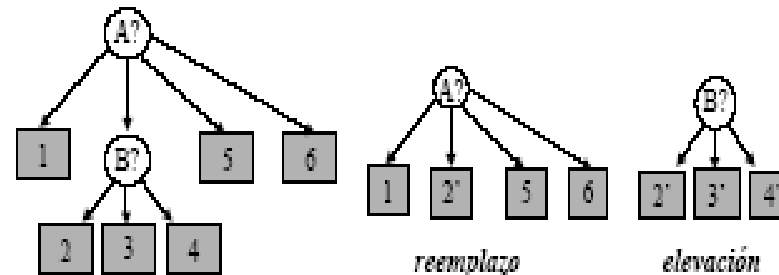
Pre-poda:

- no se divide un nodo si se tiene poca confianza en él (no es significativa la diferencia de clases), o
- se valida con un conjunto de test independiente y se para cuando la curva del conjunto de test empieza a subir



Hay dos estrategias de **post- poda** en C4.5

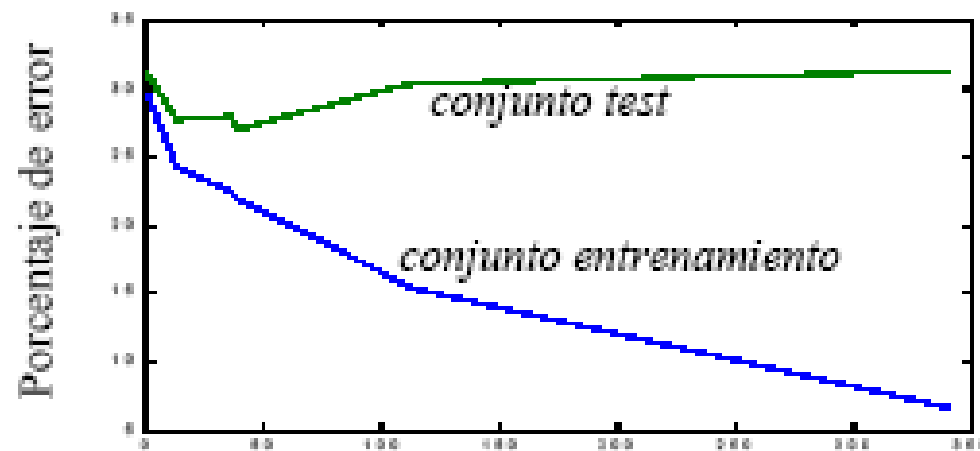
- Reemplazamiento de subárboles: Se reemplaza un subárbol por una hoja si al hacerlo el error es similar al original
- Elevación de subárbol: Reemplaza un subárbol por su subárbol más utilizado (un subárbol se mueve de su localización a un nodo superior en el árbol)



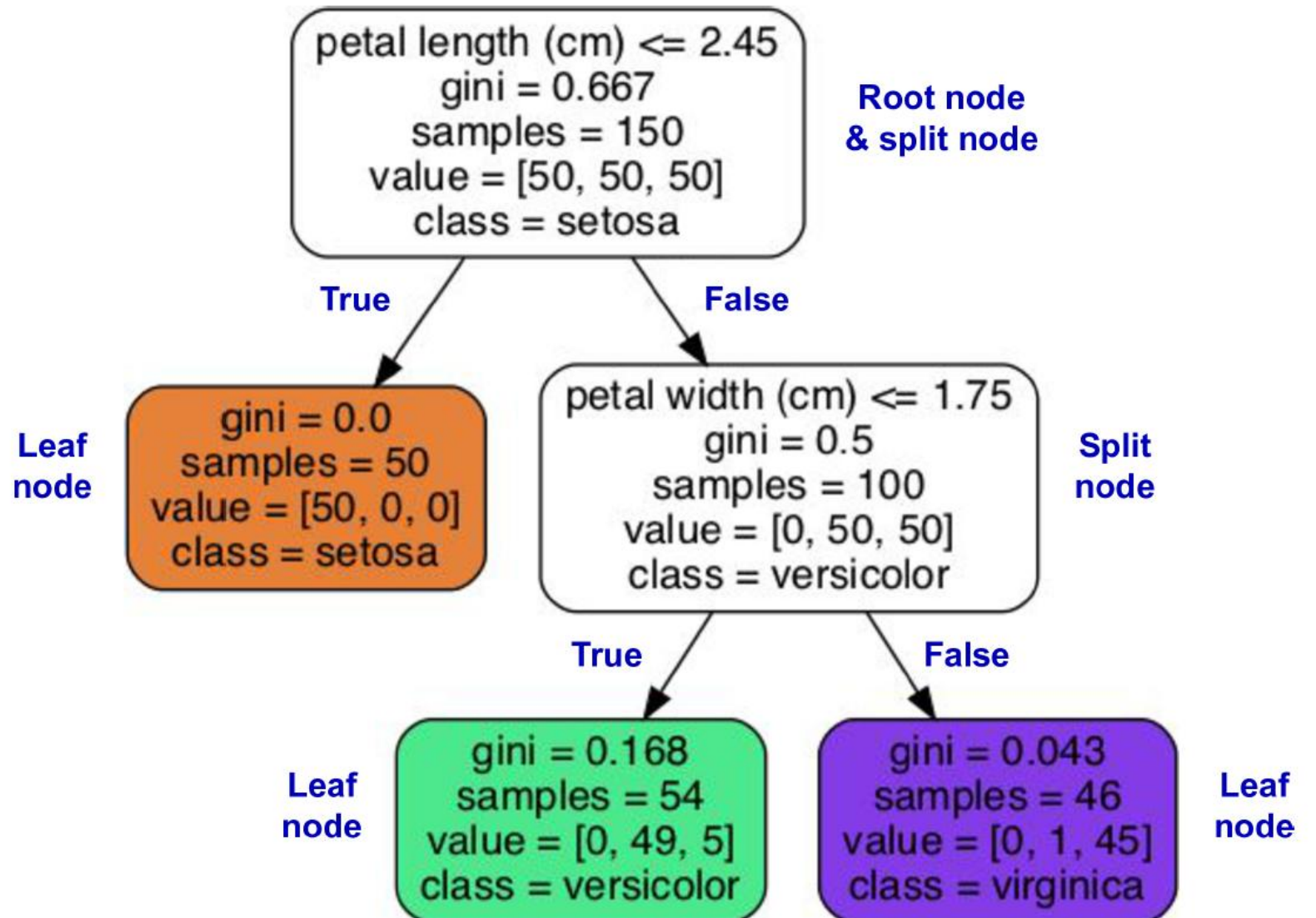
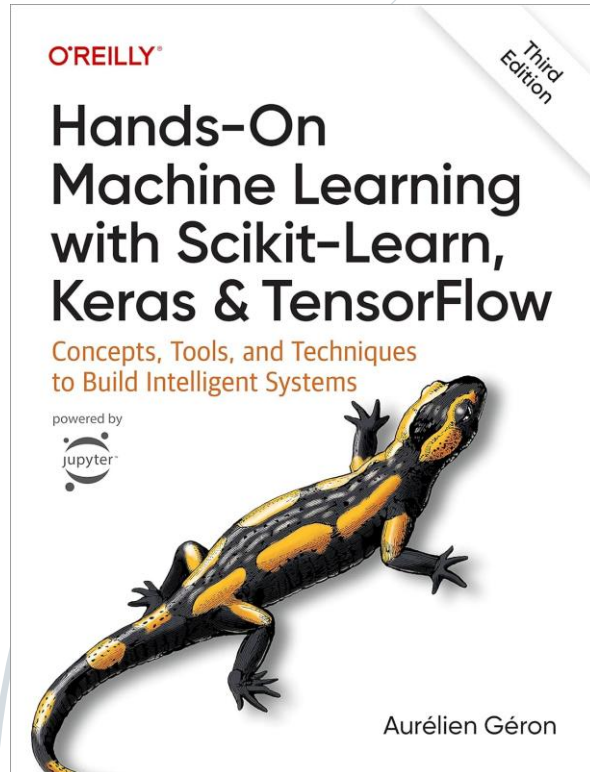
Árboles de decisión Algunos algoritmos C4.5

Efecto del tamaño del árbol

Tamaño del árbol	Conjunto de entrenamiento		Conjunto de test	
	instancias incorrectas	porcentaje de error	instancias incorrectas	porcentaje de error
1	207	29.57 %	93	31%
13	170	24.29 %	83	27.67 %
38	157	22.43%	84	28%
39	154	22%	81	27%
95	119	17%	89	29.67%
113	108	15.43%	91	30.3%
340	47	6.71%	94	31.3%



Árboles de decisión



Predicciones

- Veamos cómo hace predicciones el árbol representado en la figura anterior. Supongamos que encuentra una flor de iris y quiere clasificarla en función de sus pétalos. Se empieza por el *nodo raíz* (profundidad 0, en la parte superior): este nodo pregunta si la longitud de los pétalos de la flor es inferior a 2,45 cm. En caso afirmativo, se desciende hasta el nodo hijo izquierdo de la raíz (profundidad 1, izquierda). En este caso, es un *nodo hoja* (es decir, no tiene ningún nodo hijo), por lo que no hace ninguna pregunta: simplemente mire la clase predicha para ese nodo, y el árbol de decisión predice que su flor es una *Iris setosa*.
- Suponga ahora que encuentra otra flor, y que esta vez la longitud del pétalo es superior a 2,45 cm. Comienza de nuevo en la raíz, pero ahora desciende hasta su nodo hijo derecho (profundidad 1, derecha). Este no es un nodo hoja, es un *nodo partido*, por lo que hace otra pregunta: ¿la anchura del pétalo es menor de 1,75 cm? Si es así, lo más probable es que su flor sea un *Iris versicolor* (profundidad 2, izquierda). En caso contrario, es probable que se trate de un *Iris virginica* (profundidad 2, derecha). Así de sencillo.

Predicciones

- El atributo *samples* de un nodo cuenta a cuántas instancias de entrenamiento se aplica. Por ejemplo, 100 instancias de entrenamiento tienen una longitud de pétalo superior a 2,45 cm (profundidad 1, derecha), y de esas 100, 54 tienen una anchura de pétalo inferior a 1,75 cm (profundidad 2, izquierda).
- El atributo *value* de un nodo te indica a cuántas instancias de entrenamiento de cada clase se aplica este nodo: por ejemplo, el nodo inferior derecho se aplica a 0 *Iris setosa*, 1 *Iris versicolor* y 45 *Iris virginica*.
- El atributo *gini* de un nodo mide su *impureza de Gini*: un nodo es "puro" ($gini=0$) si todas las instancias de entrenamiento a las que se aplica pertenecen a la misma clase. Por ejemplo, dado que el nodo izquierdo de profundidad 1 sólo se aplica a las instancias de entrenamiento de *Iris setosa*, es puro y su impureza de Gini es 0.

Predicciones

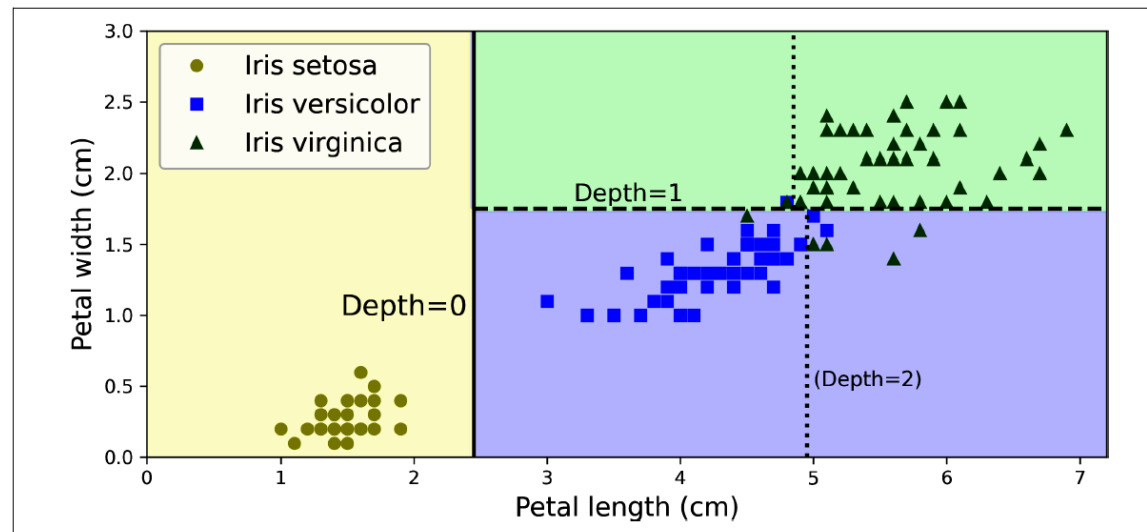
- La siguiente ecuación muestra cómo el algoritmo de entrenamiento calcula la impureza de Gini G_i del nodo i^{th} . El nodo izquierdo de profundidad 2 tiene una impureza de Gini igual a $1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0,168$.

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- G_i es la impureza Gini del nodo i^{th} .
- $p_{i,k}$ es la proporción de instancias de clase k entre las instancias de entrenamiento en el nodo i^{th} .
- Scikit-Learn utiliza el algoritmo CART, que sólo produce árboles binarios, es decir, árboles en los que los nodos divididos siempre tienen exactamente dos hijos. Sin embargo, otros algoritmos, como ID3, C4.5, See5; pueden producir árboles de decisión con nodos que tienen más de dos hijos.

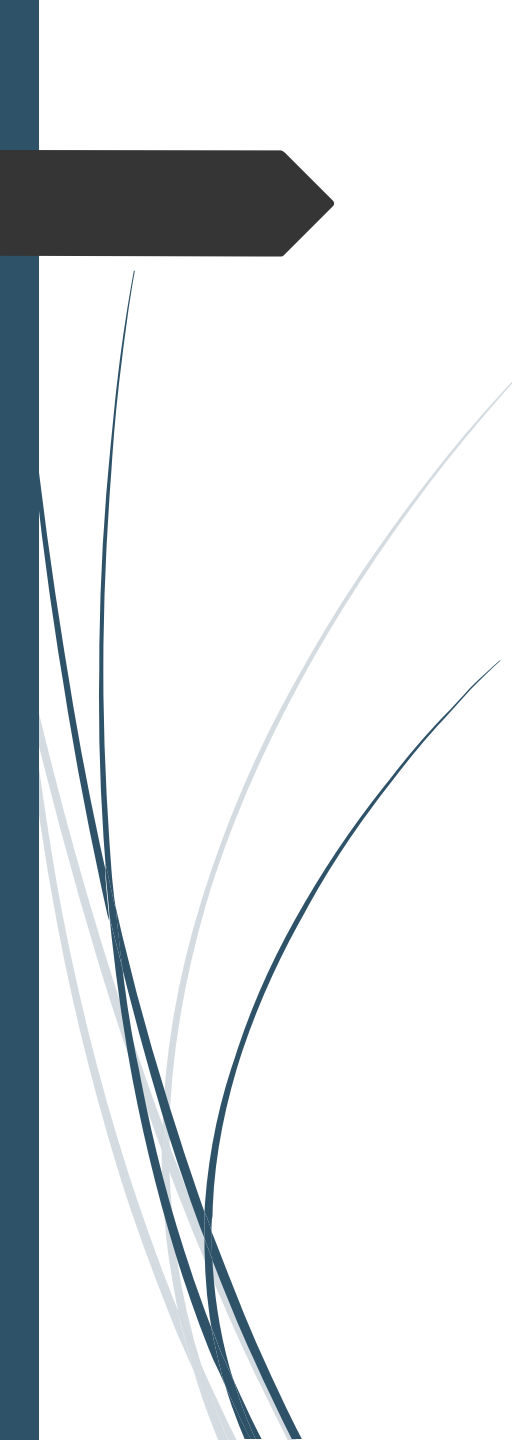
Predicciones

- Esta figura muestra los límites de decisión de este árbol de decisión. La línea vertical gruesa representa el límite de decisión del nodo raíz (profundidad 0): longitud del pétalo = 2,45 cm. Como la zona izquierda es pura (sólo *Iris setosa*), no puede dividirse más. Sin embargo, la zona derecha es impura, por lo que el nodo derecho de profundidad 1 la divide a una anchura de pétalo = 1,75 cm (representada por la línea discontinua). Como *max_depth* se ha establecido en 2, el árbol de decisión se detiene justo ahí. Si fijara *max_depth* en 3, entonces los dos nodos de profundidad-2 añadirían cada uno otro límite de decisión (representado por las dos líneas punteadas verticales).



Interpretación de modelos: Caja blanca frente a caja negra

- Los árboles de decisión son intuitivos y sus decisiones son fáciles de interpretar.
- Estos modelos suelen denominarse modelos de caja blanca. En cambio, como verás, los random forests y las redes neuronales suelen considerarse modelos de caja negra. Hacen grandes predicciones, y puedes comprobar fácilmente los cálculos que realizaron para hacer estas predicciones; sin embargo, suele ser difícil explicar en términos sencillos por qué se hicieron las predicciones.
- Por ejemplo, si una red neuronal dice que una persona concreta aparece en una foto, es difícil saber qué contribuyó a esta predicción: ¿Reconoció el modelo los ojos de esa persona? ¿la boca? ¿la nariz? ¿Sus zapatos? ¿O incluso el sofá en el que estaba sentada?
- Por el contrario, los árboles de decisión proporcionan reglas de clasificación sencillas y agradables que incluso pueden aplicarse manualmente si es necesario (por ejemplo, para la clasificación de flores). El campo del ML interpretable tiene como objetivo crear sistemas de ML que puedan explicar sus decisiones de forma comprensible para los humanos. Esto es importante en muchos ámbitos, por ejemplo, para garantizar que el sistema no tome decisiones injustas.



Estimación de las probabilidades de clase

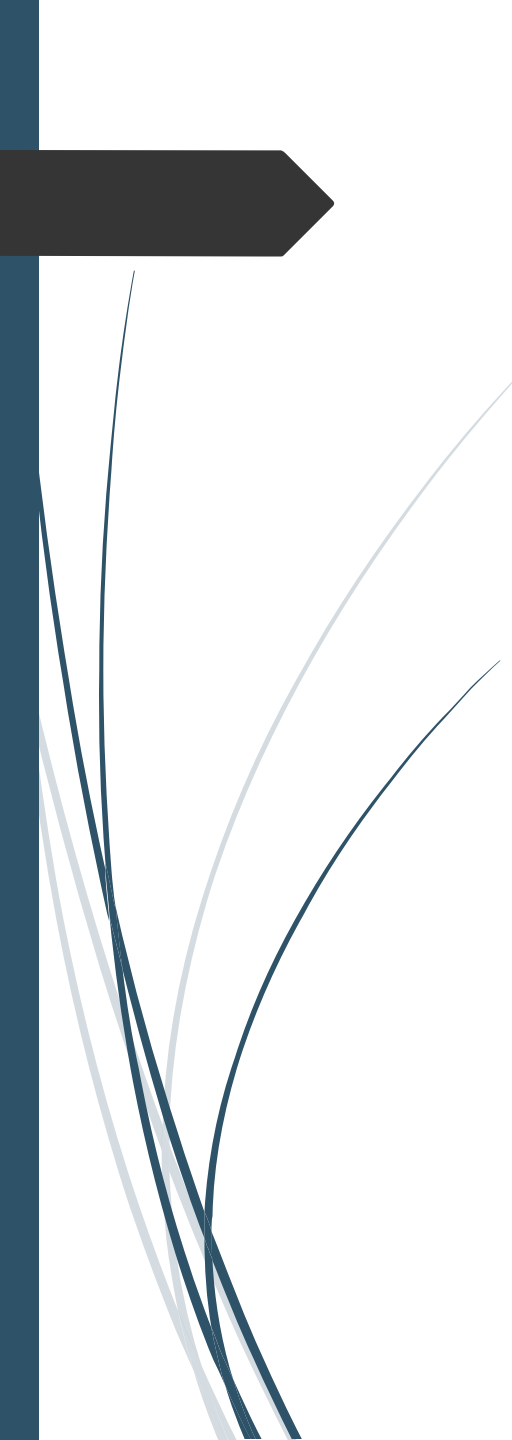
- Un árbol de decisión también puede estimar la probabilidad de que una instancia pertenezca a una determinada clase k .
- En primer lugar, recorre el árbol para encontrar el nodo hoja de esta instancia y, a continuación, devuelve la proporción de instancias de entrenamiento de la clase k en este nodo.

Algoritmo de entrenamiento CART

- Scikit-Learn utiliza el algoritmo *Classification and Regression Tree* (CART) para entrenar árboles de decisión. El algoritmo funciona dividiendo primero el conjunto de entrenamiento en dos subconjuntos utilizando una única característica k y un umbral t_k (por ejemplo, "petal length $\leq 2,45$ cm").
- ¿Cómo elige k y t_k ? Busca el par (k, t_k) que produce los subconjuntos más puros, ponderados por su tamaño. La siguiente ecuación da la función de coste que el algoritmo intenta minimizar:

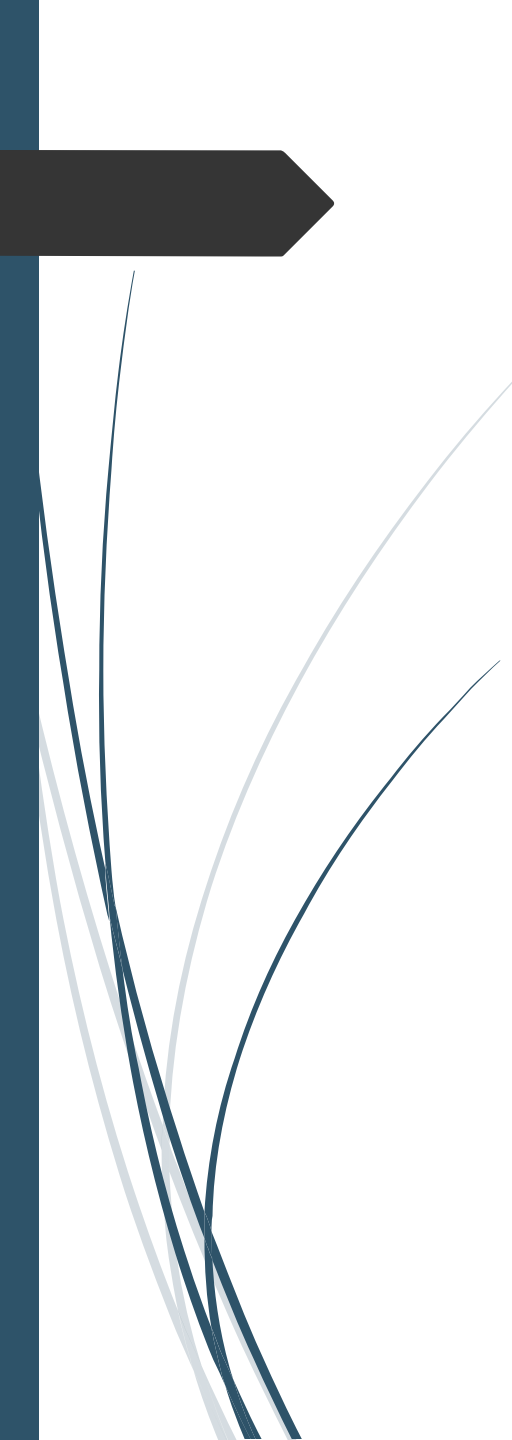
$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

where $\begin{cases} G_{\text{left/right}} \text{ measures the impurity of the left/right subset} \\ m_{\text{left/right}} \text{ is the number of instances in the left/right subset} \end{cases}$



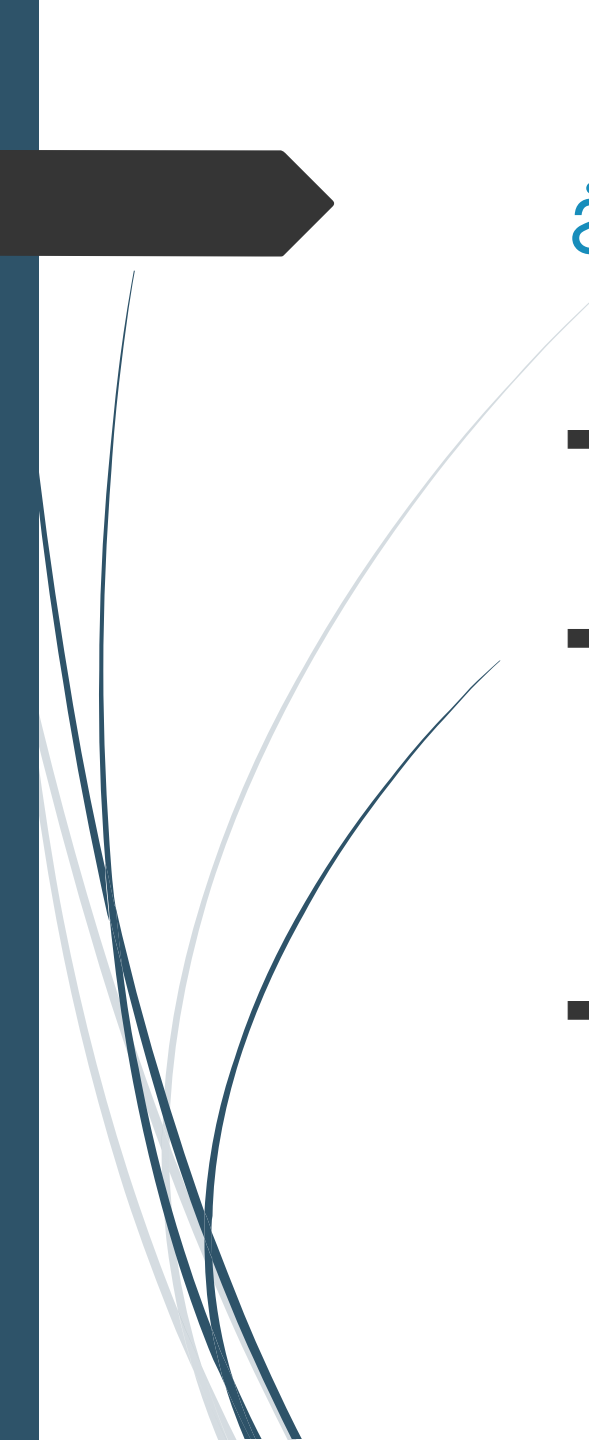
Algoritmo de entrenamiento CART

- Una vez que el algoritmo CART ha dividido con éxito el conjunto de entrenamiento en dos, divide los subconjuntos utilizando la misma lógica, luego los subsubconjuntos, y así sucesivamente, de forma recursiva.
- Deja de buscar una vez que alcanza la profundidad máxima (definida por el hiperparámetro *max_depth*), o si no puede encontrar una división que reduzca la impureza.
- Algunos otros hiperparámetros controlan condiciones de parada adicionales: *min_samples_split*, *min_samples_leaf*, *min_weight_fraction_leaf* y *max_leaf_nodes*.
- El algoritmo CART es un *algoritmo greedy*: no comprueba si la división conducirá o no a la impureza más baja posible varios niveles más abajo. Un algoritmo greedy suele producir una solución razonablemente buena, pero no garantiza que sea la óptima.
- Por desgracia, se sabe que encontrar el árbol óptimo es un problema *NP-completo*. Requiere un tiempo $O(\exp(m))$, lo que hace que el problema sea intratable incluso para conjuntos de entrenamiento pequeños.



Complejidad computacional

- Para hacer predicciones hay que recorrer el árbol de decisión desde la raíz hasta una hoja. Los árboles de decisión suelen estar aproximadamente equilibrados, por lo que recorrerlos requiere pasar por aproximadamente $O(\log_2(m))$ nodos.
- Dado que cada nodo sólo requiere comprobar el valor de una característica, la complejidad global de la predicción es $O(\log_2(m))$, independientemente del número de características.
- Las predicciones son muy rápidas, incluso cuando se trata de grandes conjuntos de entrenamiento.
- El algoritmo de entrenamiento compara todas las características (o menos si se establece *max_features*) en todas las muestras de cada nodo. La comparación de todas las características en todas las muestras de cada nodo da como resultado una complejidad de entrenamiento de $O(n \times m \log_2(m))$.



¿Impureza o entropía de Gini?

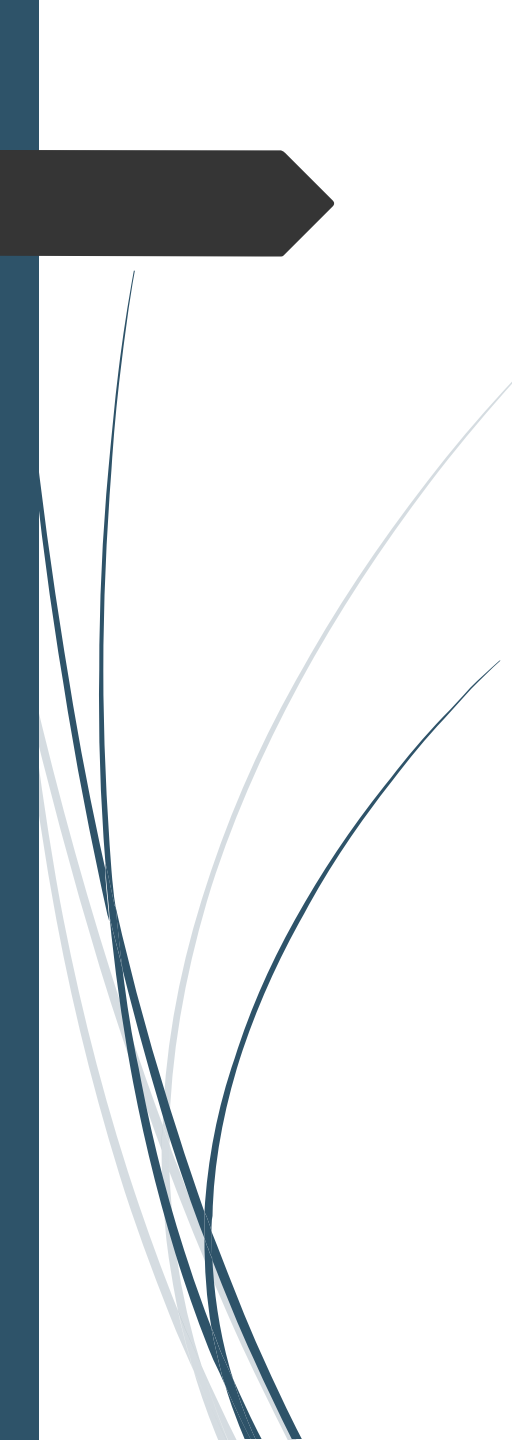
- Por defecto, la clase *DecisionTreeClassifier* utiliza la medida de impureza de Gini, pero puede seleccionar la medida de impureza de entropía en su lugar estableciendo el hiperparámetro de *criterion* en "entropy".
- El concepto de entropía se originó en la termodinámica como medida del desorden molecular: la entropía se aproxima a cero cuando las moléculas están quietas y bien ordenadas. Posteriormente, la entropía se extendió a una gran variedad de ámbitos, incluida la teoría de la información de Shannon, donde mide el contenido medio de información de un mensaje. La entropía es cero cuando todos los mensajes son idénticos.
- En el aprendizaje automático, la entropía se utiliza con frecuencia como medida de impureza: la entropía de un conjunto es cero cuando contiene instancias de una sola clase.

¿Impureza o entropía de Gini?

- La siguiente ecuación muestra la definición de la entropía del nodo i^{th} . Por ejemplo, el nodo izquierdo de profundidad 2 de la figura anterior tiene una entropía igual a $-(49/54) \log_2 (49/54) - (5/54) \log_2 (5/54) \approx 0,445$.

$$H_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log_2 (p_{i,k})$$

- ¿Hay que utilizar la impureza de Gini o la entropía?
 - La verdad es que la mayoría de las veces no hay gran diferencia: conducen a árboles similares.
 - La impureza de Gini es ligeramente más rápida de calcular, por lo que es una buena opción por defecto.
 - Sin embargo, cuando difieren, la impureza de Gini tiende a aislar la clase más frecuente en su propia rama del árbol, mientras que la entropía tiende a producir árboles ligeramente más equilibrados.



Hiperparámetros de regularización

- Los árboles de decisión hacen muy pocas suposiciones sobre los datos de entrenamiento.
- Si no se impone ninguna restricción, la estructura del árbol se adaptará a los datos de entrenamiento y se ajustará mucho a ellos; de hecho, lo más probable es que se sobreajuste.
- Un modelo de este tipo suele denominarse *modelo no paramétrico*, porque el número de parámetros no se determina antes del entrenamiento, por lo que la estructura del modelo es libre de ceñirse a los datos. En cambio, un *modelo paramétrico*, como un modelo lineal, tiene un número predeterminado de parámetros, por lo que su grado de libertad es limitado, lo que reduce el riesgo de sobreajuste (pero aumenta el riesgo de infraajuste).
- Para evitar el sobreajuste de los datos de entrenamiento, es necesario restringir la libertad del árbol de decisión durante el entrenamiento.
- Los hiperparámetros de regularización dependen del algoritmo utilizado, pero en general se puede al menos restringir la profundidad máxima del árbol de decisión.
- En Scikit-Learn, esto se controla mediante el hiperparámetro *max_depth*. El valor por defecto es *None*, que significa ilimitado. Reducir la *max_depth* regularizará el modelo y reducirá el riesgo de sobreajuste.

Hiperparámetros de regularización

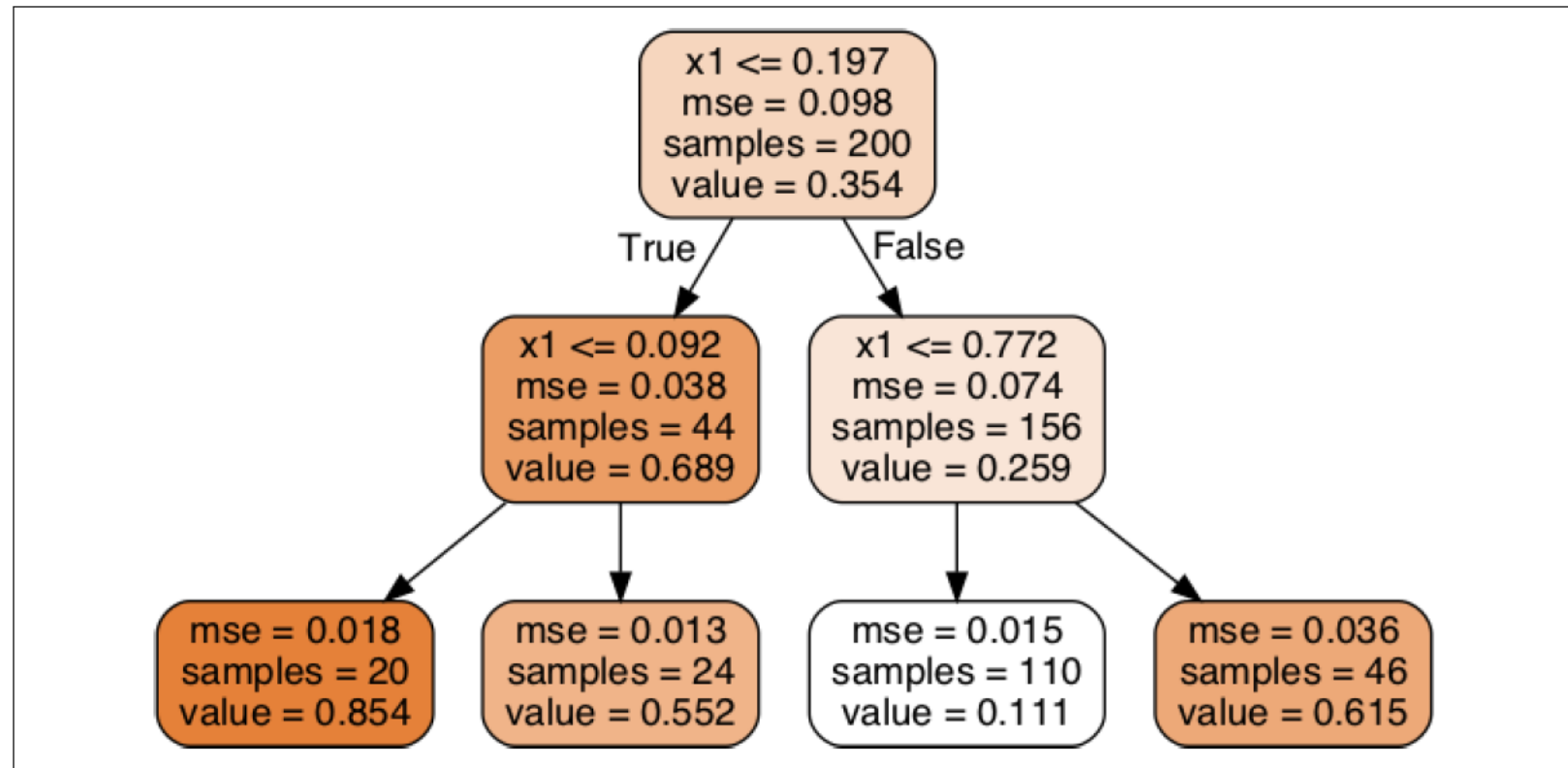
La clase *DecisionTreeClassifier* tiene algunos otros parámetros que restringen de forma similar la forma del árbol de decisión:

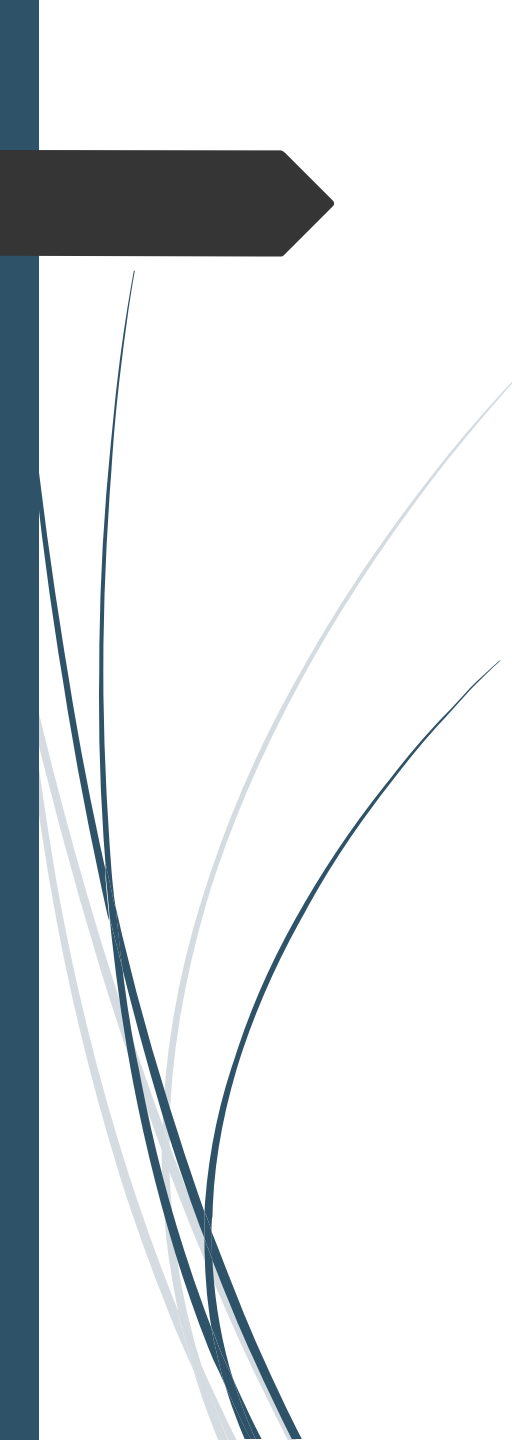
- *max_features*
 - Número máximo de características que se evalúan para la división en cada nodo
- *max_leaf_nodes*
 - Número máximo de nodos hoja
- *min_samples_split*
 - Número mínimo de muestras que debe tener un nodo para poder dividirse
- *min_samples_leaf*
 - Número mínimo de muestras que debe tener un nodo hoja para ser creado
- *min_weight_fraction_leaf*
 - Igual que *min_samples_leaf* pero expresado como fracción del número total de instancias ponderadas

Aumentar los hiperparámetros *min_** o reducir los hiperparámetros *max_** regularizará el modelo.

Regresión

- Los árboles de decisión también son capaces de realizar tareas de regresión.



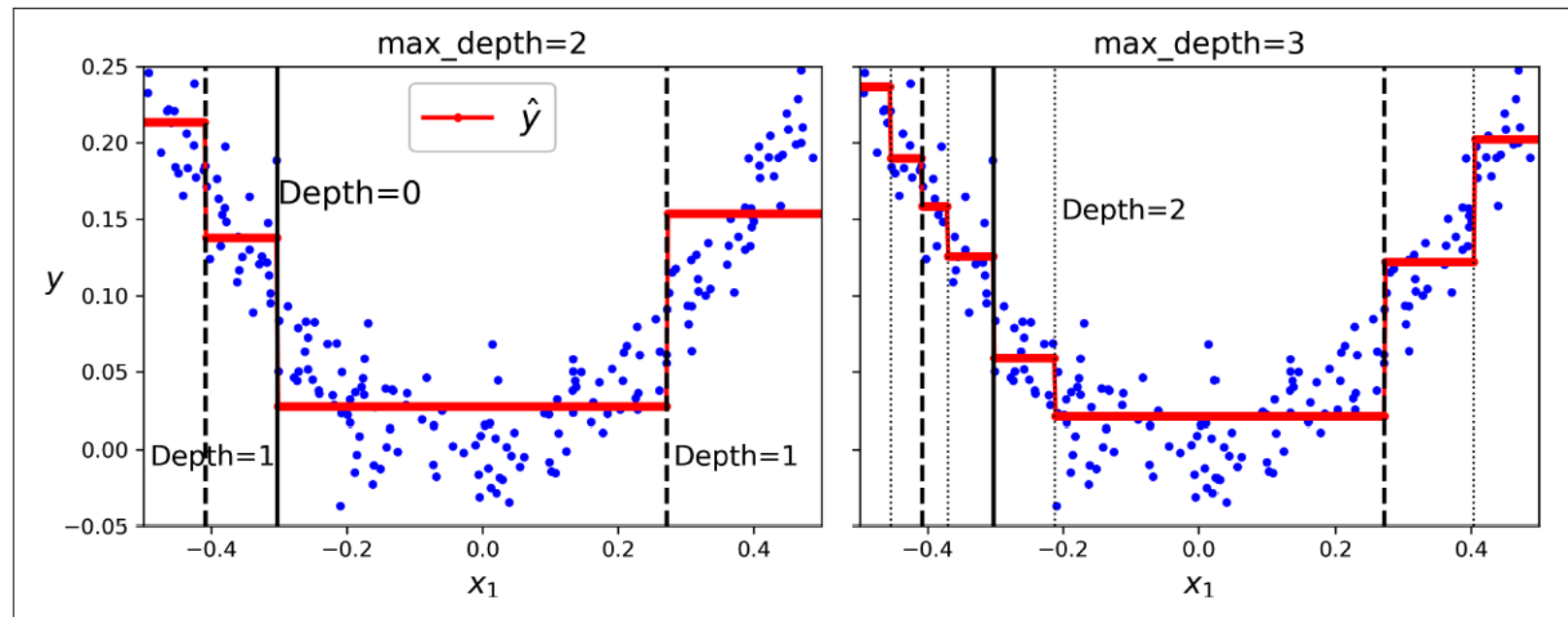


Regresión

- Este árbol se parece mucho al árbol de clasificación que construyó anteriormente.
- La principal diferencia es que en lugar de predecir una clase en cada nodo, predice un valor.
- Por ejemplo, supongamos que desea hacer una predicción para una nueva instancia con $x_1 = 0,2$. El nodo raíz pregunta si $x_1 \leq 0,197$. Si no lo es, el algoritmo pasa al nodo hijo de la derecha, que pregunta si $x_1 \leq 0,772$. Si lo es, el algoritmo pasa al nodo hijo de la derecha, que pregunta si $x \leq 0,772$. Si lo es, el algoritmo pasa al nodo hijo izquierdo. Éste es un nodo hoja y predice el valor $= 0,111$.
- Esta predicción es el valor objetivo medio de las 110 instancias de entrenamiento asociadas a este nodo hoja, y da como resultado un error cuadrático medio igual a 0,015 sobre estas 110 instancias.

Regresión

- Las predicciones de este modelo se representan en la siguiente figura.
- Observe cómo el valor predicho para cada región es siempre el valor objetivo medio de las instancias de esa región. El algoritmo divide cada región de forma que la mayoría de las instancias de entrenamiento se acerquen lo máximo posible a ese valor predicho.



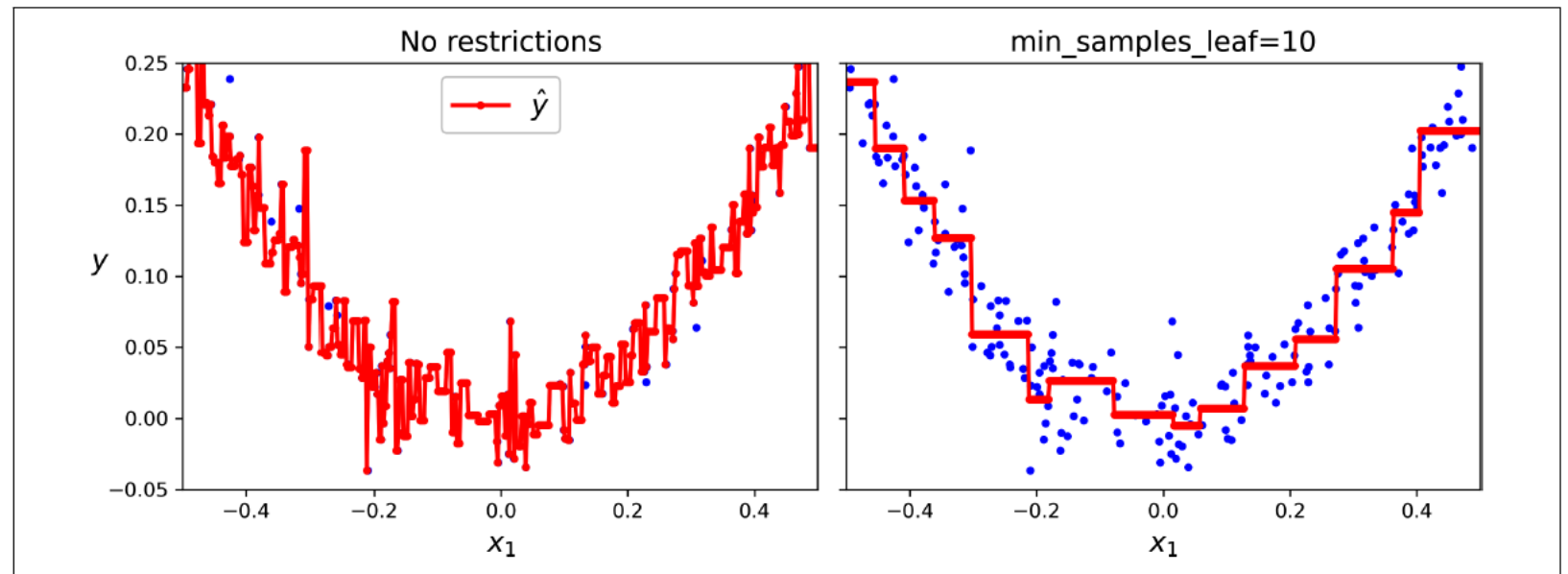
Regresión

- El algoritmo CART trabaja dividiendo el conjunto de entrenamiento de forma que se minimice el MSE. La siguiente ecuación muestra la función de coste que el algoritmo intenta minimizar.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \quad \text{where} \quad \left\{ \begin{array}{l} \text{MSE}_{\text{node}} = \frac{\sum_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2}{m_{\text{node}}} \\ \hat{y}_{\text{node}} = \frac{\sum_{i \in \text{node}} y^{(i)}}{m_{\text{node}}} \end{array} \right.$$

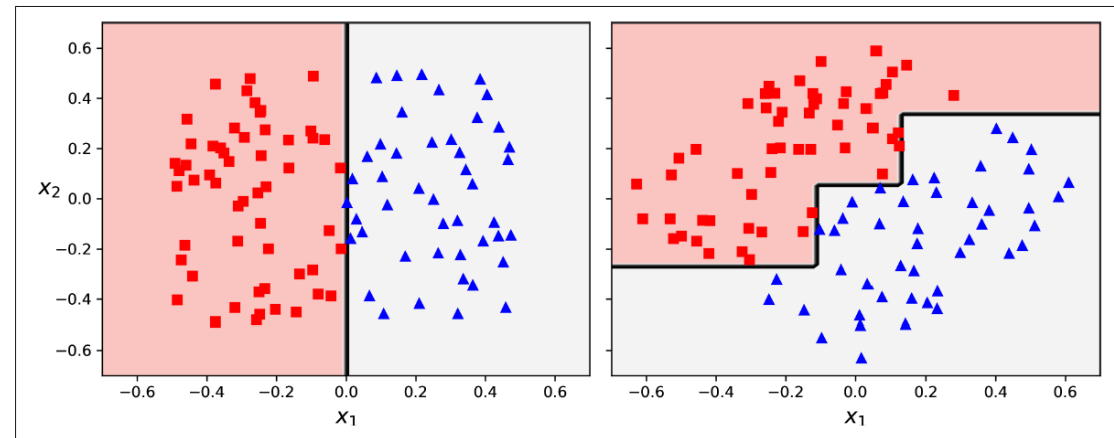
Regresión

- Al igual que en las tareas de clasificación, los árboles de decisión son propensos al sobreajuste cuando se trata de tareas de regresión.
- Sin ninguna regularización (es decir, utilizando los hiperparámetros por defecto), se obtienen las predicciones de la izquierda.
- Obviamente, estas predicciones se ajustan en exceso al conjunto de entrenamiento. Basta con establecer *min_samples_leaf=10* para obtener un modelo mucho más razonable.



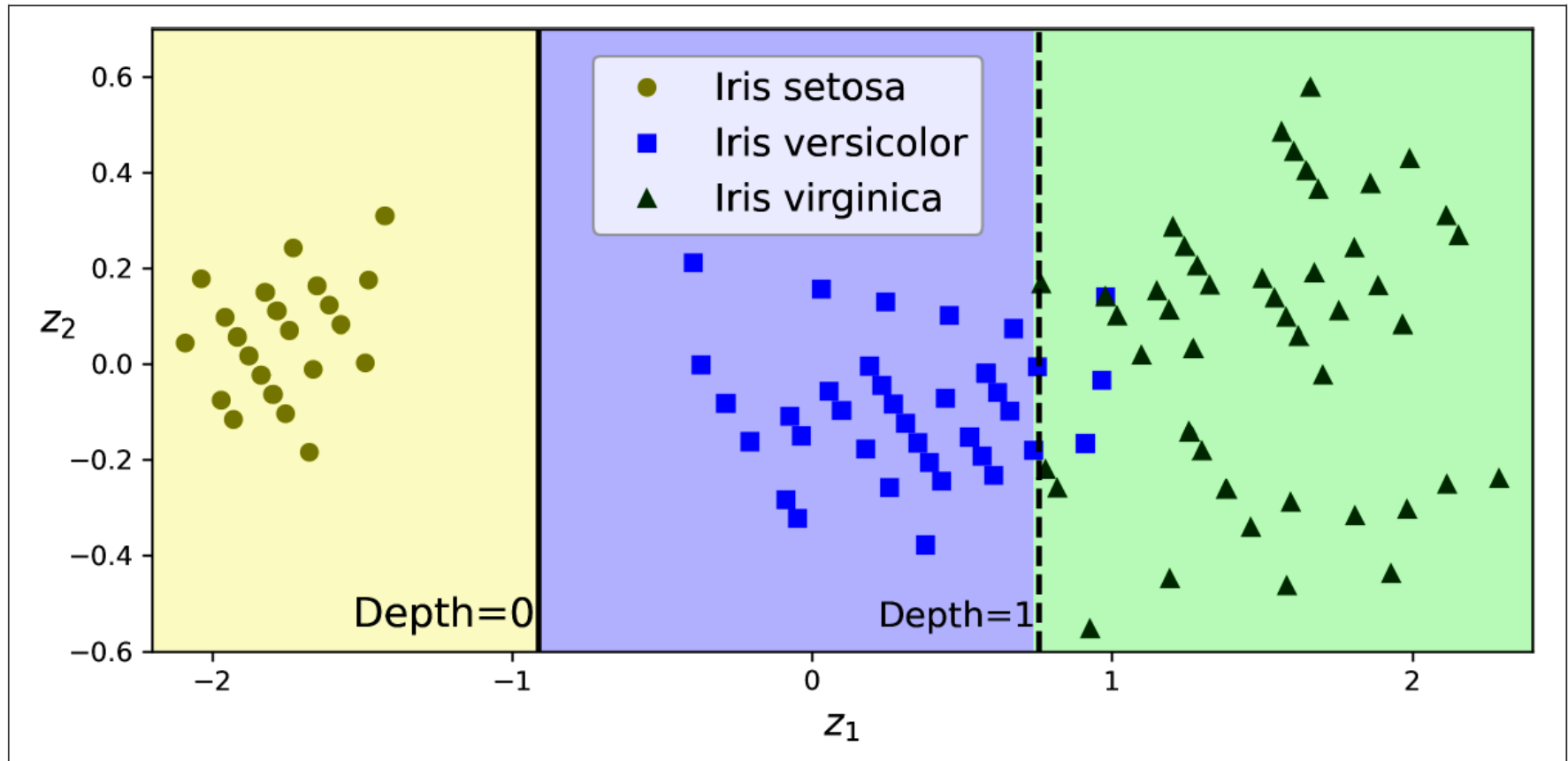
Sensibilidad a la orientación del eje

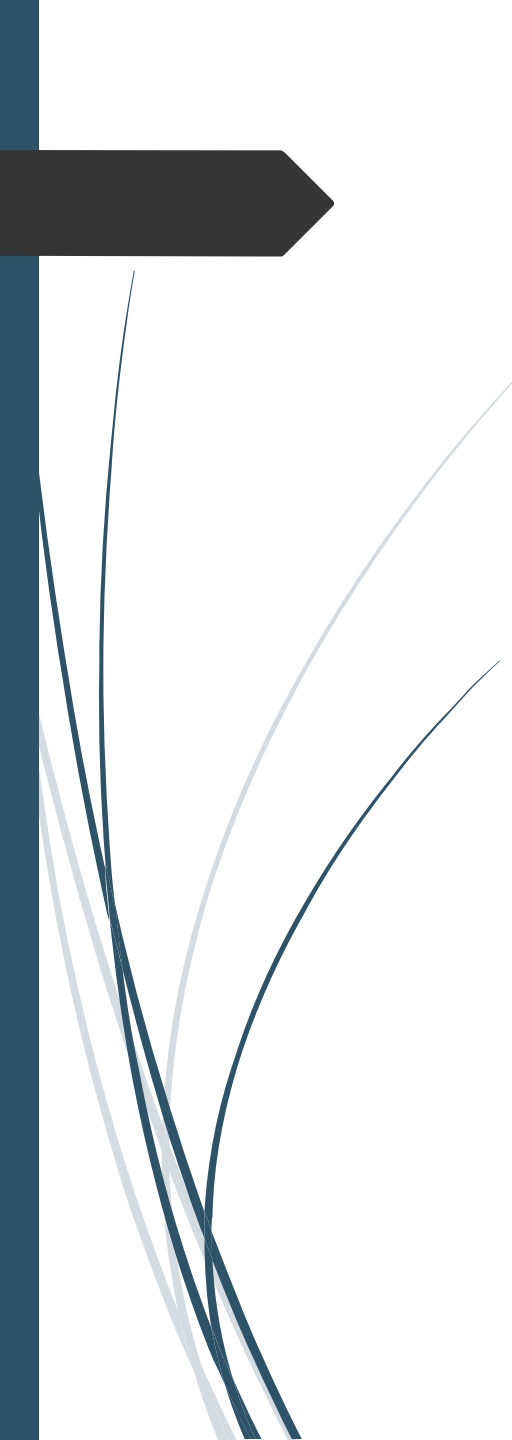
- Los Árboles de decisión son relativamente fáciles de entender e interpretar, sencillos de utilizar, versátiles y potentes.
- Sin embargo, tienen algunas limitaciones.
 - Como habrá observado, a los árboles de decisión les encantan los límites de decisión ortogonales (lo que los hace sensibles a la orientación de los datos).
 - La siguiente figura muestra un conjunto de datos sencillo linealmente separable: a la izquierda, un árbol de decisión puede dividirlo fácilmente, mientras que a la derecha, después de girar el conjunto de datos 45°, el límite de decisión parece innecesariamente enrevesado. Aunque ambos árboles de decisión se ajustan perfectamente al conjunto de entrenamiento, es muy probable que el modelo de la derecha no generalice bien.



Sensibilidad a la orientación del eje

- Una forma de limitar este problema es escalar los datos y, a continuación, aplicar una transformación de *análisis de componentes principales* (*Principal Component Analysis, PCA*).





Los árboles de decisión tienen una alta varianza

- El principal problema de los árboles de decisión es que tienen una varianza bastante alta: pequeños cambios en los hiperparámetros o en los datos pueden producir modelos muy diferentes.
- Dado que el algoritmo de entrenamiento utilizado por Scikit-Learn es estocástico -selecciona aleatoriamente el conjunto de características a evaluar en cada nodo-, incluso volver a entrenar el mismo árbol de decisión con exactamente los mismos datos puede producir un modelo muy diferente.
- Por suerte, al promediar las predicciones de muchos árboles, es posible reducir la varianza de forma significativa. Este conjunto de árboles se denomina *random forest* y es uno de los tipos de modelos más potentes que existen en la actualidad.



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Máquinas de Vector Soporte

Modelos basados en árboles de decisión

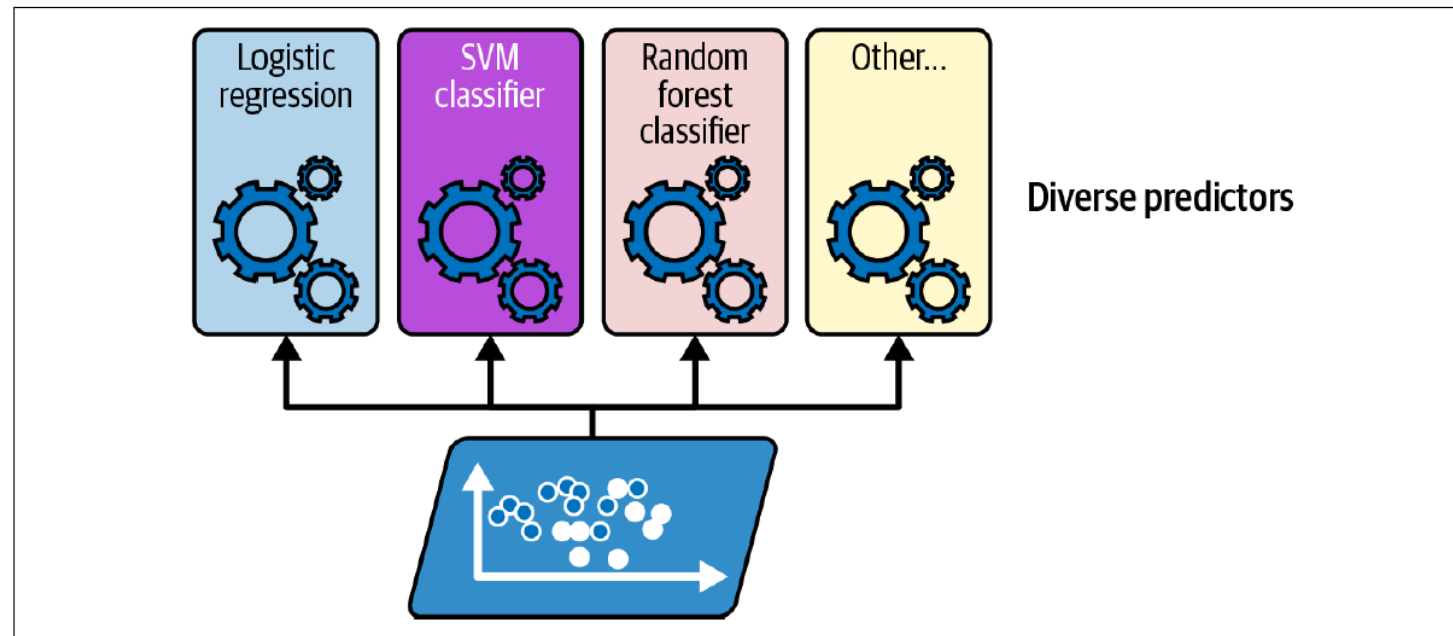
**Modelos multclasificadores y de
descomposición**

Ensembles y Random Forests

- Supongamos que planteas una pregunta compleja a miles de personas al azar y luego sumas sus respuestas.
- En muchos casos, esta respuesta agregada es mejor que la de un experto.
- **Es lo que se llama la sabiduría de la multitud.**
- Del mismo modo, si agrega las predicciones de un grupo de predictores (como clasificadores o regresores), a menudo obtendrá mejores predicciones que con el mejor predictor individual.
- Un grupo de predictores se denomina *ensemble*; por lo tanto, esta técnica se denomina *aprendizaje de ensemble*, y un algoritmo de aprendizaje de ensemble se denomina *método de ensemble* o *ensemble a secas*.

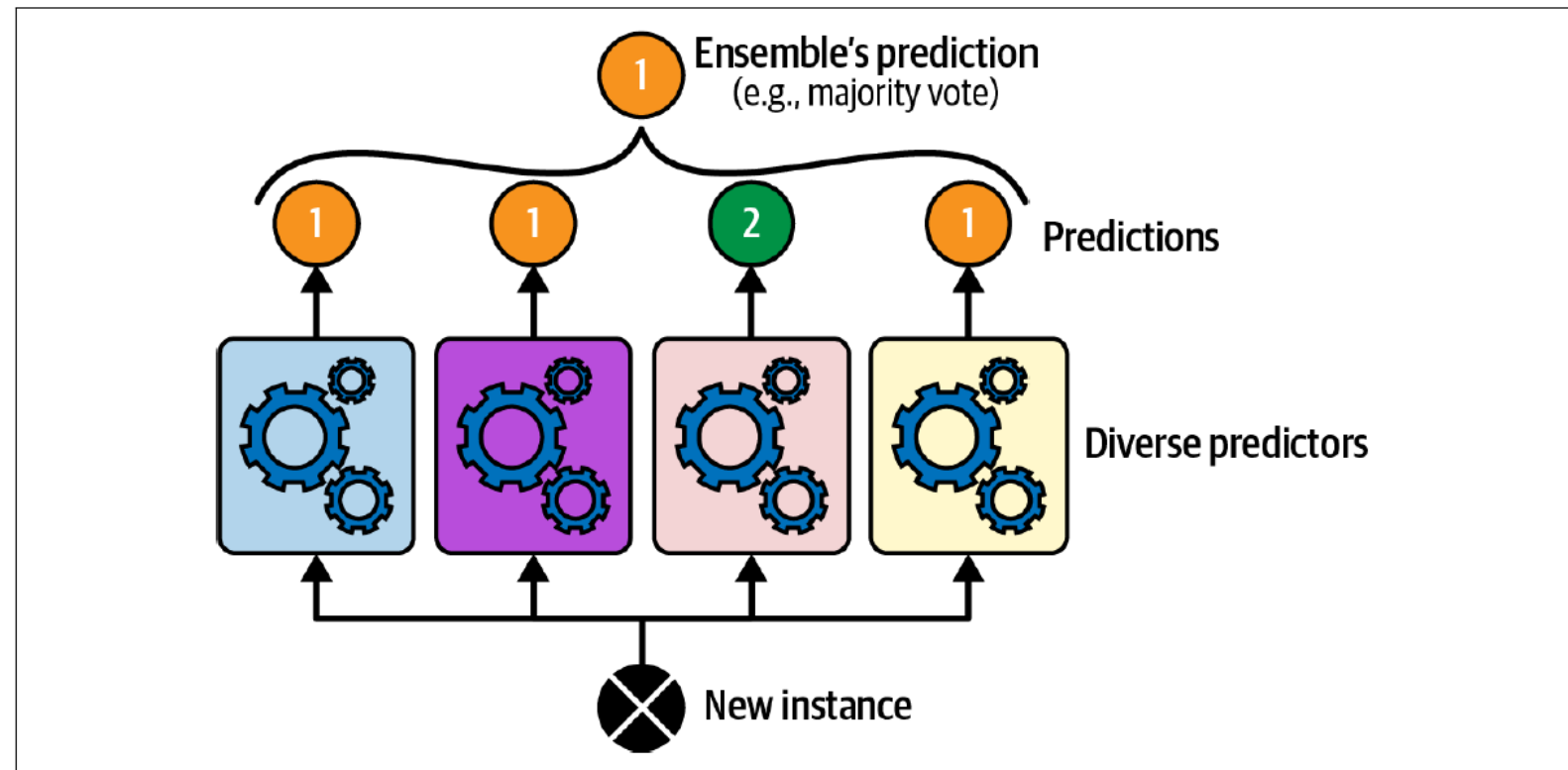
Clasificadores de voto

- Supongamos que ha entrenado unos cuantos clasificadores, cada uno de los cuales alcanza un 80% de precisión. Puede que tenga un clasificador de regresión logística, un clasificador SVM, un clasificador random forest, un clasificador de k vecinos más cercanos y quizá algunos más.



Clasificadores de voto

- Una forma muy sencilla de crear un clasificador aún mejor es agregar las predicciones de cada clasificador: la clase que obtiene más votos es la predicción del ensemble. Este clasificador por mayoría de votos se denomina clasificador de *voto duro*.



Clasificadores de voto

- Sorprendentemente, este clasificador por votación suele alcanzar una precisión mayor que el mejor clasificador del ensemble.
- De hecho, incluso **si cada clasificador es un aprendiz débil (*weak learner*) (lo que significa que lo hace sólo ligeramente mejor que la adivinación aleatoria)**, el ensemble puede seguir siendo un *aprendiz fuerte* (logrando una alta precisión), siempre que haya un número suficiente de aprendices débiles en el ensemble y que sean lo suficientemente diversos.

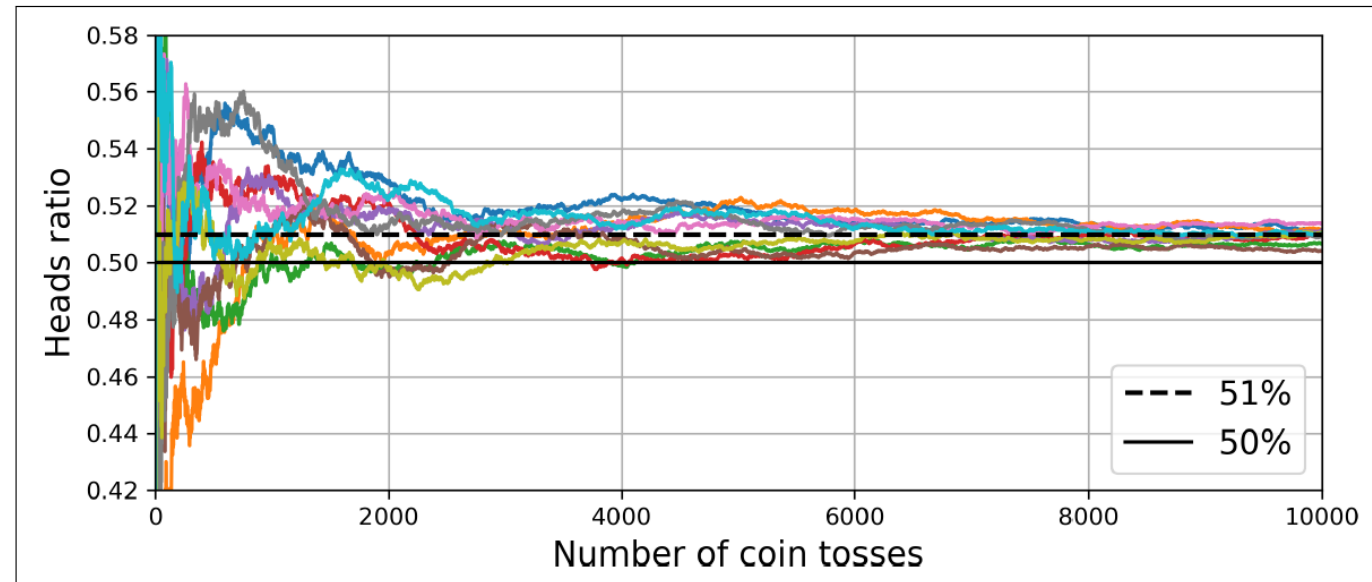
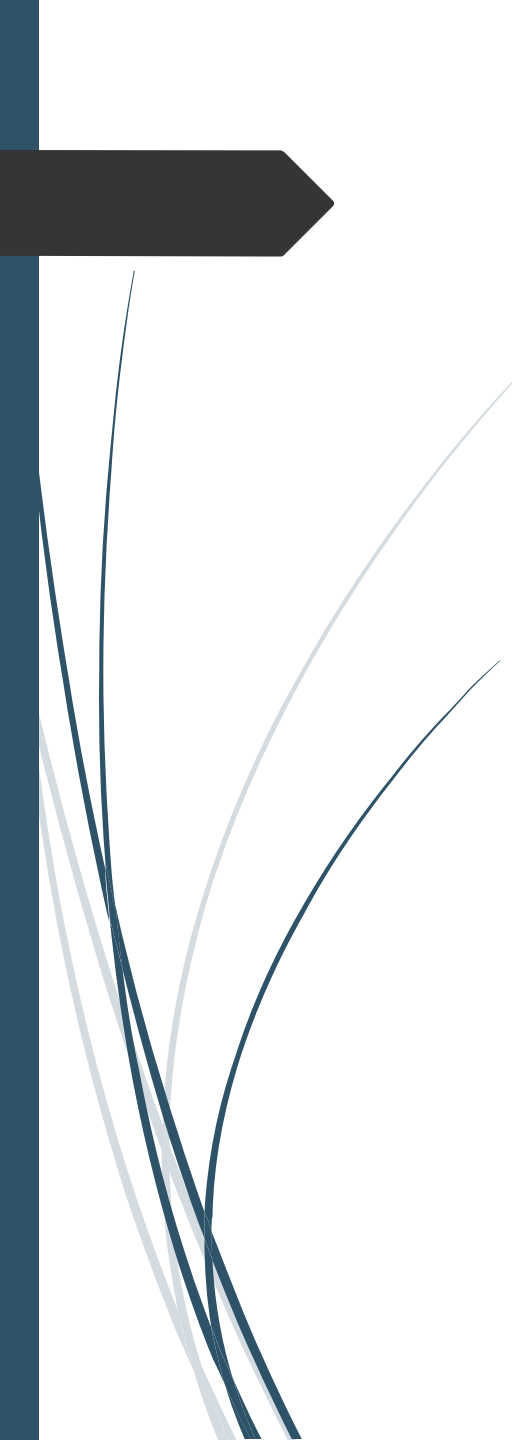


Figure 7-3. The law of large numbers

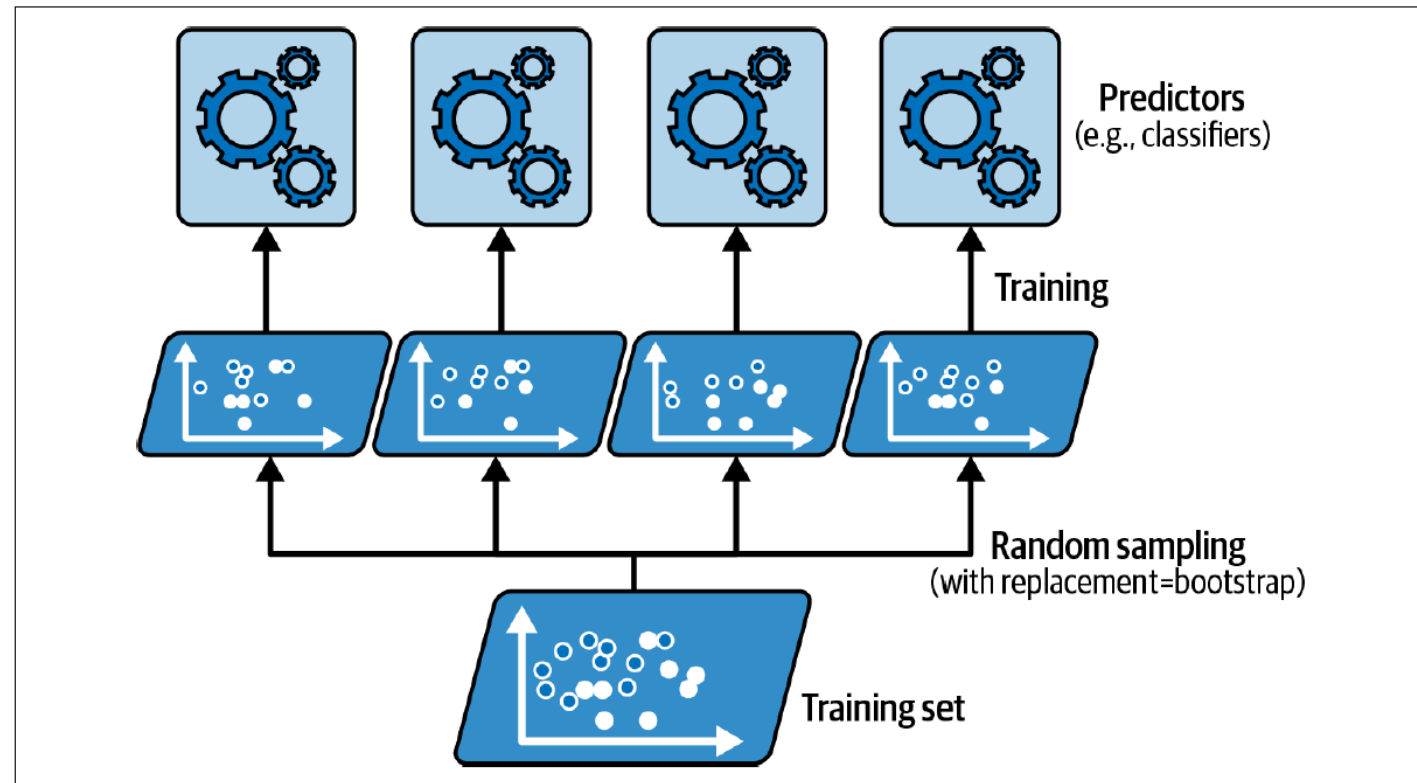


Bagging y Pasting

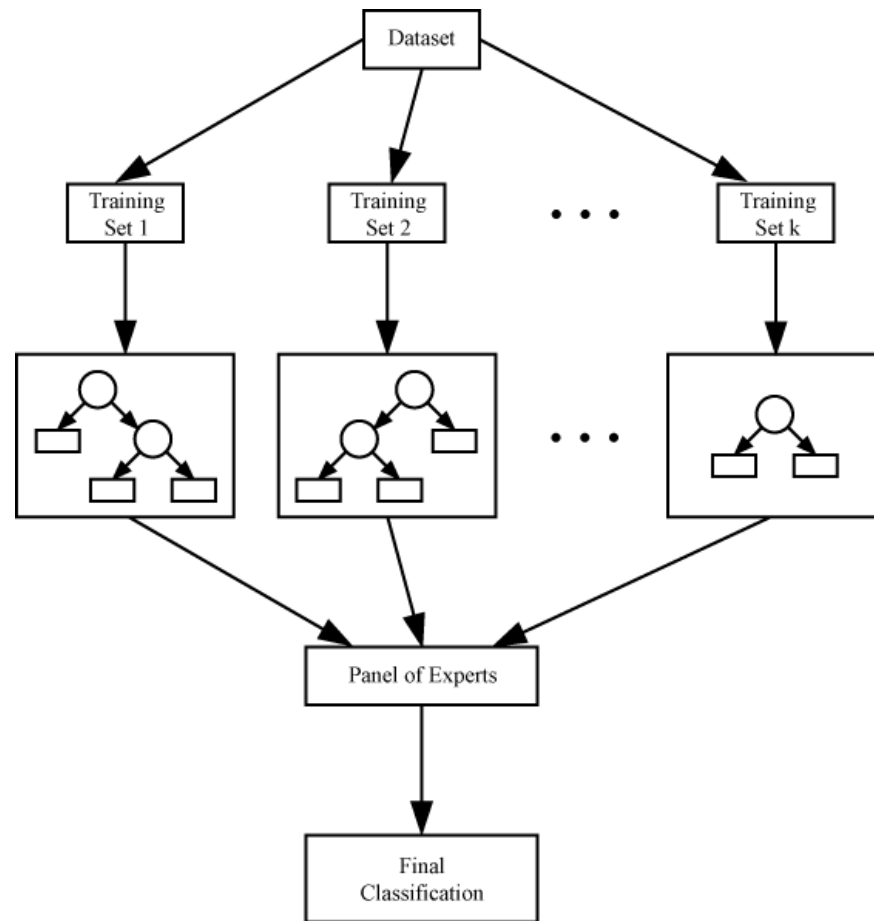
- Una forma de obtener un ensemble diverso de clasificadores es utilizar algoritmos de entrenamiento muy diferentes, como se acaba de comentar.
- Otro enfoque consiste en utilizar el mismo algoritmo de entrenamiento para cada predictor, pero entrenarlos en subconjuntos aleatorios diferentes del conjunto de entrenamiento.
- **Cuando el muestreo se realiza con sustitución, este método se denomina *bagging* (abreviatura de *bootstrap aggregating*).**
- Cuando el muestreo se realiza *sin* reemplazo, se denomina *pasting*.

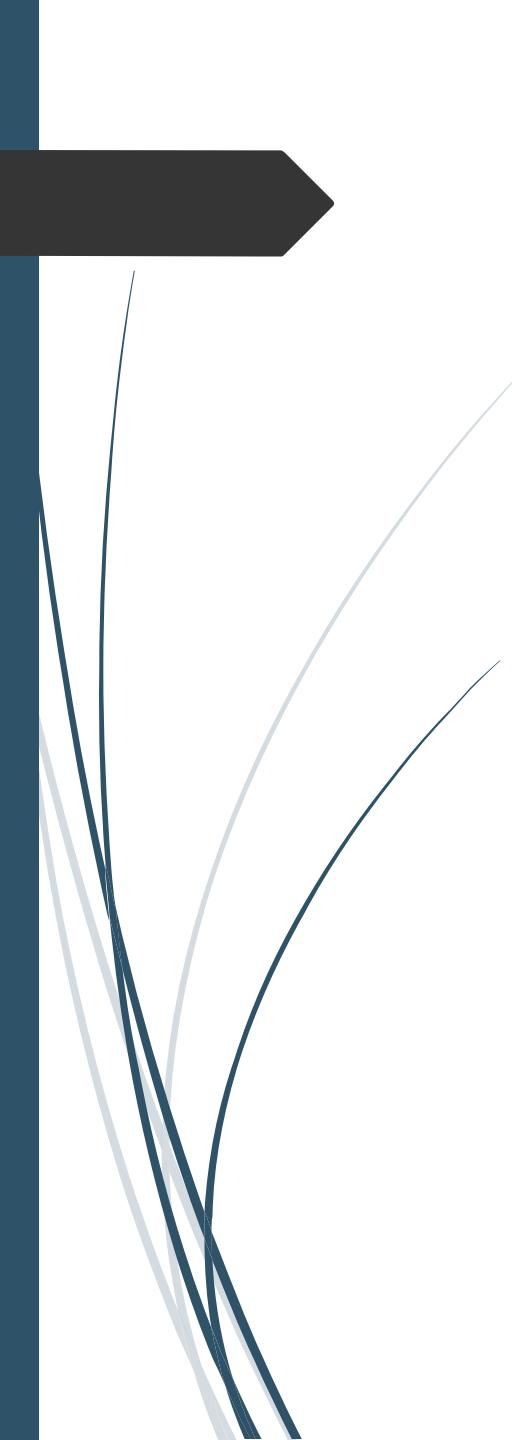
Bagging y Pasting

- Tanto el bagging como el pasting permiten muestrear instancias de entrenamiento varias veces a través de múltiples predictores, pero sólo el bagging permite muestrear instancias de entrenamiento varias veces para el mismo predictor. Este proceso de muestreo y entrenamiento se representa aquí:



Bagging y Pasting



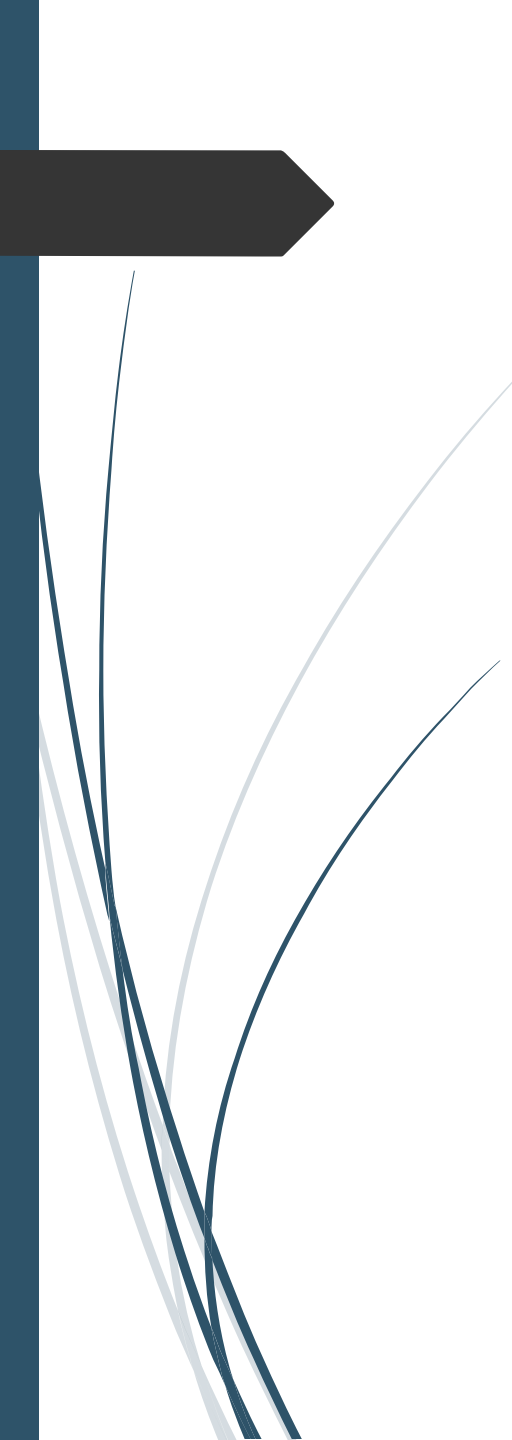


Bagging y Pasting

- Una vez entrenados todos los predictores, el ensemble puede realizar una predicción para una nueva instancia simplemente agregando las predicciones de todos los predictores.
- La función de agregación suele ser la moda estadística para la clasificación o la media para la regresión.
- Cada predictor individual tiene un sesgo mayor que si se hubiera entrenado con el ensemble de entrenamiento original, pero la agregación reduce tanto el sesgo como la varianza.
- Todos los predictores pueden entrenarse en paralelo, mediante distintos núcleos de CPU o incluso distintos servidores.
- Del mismo modo, las predicciones pueden realizarse en paralelo. Esta es una de las razones por las que el bagging y el pasting son métodos tan populares: escalan muy bien.

Evaluación Out-of-Bag

- Con bagging, algunas instancias de entrenamiento pueden ser muestreadas varias veces para cualquier predictor dado, mientras que otras pueden no ser muestreadas en absoluto. Por defecto, un *BaggingClassifier* muestrea m instancias de entrenamiento con reemplazo (*bootstrap=True*).
- Se puede demostrar matemáticamente que sólo se muestrea una media del 63% de las instancias de entrenamiento para cada predictor. El 37% restante de las instancias de entrenamiento que no se muestrean se denominan instancias *fuera de la bolsa* (OOB). Tenga en cuenta que no es el mismo 37% para todos los predictores.
- Un ensemble de bagging puede evaluarse utilizando instancias OOB, sin necesidad de un ensemble de validación separado. Una vez que tenga una predicción para cada instancia, puede calcular la precisión de predicción del ensemble (o cualquier otra métrica).



Random Patches y Random Subspaces

- La clase ***BaggingClassifier*** también admite el muestreo de las **características**. El muestreo se controla mediante dos hiperparámetros: *max_features* y *bootstrap_features*. Funcionan del mismo modo que *max_samples* y *bootstrap*, pero para el muestreo de características en lugar del muestreo de instancias. Así, cada predictor se entrenará con un subconjunto aleatorio de las características de entrada.
- Esta técnica es especialmente útil cuando se trabaja con entradas de alta dimensión (como imágenes), ya que puede acelerar considerablemente el entrenamiento.
- El muestreo tanto de las instancias de entrenamiento como de las características se denomina método de **Random Patches**.
- Mantener todas las instancias de entrenamiento (estableciendo *bootstrap=False* y *max_samples=1.0*) pero muestreando características (estableciendo *bootstrap_features* a True y/o *max_features* a un valor menor que 1.0) se denomina método de **Random Subspaces**.
- El muestreo de características da lugar a una mayor diversidad de predictores, cambiando un poco más de sesgo por una menor varianza.

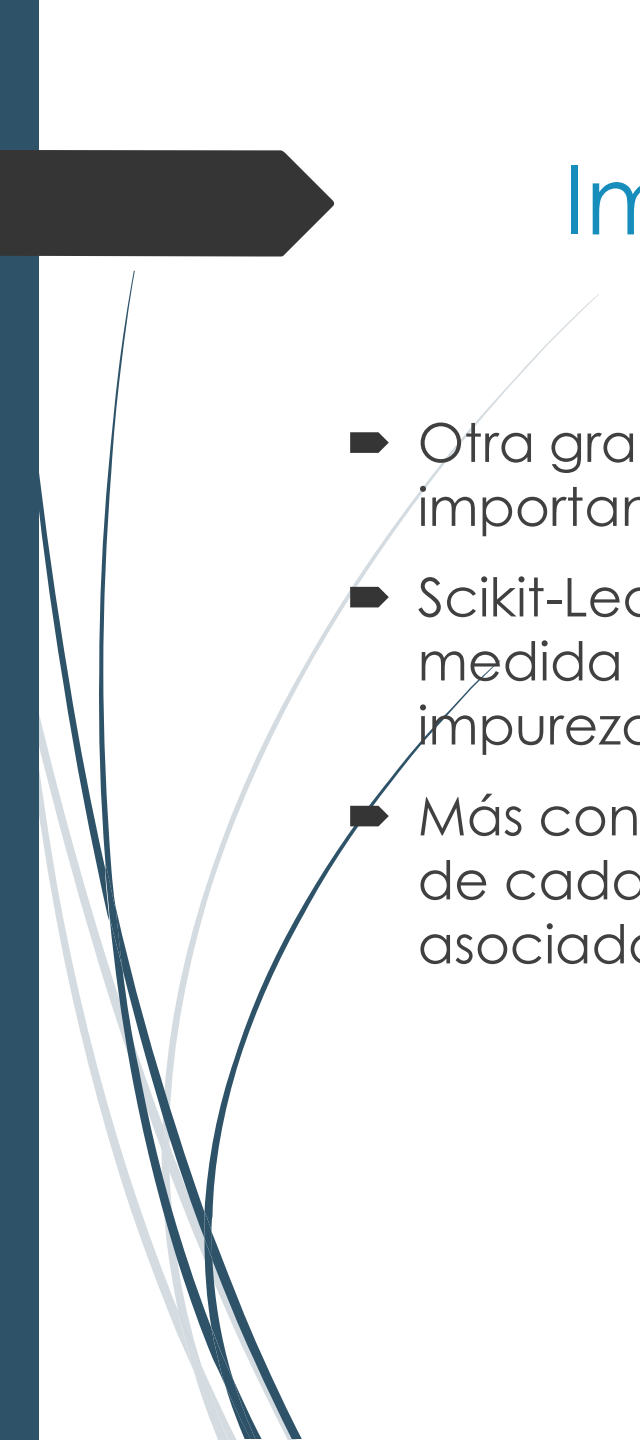


Random Forests

- Un *random forest* es un ensemble de árboles de decisión, generalmente entrenados mediante el método bagging, normalmente con *max_samples* establecido en el tamaño del ensemble de entrenamiento.
- En lugar de construir un *BaggingClassifier* y pasarle un *DecisionTreeClassifier*, puedes utilizar la clase *RandomForestClassifier*, que es más conveniente y está optimizada para árboles de decisión (de forma similar, existe una clase *RandomForestRegressor* para tareas de regresión).
- La clase *BaggingClassifier* sigue siendo útil si se desea un ensemble de algo distinto a los árboles de decisión.

Extra-Trees

- Cuando se hace crecer un árbol en un random forest, en cada nodo sólo se considera un subensemble aleatorio de las características para la división.
- Es posible hacer que los árboles sean aún más aleatorios utilizando también umbrales aleatorios para cada característica en lugar de buscar los mejores umbrales posibles (como hacen los árboles de decisión normales).
- Para ello, basta con establecer *splitter="random"* al crear un *DecisionTreeClassifier*.
- Un bosque de estos árboles extremadamente aleatorios se denomina ensemble de árboles extremadamente aleatorios (o extra-árboles, para abreviar). Una vez más, esta técnica cambia un mayor sesgo por una menor varianza. También hace que los clasificadores extra-árboles sean mucho más rápidos de entrenar que los random forests normales, ya que encontrar el mejor umbral posible para cada característica en cada nodo es una de las tareas que más tiempo consume en el crecimiento de un árbol.
- Puedes crear un clasificador extra-árboles utilizando la clase *ExtraTreesClassifier* de Scikit-Learn. Su API es idéntica a la de la clase *RandomForestClassifier*, excepto que el valor predeterminado de *bootstrap* es *False*. Del mismo modo, la clase *ExtraTreesRegressor* tiene la misma API que la clase *RandomForestRegressor*, excepto que el valor predeterminado de *bootstrap* es *False*.



Importancia de las Características

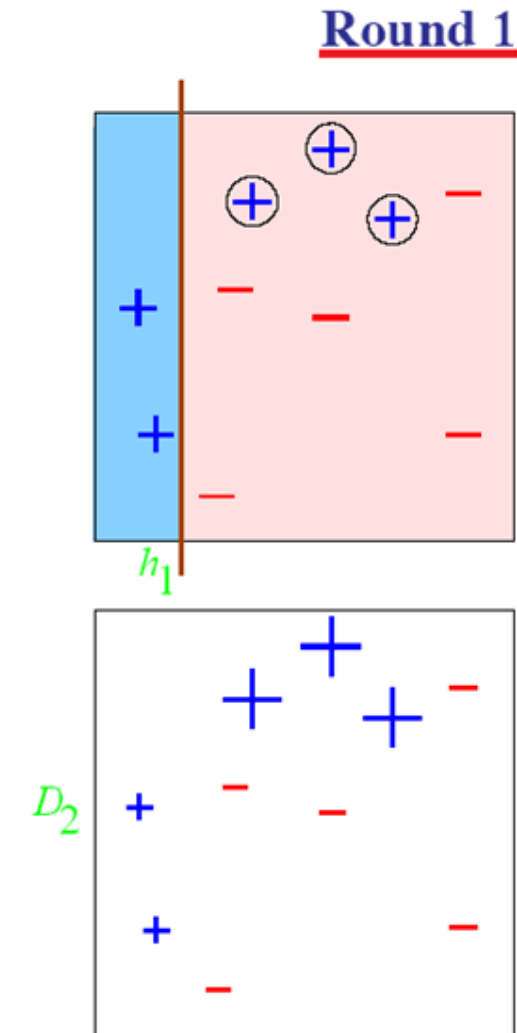
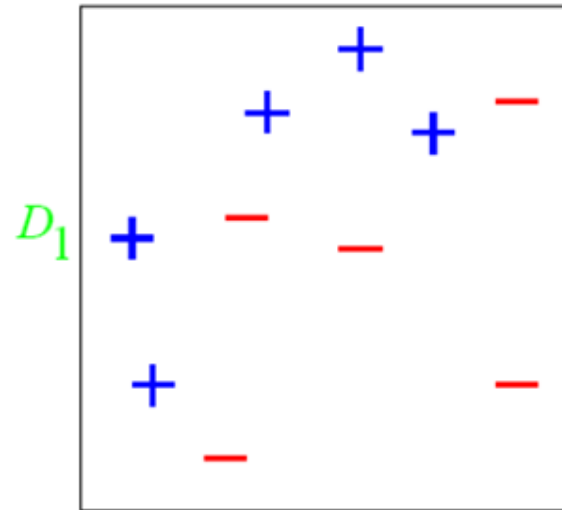
- Otra gran cualidad de los random forests es que facilitan la medición de la importancia relativa de cada característica.
- Scikit-Learn mide la importancia de una característica observando en qué medida los nodos del árbol que utilizan esa característica reducen la impureza de media, en todos los árboles del bosque.
- Más concretamente, se trata de una media ponderada, en la que el peso de cada nodo es igual al número de muestras de entrenamiento que tiene asociadas.

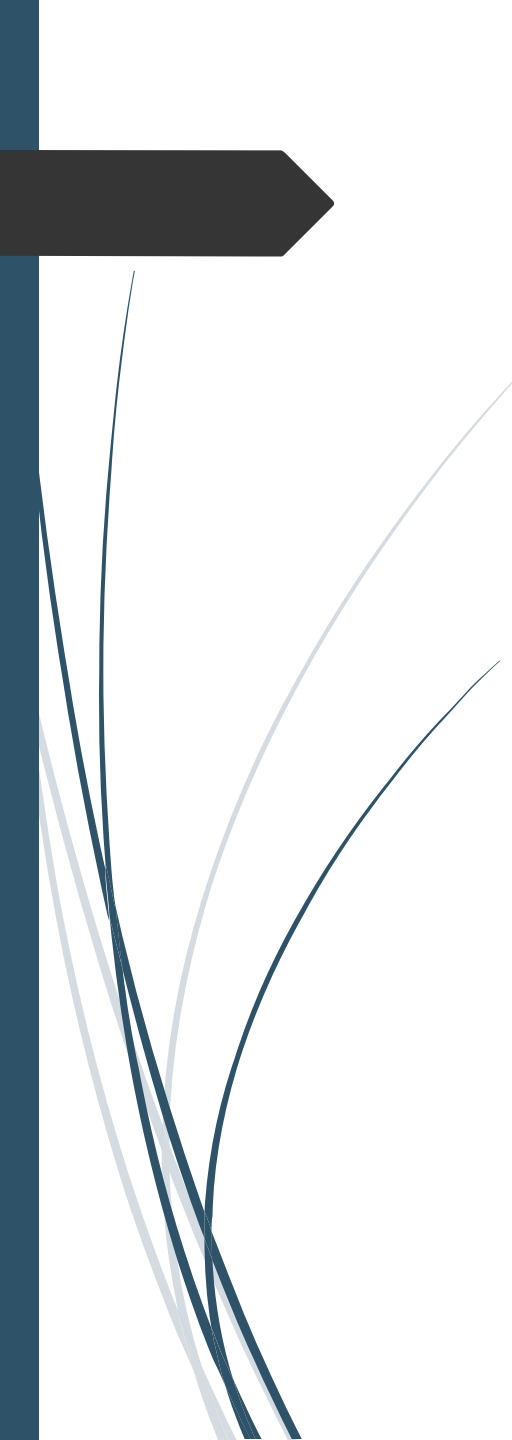
Boosting

- **El boosting** (originalmente llamado *hypothesis boosting*) se refiere a cualquier método ensemble que pueda combinar varios aprendices débiles en un aprendiz fuerte.
- La idea general de la mayoría de los métodos de refuerzo es entrenar predictores secuencialmente, cada uno de los cuales intenta corregir a su predecesor.
- Existen muchos métodos de boosting, pero los más populares con diferencia son **AdaBoost** (abreviatura de *boostig adaptativo*) y **gradient boosting**.

Yoav Freund y Robert E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting", *Journal of Computer and System Sciences* 55, nº 1 (1997): 119-139.

Boosting

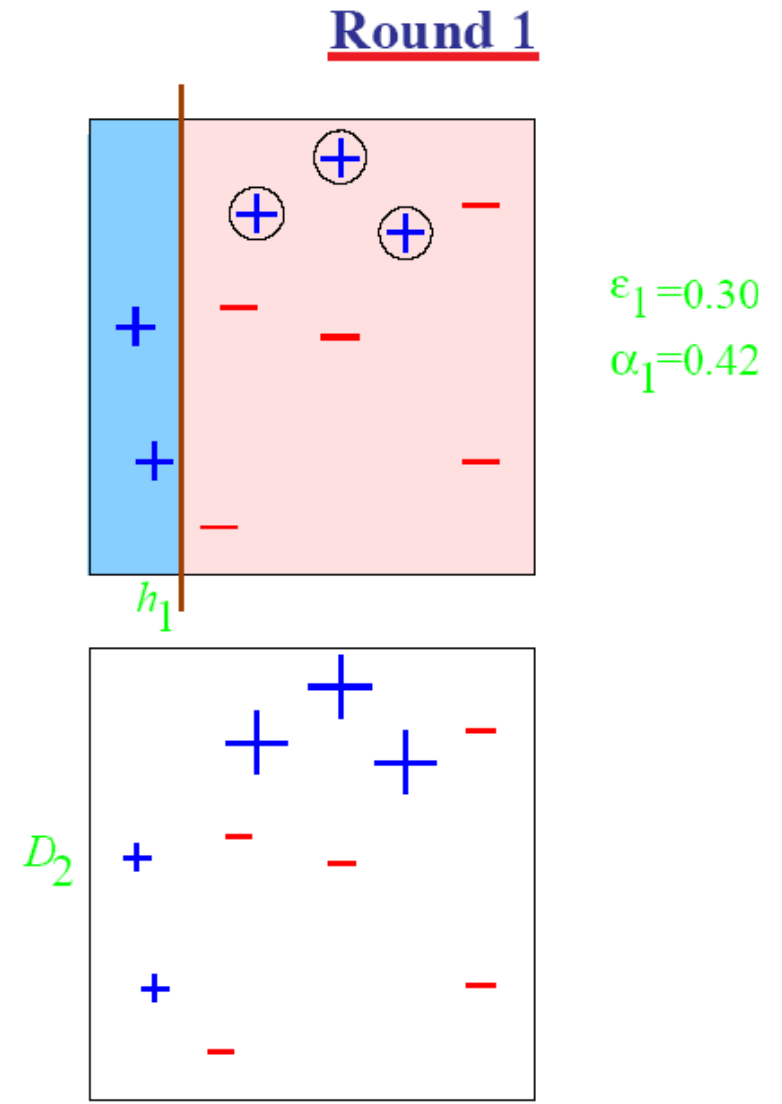
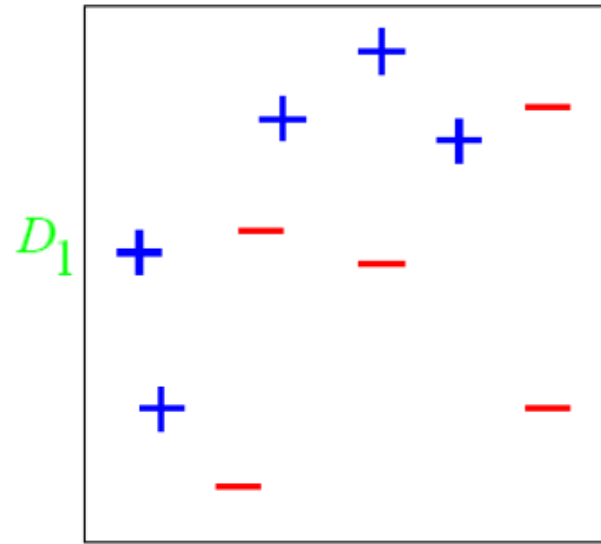




AdaBoost

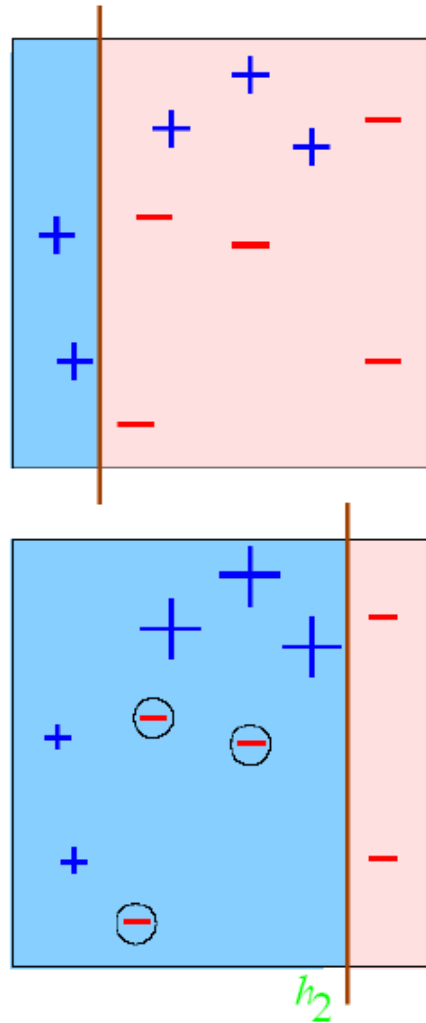
- Una forma de que un nuevo predictor corrija a su predecesor es prestar un poco más de atención a las instancias de entrenamiento que el predecesor infra-ajustó.
- Esto hace que los nuevos predictores se centren cada vez más en los casos difíciles.
- Esta es la técnica utilizada por *AdaBoost*.
- Al entrenar un clasificador AdaBoost, el algoritmo entrena primero un clasificador base (como un árbol de decisión) y lo utiliza para hacer predicciones sobre el conjunto de entrenamiento. A continuación, el algoritmo aumenta el peso relativo de las instancias de entrenamiento mal clasificadas. Después, entrena un segundo clasificador, utilizando los pesos actualizados, y vuelve a realizar predicciones sobre el conjunto de entrenamiento, actualiza los pesos de las instancias, y así sucesivamente.

Boosting

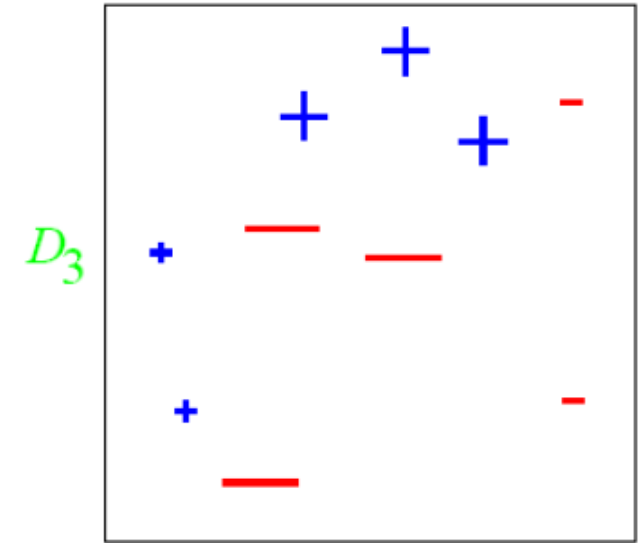


Boosting

Round 2

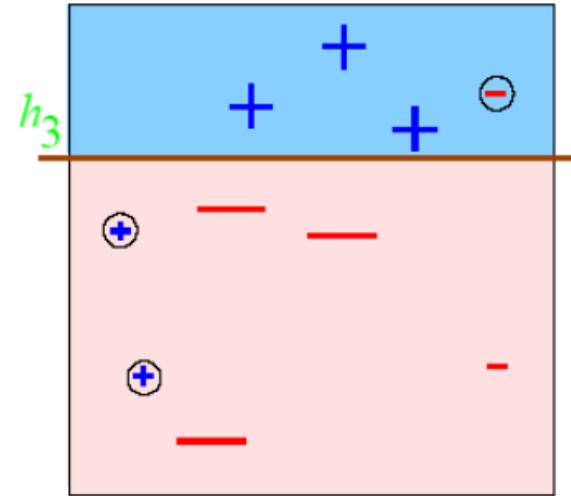
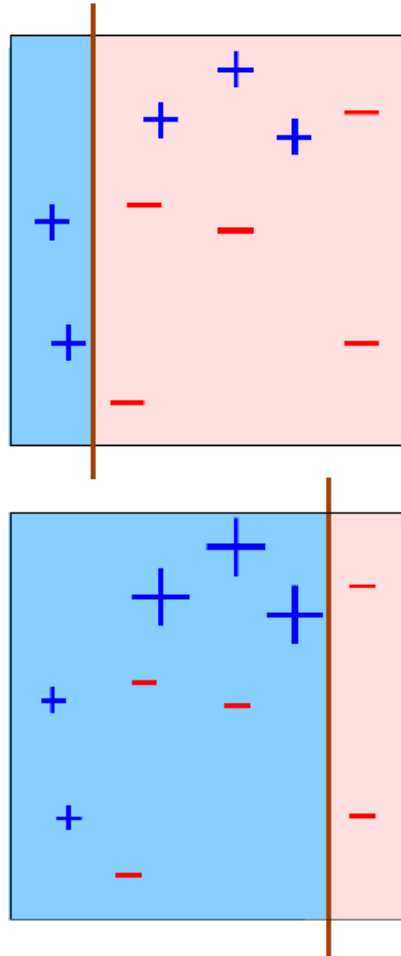


$$\epsilon_2 = 0.21$$
$$\alpha_2 = 0.65$$



Boosting

Round 3



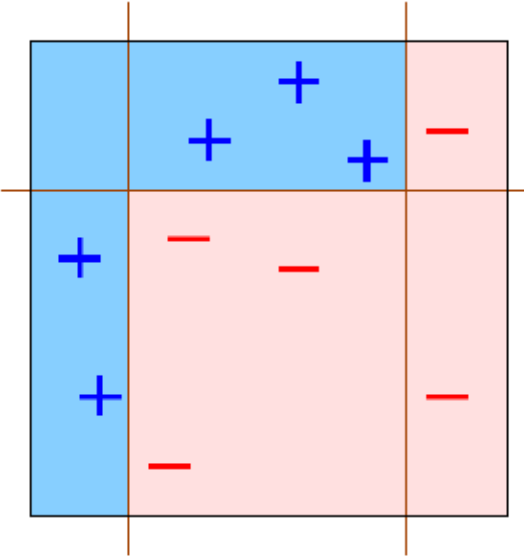
$\epsilon_3 = 0.14$
 $\alpha_3 = 0.92$

Boosting

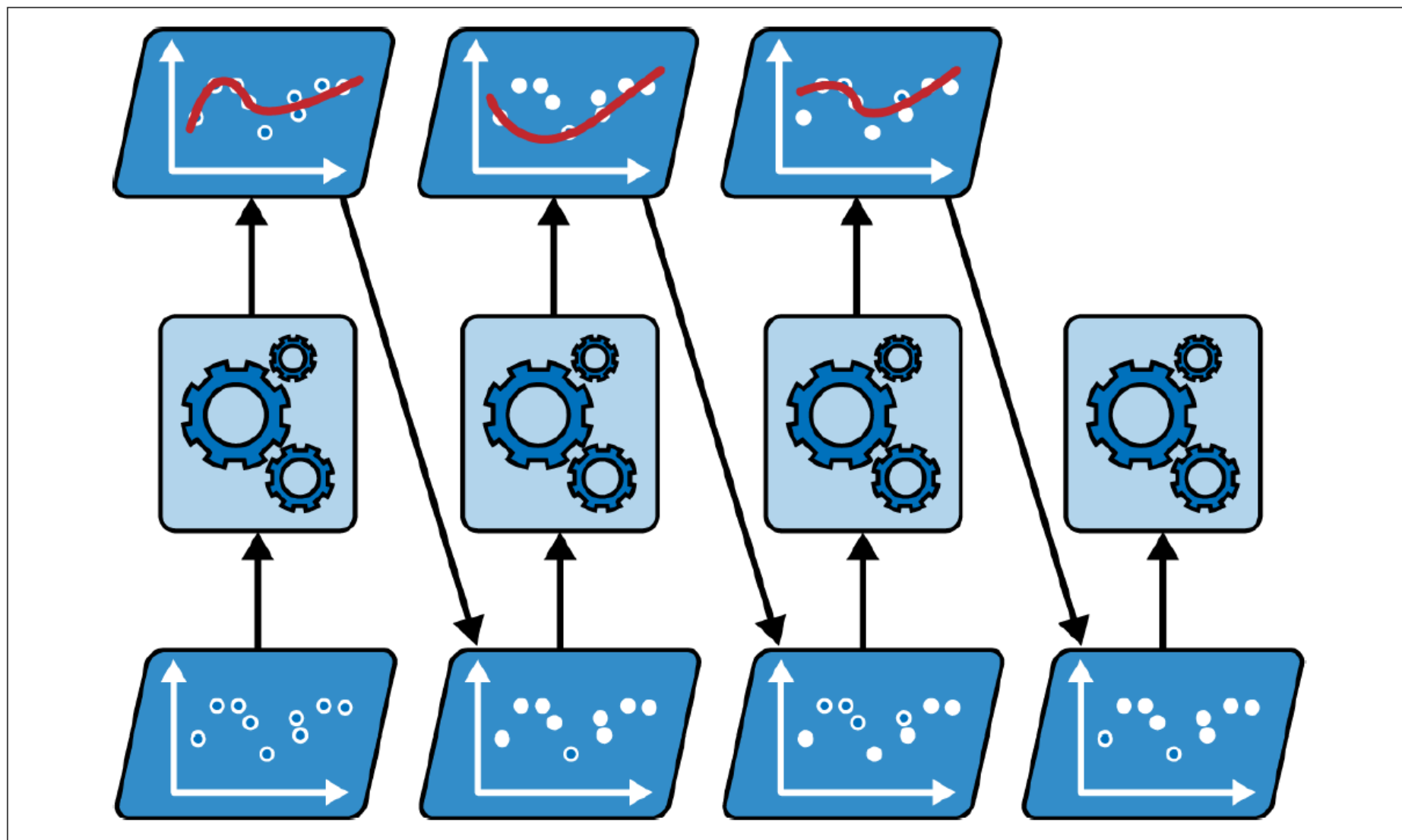
Final Hypothesis

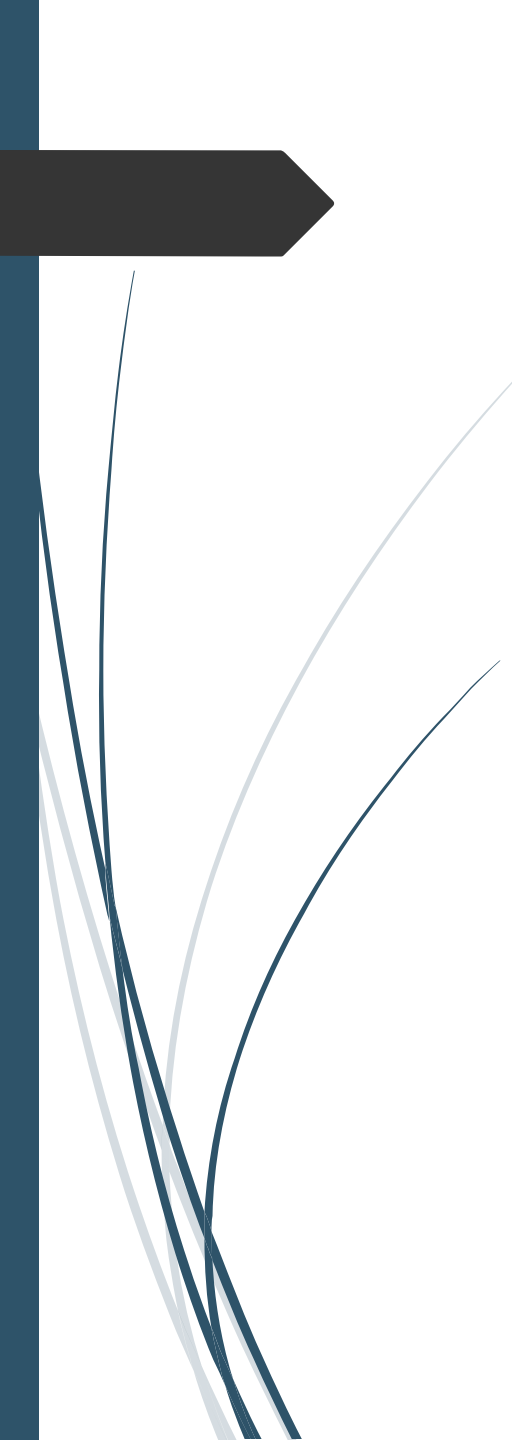
$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} + 0.65 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} + 0.92 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} \right)$$

=



AdaBoost

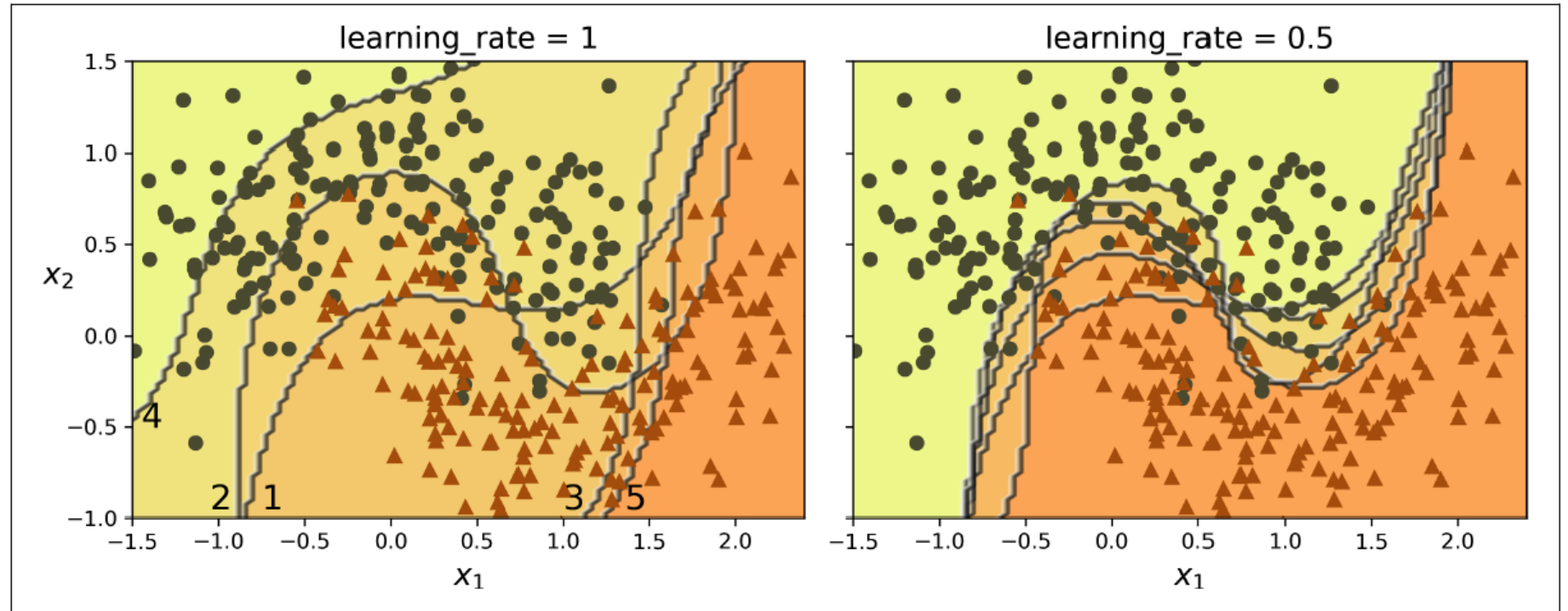




AdaBoost

- La siguiente figura muestra los límites de decisión de cinco predictores consecutivos en el conjunto de datos *moons* (en este ejemplo, cada predictor es un clasificador SVM altamente regularizado con un núcleo RBF).
- El primer clasificador se equivoca en muchos casos, por lo que su peso aumenta. Por tanto, el segundo clasificador hace un mejor trabajo en estos casos, y así sucesivamente. El gráfico de la derecha representa la misma secuencia de predictores, salvo que la tasa de aprendizaje se reduce a la mitad.
- Como puede ver, esta técnica de aprendizaje secuencial tiene algunas similitudes con el descenso por gradiente, salvo que en lugar de ajustar los parámetros de un único predictor para minimizar una función de coste, AdaBoost añade predictores al ensemble, mejorándolo gradualmente.
- Una vez entrenados todos los predictores, el ensemble realiza predicciones muy parecidas a las de bagging o pasting, con la diferencia de que los predictores tienen distintos pesos en función de su precisión global en el conjunto de entrenamiento ponderado.

AdaBoost



- Esta técnica de aprendizaje secuencial tiene un importante inconveniente: el entrenamiento no puede paralelizarse, ya que cada predictor sólo puede entrenarse después de que el anterior haya sido entrenado y evaluado. Como resultado, no es tan escalable como el bagging o el pasting.

AdaBoost

- El peso de cada instancia $w^{(i)}$ se fija inicialmente en $1/m$. Se entrena un primer predictor y se calcula su tasa de error ponderado r_1 en el ensemble de entrenamiento:

$$r_j = \sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^m w^{(i)} \quad \text{where } \hat{y}_j^{(i)} \text{ is the } j^{\text{th}} \text{ predictor's prediction for the } i^{\text{th}} \text{ instance}$$

- El peso del predictor α_j se calcula utilizando la siguiente ecuación, donde η es el hiperparámetro de la tasa de aprendizaje (por defecto es 1). Cuanto más preciso sea el predictor, mayor será su peso. Si sólo adivina al azar, su peso será cercano a cero. Sin embargo, si la mayoría de las veces se equivoca (es decir, es menos preciso que adivinar al azar), su peso será negativo.

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

AdaBoost

- A continuación, el algoritmo AdaBoost actualiza los pesos de las instancias, utilizando esta ecuación, que potencia los pesos de las instancias mal clasificadas.

$$\text{for } i = 1, 2, \dots, m$$
$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

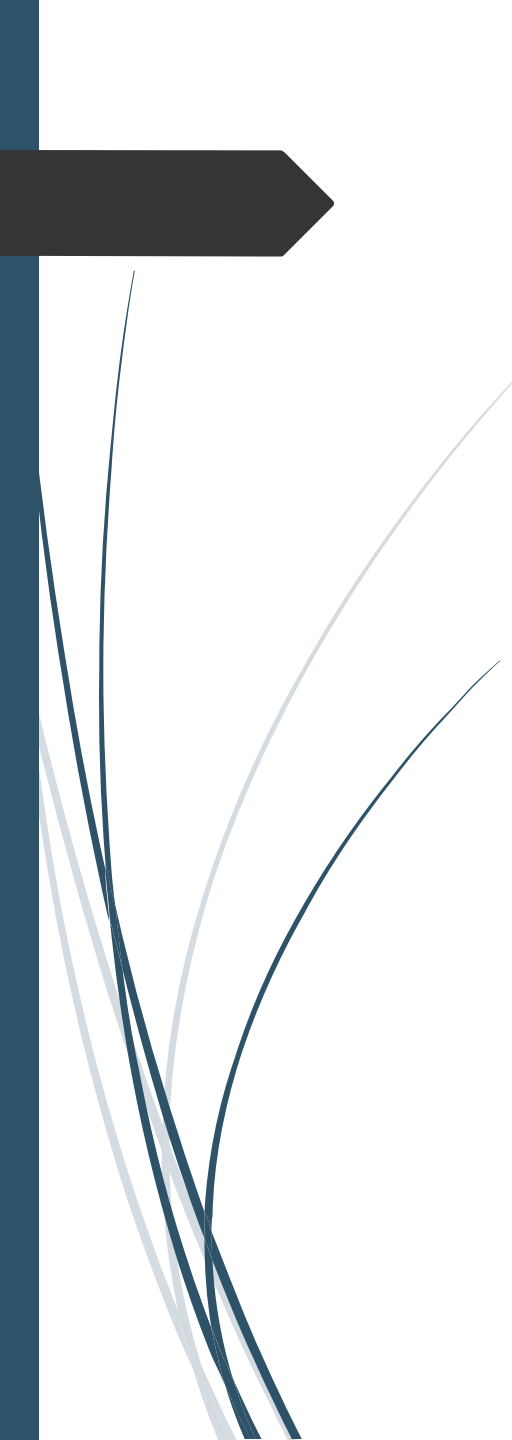
- A continuación, se normalizan todas las ponderaciones de instancia (divididas por $\sum_{i=1}^m w^{(i)}$)
- Por último, se entrena un nuevo predictor con las ponderaciones actualizadas y se repite todo el proceso: se calcula la ponderación del nuevo predictor, se actualizan las ponderaciones de la instancia, se entrena otro predictor y así sucesivamente.
- El algoritmo se detiene cuando se alcanza el número deseado de predictores o cuando se encuentra un predictor perfecto.

AdaBoost

- Para hacer predicciones, AdaBoost simplemente calcula las predicciones de todos los predictores y las pondera utilizando los pesos de los predictores α_j . La clase predicha es la que recibe la mayoría de los votos ponderados:

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x}) = k}}^N \alpha_j \quad \text{where } N \text{ is the number of predictors}$$

- Scikit-Learn utiliza una versión multiclase de AdaBoost llamada SAMME (que significa *Stagewise Additive Modeling using a Multiclass Exponential loss function*). Cuando sólo hay dos clases, SAMME es equivalente a AdaBoost.
- Si los predictores pueden estimar probabilidades de clase (es decir, si tienen un método `predict_proba()`), Scikit-Learn puede utilizar una variante de SAMME llamada SAMME.R (la R significa "Real"), que se basa en probabilidades de clase en lugar de predicciones y, por lo general, tiene un mejor rendimiento.

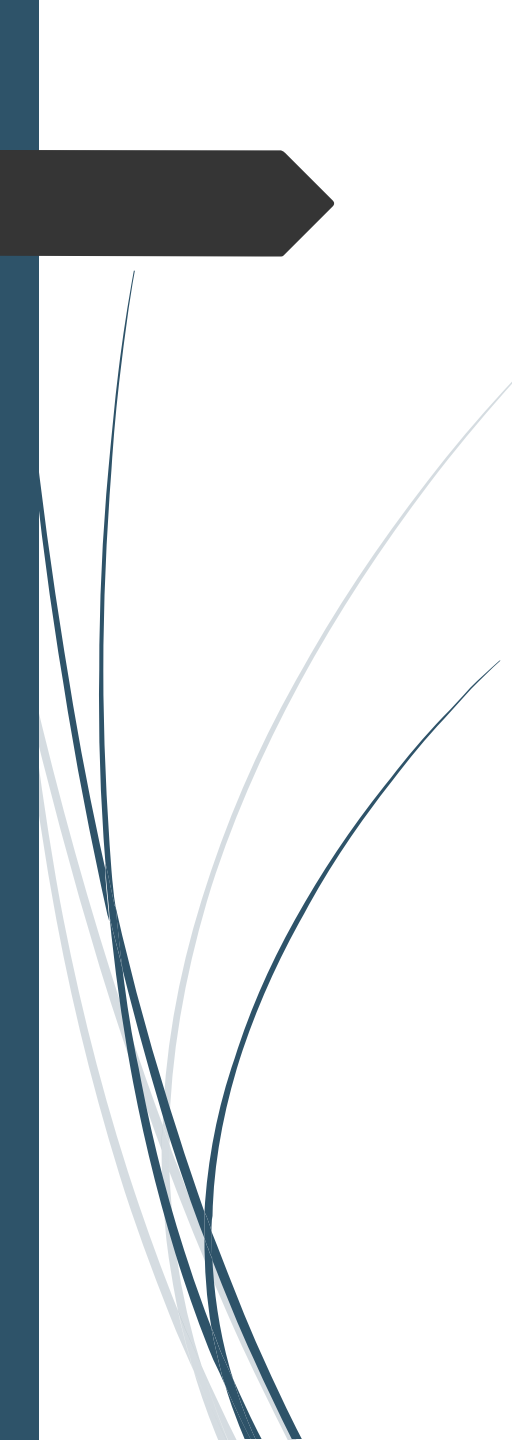


Gradient Boosting

- Otro algoritmo de refuerzo muy popular es *gradient boosting*.
- Al igual que AdaBoost, gradient boosting funciona añadiendo secuencialmente predictores a un ensemble, cada uno de los cuales corrige a su predecesor.
- Sin embargo, en lugar de ajustar los pesos de instancia en cada iteración como hace AdaBoost, este método intenta ajustar el nuevo predictor a los errores residuales cometidos por el predictor anterior.

Histogram-Based Gradient Boosting

- Scikit-Learn también proporciona otra implementación de GBRT, optimizada para grandes conjuntos de datos: *histogram-based gradient boosting* (HGB).
- Funciona dividiendo las características de entrada, sustituyéndolas por números enteros. El número de bins se controla mediante el hiperparámetro *max_bins*, que por defecto es 255 y no puede ser mayor. La discretización puede reducir en gran medida el número de umbrales posibles que el algoritmo de entrenamiento tiene que evaluar. Además, trabajar con números enteros permite utilizar estructuras de datos más rápidas y eficientes en cuanto a memoria. Y la forma en que se construyen los bins elimina la necesidad de ordenar las características al entrenar cada árbol.
- Como resultado, esta implementación tiene una complejidad computacional de $O(b \times m)$ en lugar de $O(n \times m \times \log(m))$, donde b es el número de bins.
- En la práctica, esto significa que HGB puede entrenar cientos de veces más rápido que GBRT normal en grandes conjuntos de datos. Sin embargo, el binning provoca una pérdida de precisión, que actúa como un regularizador: dependiendo del ensemble de datos, esto puede ayudar a reducir el sobreajuste, o puede causar un infraajuste.
- Scikit-Learn proporciona dos clases para HGB: *HistGradientBoostingRegressor* y *HistGradientBoostingClassifier*.
- Admiten tanto características categóricas como valores perdidos.

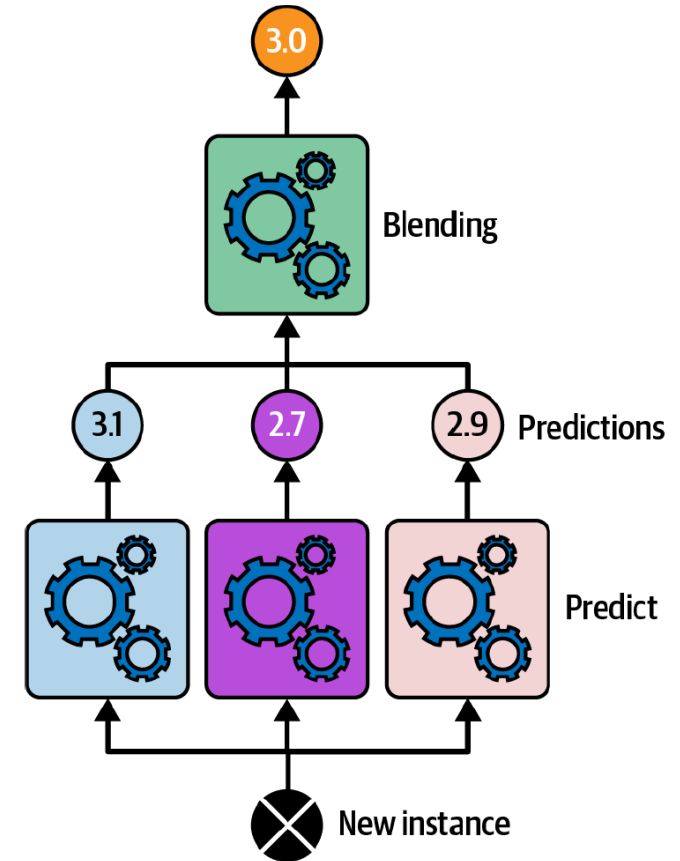


Gradient Boosting Avanzado

- En el ecosistema Python ML existen otras implementaciones optimizadas de gradient boosting.
- En particular, **XGBoost**, **CatBoost** y **LightGBM**.
- Estas bibliotecas existen desde hace varios años. Todas están especializadas en gradient boosting, sus API son muy similares a las de Scikit-Learn y proporcionan muchas funciones adicionales, incluida la aceleración por GPU.
- Además, la librería **TensorFlow Random Forests** proporciona implementaciones optimizadas de una variedad de algoritmos de random forests, incluyendo random forests simples, extra-trees, GBRT, y varios más.

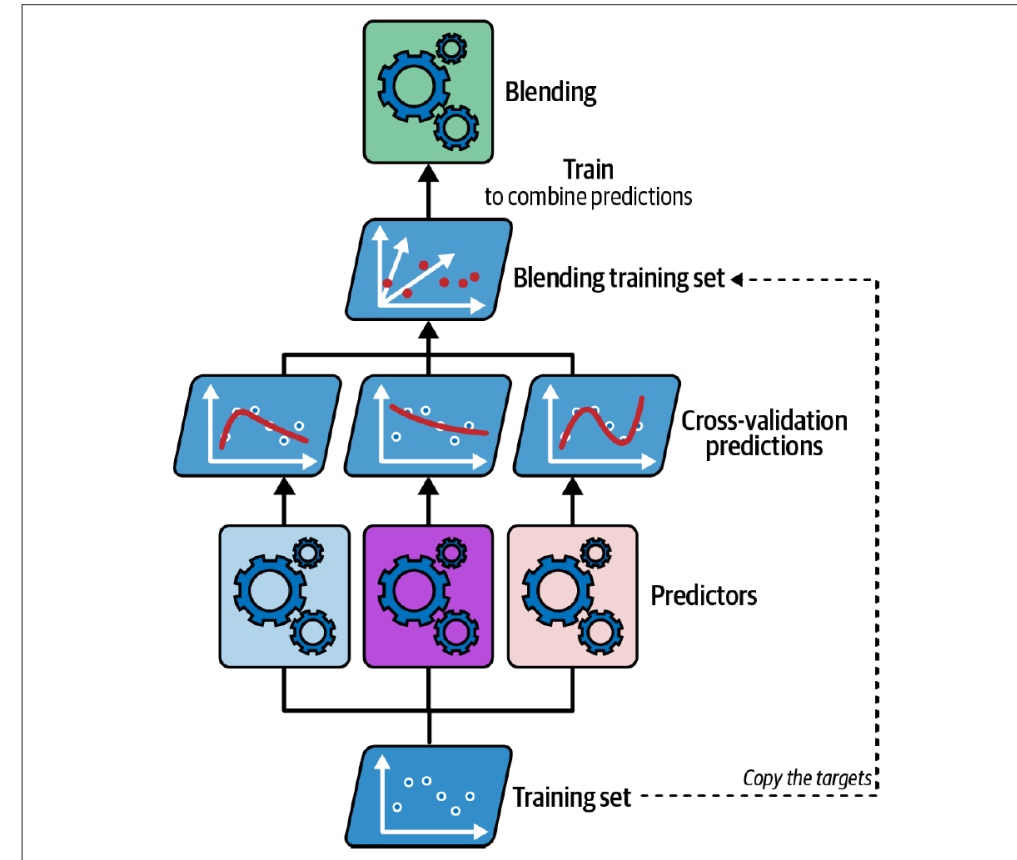
Stacking

- La generalización apilada (*stacking*) se basa en una idea sencilla: en lugar de utilizar funciones triviales (como el voto duro) para agregar las predicciones de todos los predictores de un ensemble, ¿por qué no entrenamos un modelo para que realice esta agregación?
- Esta figura muestra un ensemble de este tipo realizando una tarea de regresión sobre una nueva instancia. Cada uno de los tres predictores inferiores predice un valor diferente (3,1, 2,7 y 2,9) y, a continuación, el predictor final (denominado *mezclador* o *metaaprendiz*) toma estas predicciones como entradas y realiza la predicción final (3,0).



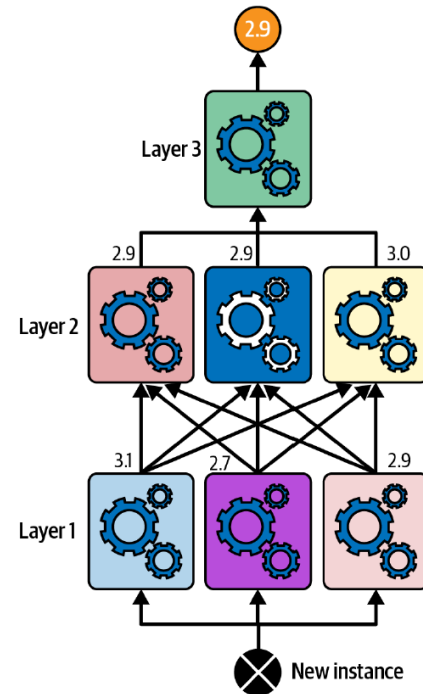
Stacking

- Para entrenar el mezclador, primero necesita construir el conjunto de entrenamiento de mezclado. Puedes utilizar `cross_val_predict()` en cada predictor del ensemble para obtener predicciones fuera de muestra para cada instancia del conjunto de entrenamiento original, y utilizarlas como características de entrada para entrenar el mezclador; y los objetivos pueden copiarse simplemente del conjunto de entrenamiento original.
- Ten en cuenta que, independientemente del número de características del conjunto de entrenamiento original, el conjunto de entrenamiento de la mezcla contendrá una característica de entrada por predictor (tres en este ejemplo).
- Una vez entrenado el mezclador, los predictores base se vuelven a entrenar una última vez en el ensemble de entrenamiento original completo.



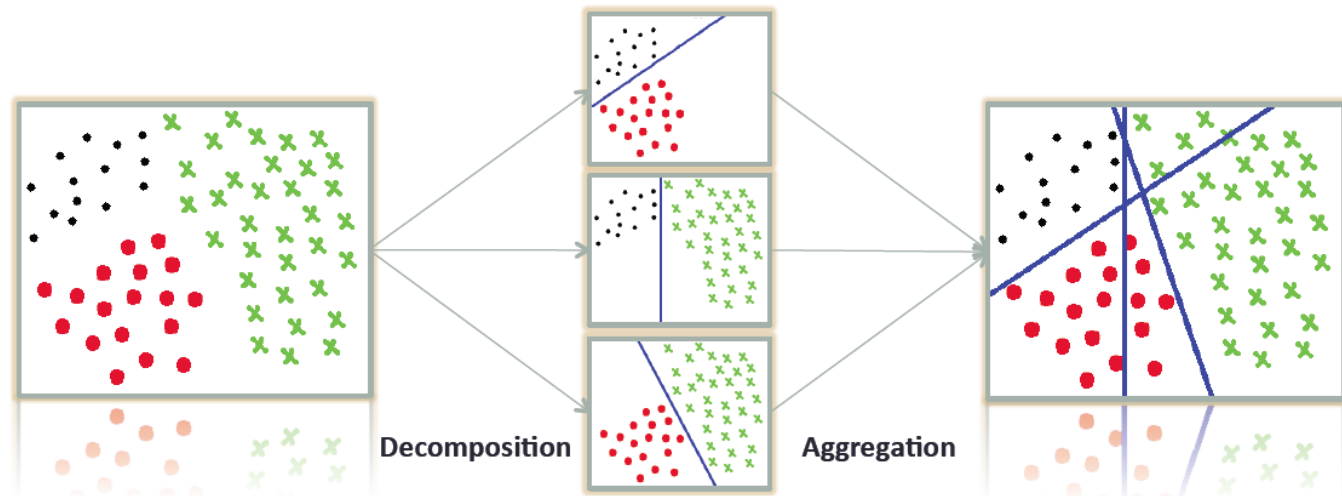
Stacking

- De hecho, es posible entrenar varios mezcladores diferentes de esta forma (por ejemplo, uno usando regresión lineal, otro usando regresión de random forest) para obtener una capa completa de mezcladores, y luego añadir otro mezclador encima para producir la predicción final, como se muestra aquí. Puede que consigas algunas gotas más de rendimiento haciendo esto, pero te costará tanto en tiempo de entrenamiento como en complejidad del sistema.



Descomposición OvO

- Para un problema de clasificación con c clases, se crean $c(c-1)/2$ modelos de clasificación binarios, que corresponden a todas las combinaciones posibles entre pares de clases.
- Cada clasificador binario aprende a distinguir entre las clases i -ésima y j -ésima, por lo que se entrena sólo con instancias de estas clases.
- Para clasificar un patrón, se combinan las respuestas de todos los clasificadores construidos $c(c-1)/2$.
- Clasificación con OvO para tres clases:



Descomposición OvO

- Sea $r_{i,j}$ la respuesta de un clasificador entrenado para discriminar las clases ω_i y ω_j , tal que $r_{i,j} \in [-\infty, \infty]$ y $r_{j,i} = -r_{i,j}$.
- La regla de decisión es:

$$\text{Decidir} \begin{cases} \omega_i & \text{si } r_{i,j} > 0 \\ \omega_j & \text{otro caso} \end{cases}$$

- Sea R la matriz de respuesta de todos los clasificadores creados por la descomposición OvO para un patrón arbitrario:

$$R = \begin{pmatrix} 0 & r_{1,2} & \cdots & r_{1,j} & \cdots & r_{1,c} \\ r_{2,1} & 0 & \cdots & r_{2,j} & \cdots & r_{2,c} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ r_{i,1} & r_{i,2} & \cdots & 0 & \cdots & r_{i,c} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{c,1} & r_{c,2} & \cdots & r_{c,j} & \cdots & 0 \end{pmatrix}$$

- La predicción de clase se realiza utilizando la información de la matriz R aplicando una estrategia de agregación.

Descomposición OvO-MV

- En la estrategia de agregación "voto mayoritario" (VM) la clase ganadora es la que tiene el mayor número de respuestas positivas y se expresa como:

$$\omega_i = \arg \max_{i=1,\dots,c} \sum_{1 \leq j \leq c} v_{i,j}, \quad \text{para } i \neq j$$

- donde

$$v_{i,j} = \begin{cases} 1 & \text{si } r_{i,j} > 0 \\ 0 & \text{otro caso} \end{cases}$$

- Ventaja: es una estrategia sencilla con resultados competitivos.
- Desventaja: es posible que las zonas en el espacio de características provoquen empates en el número de votos.

Descomposición OvO-WV

- En la estrategia de agregación "voto ponderado" (WV), la clase ganadora es aquella cuya suma de respuestas es la mayor en magnitud y se expresa como:

$$\omega_i = \arg \max_{i=1,\dots,c} \sum_{1 \leq j \leq c} r_{i,j}, \quad \text{para } i \neq j$$

- Ventaja: es poco probable que haya zonas en el espacio de características que produzcan empates con la misma suma de respuestas.
- Desventaja: es posible que la clase con la mayor suma de respuestas no sea la verdadera clase.

OvO: Ejemplo de predicción

Classify x , whose real class is c_1

$$R(x) = \begin{pmatrix} & c1 & c2 & c3 & c4 & c5 \backslash \\ c1 & - & 0,55 & 0,6 & 0,75 & 0,7 \\ c2 & 0,45 & - & 0,4 & 1 & 0,8 \\ c3 & 0,4 & 0,6 & - & 0,5 & 0,4 \\ c4 & 0,25 & 0,0 & 0,5 & - & 0,1 \\ c5 & 0,30 & 0,2 & 0,6 & 0,9 & - \end{pmatrix} /$$

OvO: Ejemplo de predicción

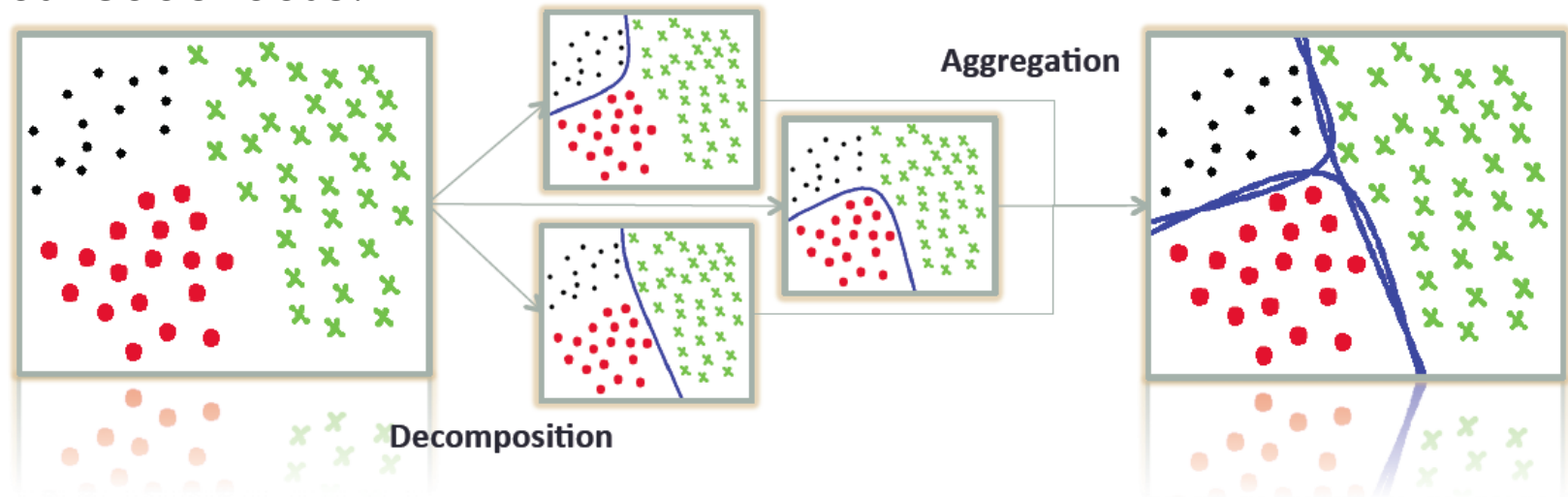
➡ Cuidado con los clasificadores **no competentes**:

- Consider WV aggregation, c_2 is predicted
- **None** of the classifiers **considering c_1 failed**
- **Non-competent** classifiers **strongly voted for c_2**

$$\square R(x) = \begin{pmatrix} & c1 & c2 & c3 & c4 & c5 & WV \\ c1 & - & 0,55 & 0,6 & 0,75 & 0,7 & 2,6 \\ c2 & 0,45 & - & 0,4 & 1 & 0,8 & \mathbf{2,65} \\ c3 & 0,4 & 0,6 & - & 0,5 & 0,4 & 1,9 \\ c4 & 0,25 & 0,0 & 0,5 & - & 0,1 & 0,85 \\ c5 & 0,30 & 0,2 & 0,6 & 0,9 & - & 2,1 \end{pmatrix}$$

Descomposición OvA/OvR

- Para un problema de clasificación con c clases, se crean c modelos de clasificación binaria, donde cada modelo aprende a distinguir una clase determinada del resto.
- El vector de respuesta es $\mathbf{r}=[r_1, \dots, r_c]$, tal que $r_i \in [-\infty, \infty]$ y $r_i > 0$ para los simples de clase ω_i .
- La principal desventaja de esta descomposición es el elevado desequilibrio en el número de muestras de entrenamiento para cada clasificador base.

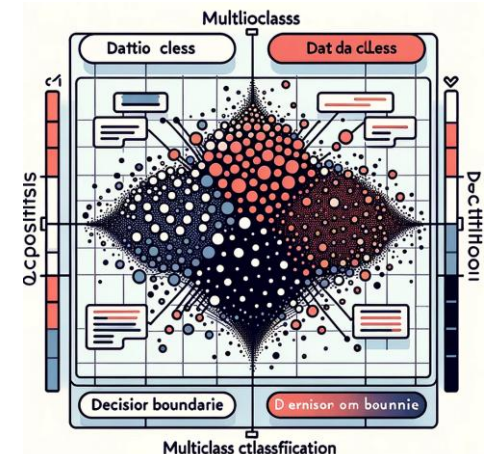


Descomposición OvA-MAX

- La estrategia de agregación más común se denomina "confianza máxima" (MAX), en la que la clase ganadora es la que tiene la máxima respuesta positiva:

$$\omega_i = \arg \max_{i=1, \dots, c} r_i$$

- Ventaja: es poco probable que se produzcan empates en la respuesta positiva máxima.
- Inconveniente: tomar el valor máximo no garantiza una clasificación correcta. Puede ocurrir que no haya respuestas positivas, aunque se tome la respuesta negativa más cercana a cero.



Clasificación no balanceada: Los clasificadores estandar tienden a sobreaprender la clase mayoritaria, ignorando la minoritaria.

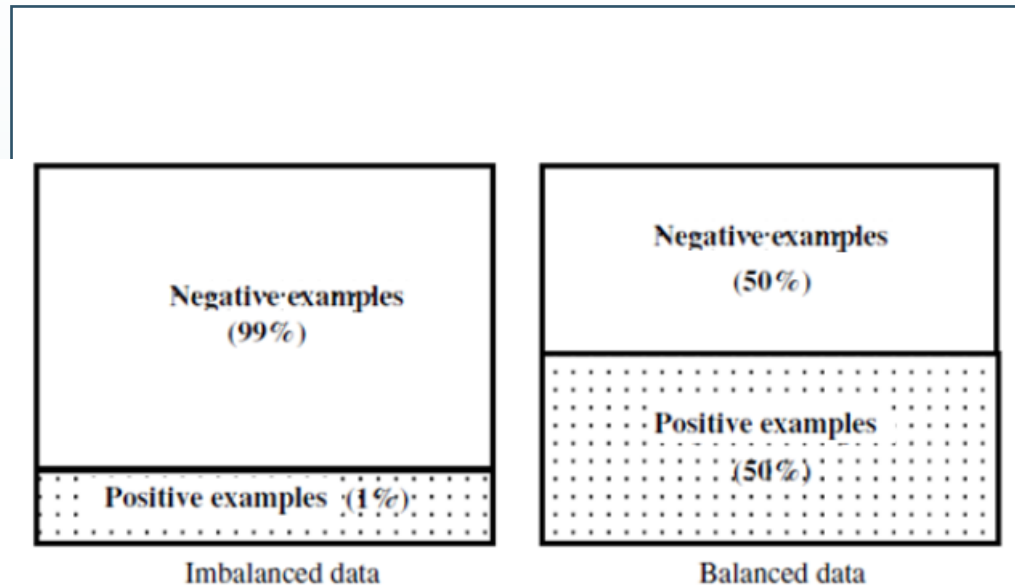


Fig. 1. Imbalanced and balanced data sets.

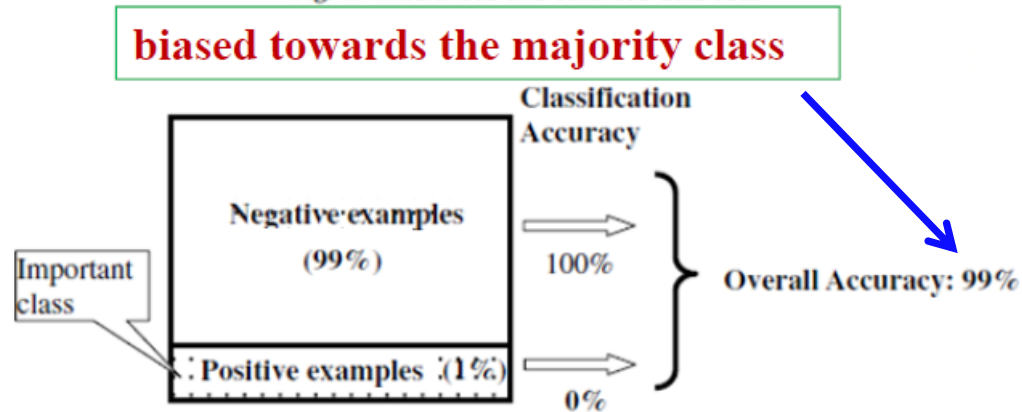


Fig. 2. The illustration of class imbalance problems.

Clasificación
no
equilibrada
(no
balanceada)

¡Necesitamos cambiar la forma de evaluar el comportamiento de los algoritmos y modificar éstos!

Case of study: MNIST

Handwriting recognition (60.000 training, 10.000 test)
Assign a digit from 0 to 9.



Case study: Data-augmentation for CNNs using MNIST

- Objective: Analyze the benefit of data-augmentation and ensembles on CNNs

- Methodology:

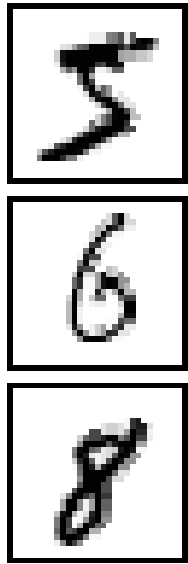
- MNIST (60.000 train + 10.000 test) & 10 classes
 - Three CNNs: LeNet, Network3, Dropconnect

- Results

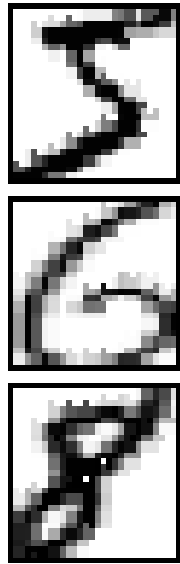


Data augmentation

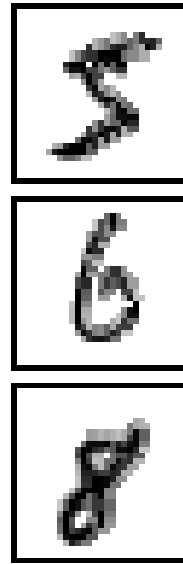
- ❑ Increase the training dataset volume artificially using transformations (tackle the large amount of data limitations)
- ❑ Objective: Improve model robustness



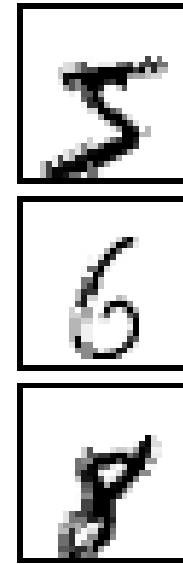
Original
rotation etc



Scaling



Centering



translation

Case study: Data-augmentation for CNNs using MNIST

Lenet-5 like CNNs:
LeNet,
Network3, DropConnect

Dataset	Combination	# of training instances
1	Original	60,000
2	Centering	60,000
3	Elastic	300,000
4	Translation	300,000
5	Rotation	300,000
6	Elastic-centering	300,000
7	Rotation-centering	300,000
8	Translation-elastic	1,500,000
9	Translation-rotation	1,500,000
10	Rotation-elastic	1,500,000
11	Rotation-elastic-centering	1,500,000
12	Elastic-elastic	1,500,000

Data-augmentation techniques

Case study: Data-augmentation for CNNs using MNIST

Test-set accuracies

Dataset	LeNet (10,000 iter.)				LeNet (50,000 iter.)			
	Average	Best	Epochs	Time(s)	Average	Best	Epochs	Time(s)
Original	99.08%	99.18%	10.67	267.91	99.05%	99.21%	213.33	1070.29
Centered	98.85%	99.06%	10.67	203.52	98.95 %	98.09%	213.33	926.38
Elastic	99.09%	99.19%	2.13	232.75	99.36%	99.44%	42.67	1065.38
Translation	99.09%	99.32%	2.13	268.75	99.30%	99.41%	42.67	1065.38
Rotation	99.05%	99.10%	2.13	268.03	99.25%	99.37%	42.67	1065.38
Elastic-centered	⁵ 99.17%	99.26%	2.13	267.20	99.27%	99.36%	42.67	925.51
Rotation-centered	98.90%	99.07%	2.13	232.73	99.19%	99.33%	42.67	950.38
Translation-elastic	⁴ 99.18%	99.32%	0.43	267.43	⁵ 99.39%	99.54%	8.53	1050.38
Translation-rotation	99.16%	99.40%	0.43	267.41	³ 99.40%	97.55%	8.53	1045.38
Rotation-elastic	¹ 99.31%	99.39%	0.43	268.14	¹ 99.47%	99.57%	8.53	1046.25
Rotation-elastic-centered	³ 99.19%	99.24%	0.43	232.30	² 99.43%	99.52%	8.53	925.68
Elastic-elastic	² 99.27%	99.45%	0.43	268.10	⁴ 99.40%	99.50%	8.53	1047.64

Case study: Data-augmentation for CNNs using MNIST

Test-set accuracies

	Network3(10 epochs)			Network3(20 epochs)		
Dataset	Average	Best	Time(s)	Average	Best	Time(s)
Original	99.01%	99.07%	124.45	99.25%	99.25%	205,21
Centered	98.73%	98.80%	118.32	98.97%	99.01%	196.92
Elastic	99.49%	99.54%	656,85	³ 99.61%	99.67%	1200,33
Translation	⁵ 99.49%	99.55%	631.53	⁴ 99.59%	99.63%	1228,71
Rotation	99.44%	99.50%	636.25	99.44%	99.50%	1256,95
Elastic-centered	99.32%	99.39%	566.44	99.57%	99.60%	1109,43
Rotation-centered	98.88%	98.94%	569.04	99.31%	99.32%	1167,32
Translation-elastic	⁴ 99.54%	99.57%	3647.78	⁵ 99.58%	99.63%	7111,65
Translation-rotation	³ 99.57%	99.61%	3650.66	99.58%	99.60%	7149,25
Rotation-elastic	² 99.62%	99.67%	3642,85	² 99.67%	99.69%	6996,23
Rotation-elastic-centered	99.43%	99.51%	3054,43	99.51%	99.52%	6908,70
Elastic-elastic	¹ 99.65%	99.66%	3607.32	¹ 99.67%	99.70%	7189,16

Case study: Data-augmentation for CNNs using MNIST

Test-set accuracies

	DropConnet(100 epochs)			DropConnet(200 epochs)		
Dataset	Average	Best	Time(s)	Average	Best	Time(s)
Original	98,32%	98,83%	7803.43	98.98%	98,99%	18748.53
Centered	95.35%	94,46%	6659.31	95.13%	98,85%	18635.54
Elastic	99.33 %	99,35%	7512.25	99.36%	99,36%	18606.15
Translation	⁵ 99.43%	99,46%	7736.41	⁵ 99.47%	99,47%	18710.45
Rotation	99.18%	99,29%	7151.73	99.37%	99,47%	18729.29
Elastic-centered	96.58%	96,69%	6969.89	97.08%	97,09%	18661.80
Rotation-centered	98.30%	98,41%	6974.23	98.55%	98,63%	18668.05
Translation-elastic	99.40%	99,57%	7162.37	³ 99.58%	99,58%	18745.93
Translation-rotation	² 99.57%	99,59%	7410.32	¹ 99.69%	99,69%	18772.40
Rotation-elastic	³ 99.54%	99,60%	7397.40	⁴ 99.56%	99,56%	18724.38
Rotation-elastic-centered	⁴ 99.47%	99,49%	7803.73	99,44%	99,46%	18220.50
Elastic-elastic	¹ 99,58%	99,59%	7911.30	² 99,59%	99,61%	18712.22

Case study: Data-augmentation for CNNs using MNIST

Results

	LeNet(500 neurons)		Network3		DropConnect	
	10,000 iter	50,000 iter	10 epochs	20 epochs	100 epochs	200 epochs
Ensemble-5	99,55%	99,57%	99,72%	99,69%	99,72%	99,66%
Ensemble-3	99,43%	99,54%	99,69%	99,67%	99,69%	99,68%

Error : 0.28% versus state of the art ensemble 0.16%

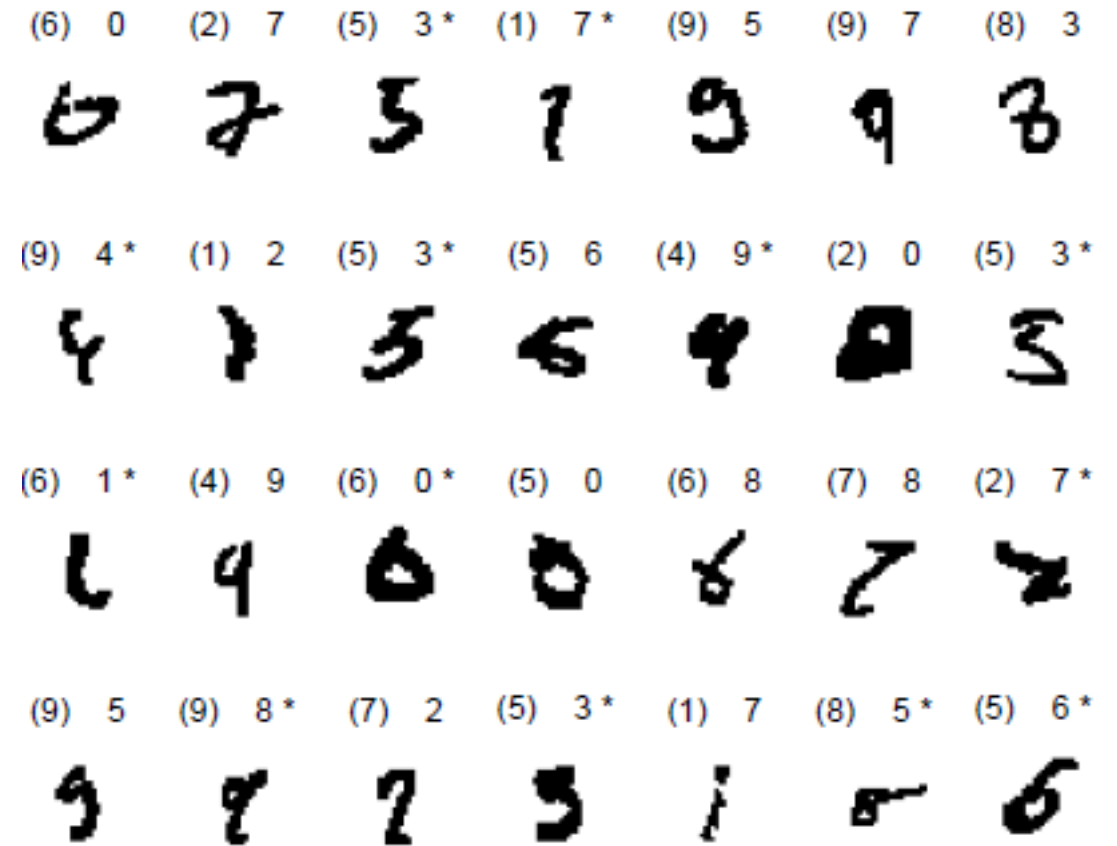
Case study: Data-augmentation for CNNs using MNIST

Results

The 28 misclassified characters (15 different)



ensemble-5 (Network3)



ensemble-5 (DropConnect)

Case study: Data-augmentation for CNNs using MNIST

The 13
handwritten digits
misclassified by
ensemble-5 of
DropConnet
and Network3

5, Net(3), DC(3)



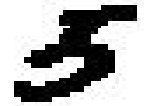
1, Net(7), DC(7)



9, Net(4), DC(4)



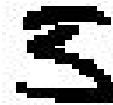
5, Net(3), DC(3)



4, Net(9), DC(9)



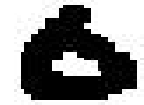
5, Net(3), DC(3)



6, Net(1), DC(1)



6, Net(0), DC(0)



2, Net(7), DC(7)



9, Net(4), DC(8)



5, Net(3), DC(3)



8, Net(5), DC(5)



5, Net(6), DC(6)



Case study: Data-augmentation for CNNs using MNIST



Deep Learning: MNIST
data (10.000 test)
Ensemble with
different top CNN
models

The digit
between ()
represents the
correct class.

May 2019, Granada team (12 errors). World RECORD

Case study: Data-augmentation for CNNs using MNIST



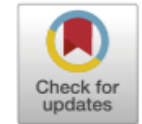
Contents lists available at [ScienceDirect](#)

Information Fusion

journal homepage: www.elsevier.com/locate/infus



MNIST-NET10: A heterogeneous deep networks fusion based on the degree of certainty to reach 0.1% error rate. Ensembles overview and proposal



Siham Tabik^{a,*}, Ricardo F. Alvear-Sandoval^b, María M. Ruiz^a, José-Luis Sancho-Gómez^c, Aníbal R. Figueiras-Vidal^b, Francisco Herrera^a

^a Andalusian Research Institute in Data Science and Computational Intelligence, University of Granada, 18071, Spain

^b GAMMA-L + /DTSC, Universidad Carlos III de Madrid, Spain

^c TDAM/DTIC, Universidad Politécnica de Cartagena, Spain

Case study: Data-augmentation for CNNs using MNIST

Table 6

Test error of fusion schemes, F1, F2, and FS1. Symbol | indicates that the involved models are run in parallel.

Fusion scheme	Fusion structure	Test error in %
F1	CapsNet MCDNN DropConnect_2	0.14
F2	CapsNet MCDNN DropConnect_1 DropConnect_2 Network3	0.14
FS1	F1 F2 Dropconnect_2	0.12

Case study: Data-augmentation for CNNs using MNIST

We propose MNIST-NET10 by aggregating the best ensembles, **FS1** from [Section 4.2](#) and **FS2** from [Section 4.3](#). **F1**, CapsNet | MCDNN | DropConnect_2, and **F2**, CapsNet | MCDNN | DropConnect_1 | DropConnect_2,

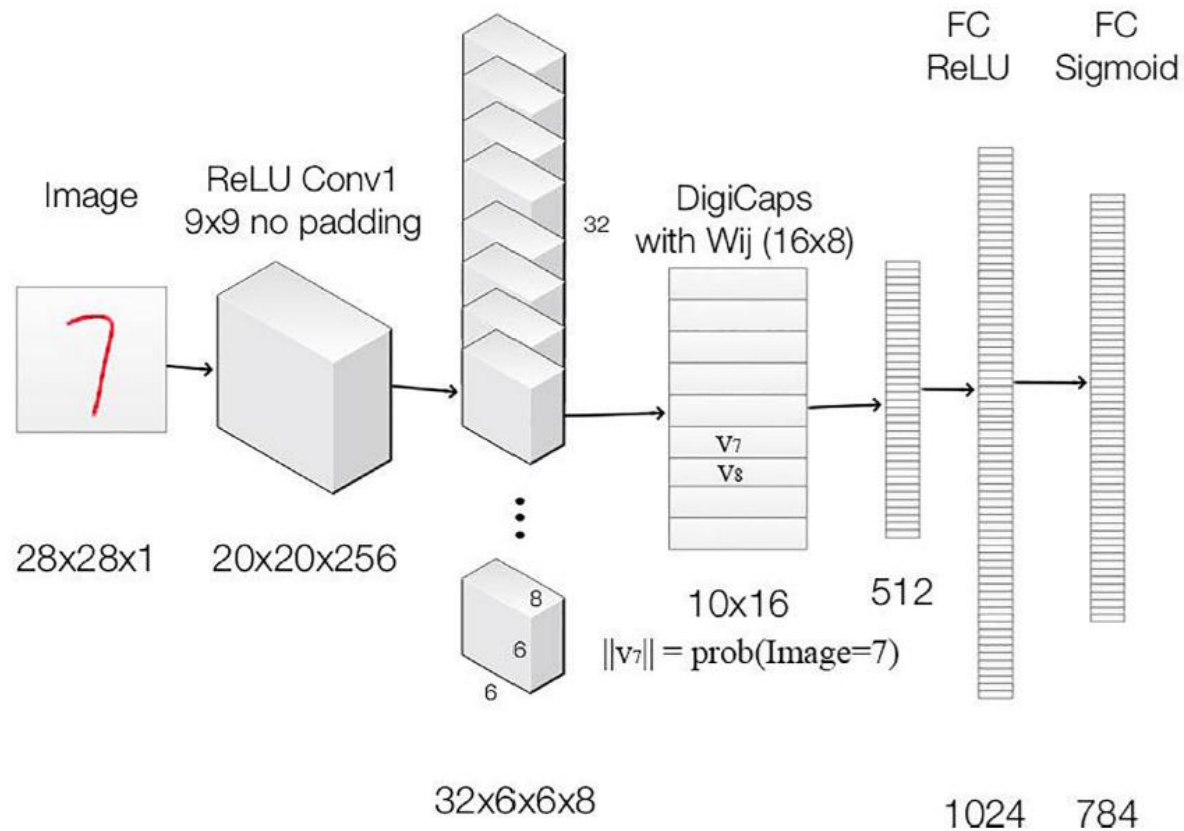
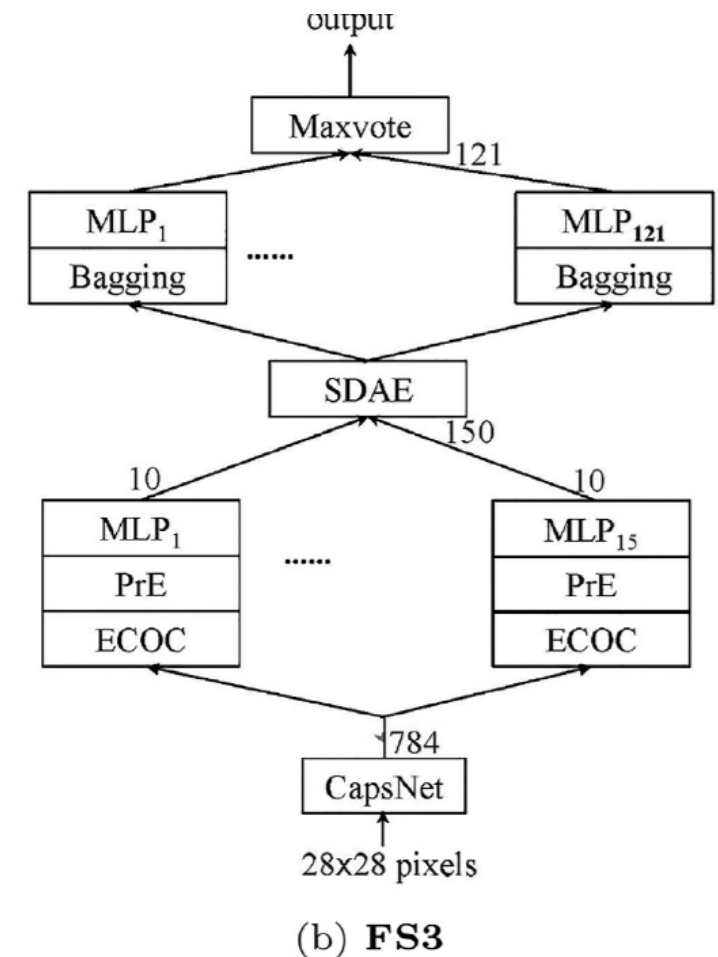


Fig. 1. An illustration of CapsNet architecture.



(b) **FS3**

Case study: Data-augmentation for CNNs using MNIST

We propose MNIST-NET10 by aggregating the best ensembles, **FS1** from [Section 4.2](#) and **FS2** from [Section 4.3](#). **F1**, CapsNet | MCDNN | DropConnect_2, and **F2**, CapsNet | MCDNN | DropConnect_1 | DropConnect_2,

S. Tabik, R.F. Alvear-Sandoval and M.M. Ruiz et al.

Information Fusion 62 (2020) 73–80

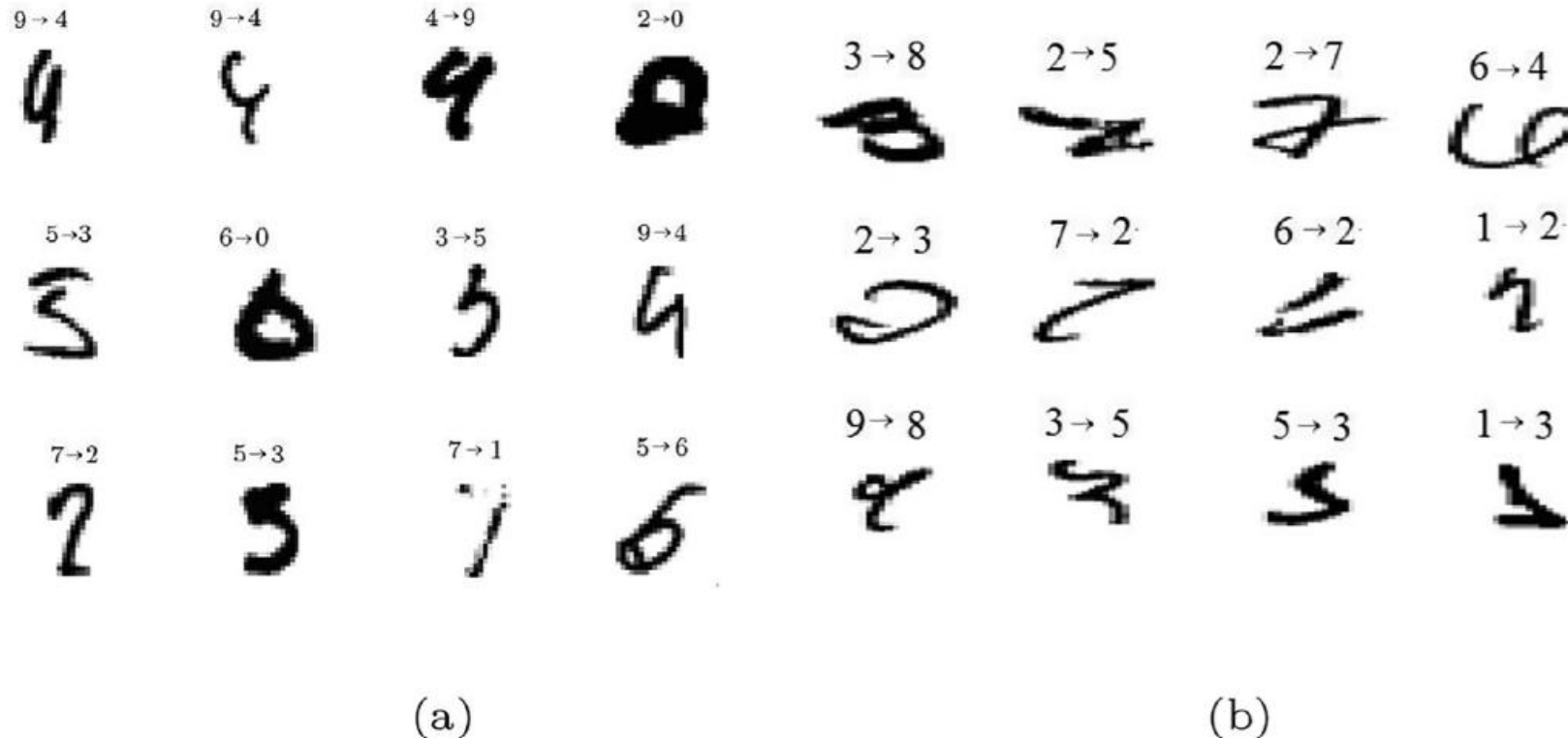


Fig. 3. Misclassified test digits (label → results) by (a) a run of ensemble FS1 ([Section 4.2](#)) and (b) a run of ensemble FS2 with CapsNet input ([Section 4.3](#)).

Case study: Data-augmentation for CNNs using MNIST

We propose MNIST-NET10 by aggregating the best ensembles, **FS1** from [Section 4.2](#) and **FS2** from [Section 4.3](#). **F1**, CapsNet | MCDNN | DropConnect_2, and **F2**, CapsNet | MCDNN | DropConnect_1 | DropConnect_2,

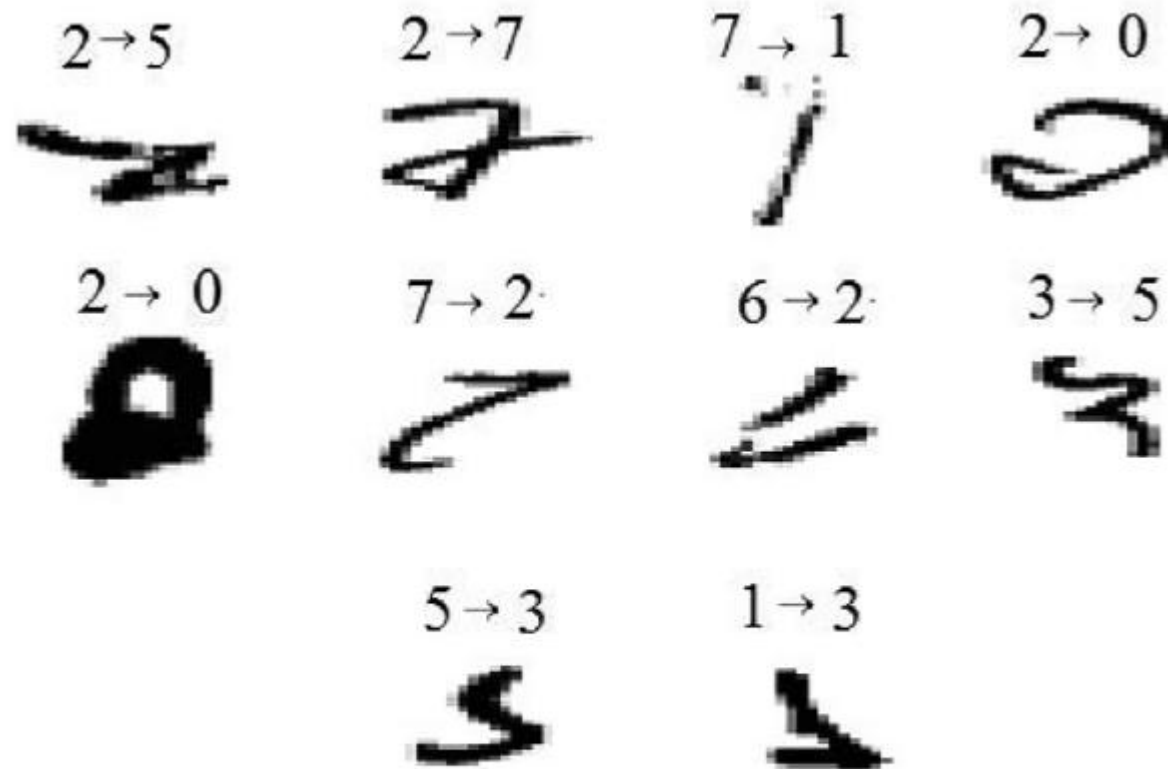


Fig. 4. Misclassified test digits (label → result) obtained from fusing FS1 and FS2.

Aprendizaje Automático

Curso 2025-2026

Tema 2. Aprendizaje Supervisado



Bibliografía

